

Software Engineering

- Evaluation Scheme Theory:
- MSE: 30 Marks, TA: 20 Marks, ESE: 50
- Marks Laboratory: CIE: 50 Marks, ESE: 50 Marks

Software Engineering

- Software Engineering is the systematic process of designing, developing, testing, and maintaining software to create reliable, efficient, and high-quality applications.
- It applies engineering principles, methods, and tools to develop software in a systematic manner.
- It supports teamwork, planning, documentation, testing, and maintenance to ensure successful software development throughout its lifecycle.
- It helps create software that meets user requirements while ensuring reliability,

Need of Software Engineering

- Software engineering is a technique through which we can develop or create software for computer systems or any other electronic devices.
- It is a systematic, scientific and disciplined approach to the development, functioning, and maintenance of software.

Effectiveness

**Handling Big
Projects**

Reliable

Purpose of Software Engineering

Decrease Time

Productivity

**Reduce
Complexity**

Manage the cost

Purpose of Software Engineering

- **Handling Big Projects:** A corporation must use a software engineering methodology in order to handle large projects without any issues.
- **To manage the cost:** Software engineering programmers plan everything and reduce all those things that are not required.
- **To decrease time:** It will save a lot of time if you are developing software using a software engineering technique.
- **Reliable software:** It is the company's responsibility to deliver software products on schedule and to address any defects that may

- **Effectiveness:** Effectiveness results from things being created in accordance with the standards.
- **Reduces complexity:** Large challenges are broken down into smaller ones and solved one at a time in software engineering. Individual solutions are found for each of these issues.
- **Productivity:** Because it contains testing systems at every level, proper care is done to maintain software productivity.

Software Crisis and Myths,

- **What is Software Crisis?**
- **Software Crisis** is a term used in computer science for the difficulty of writing useful and efficient computer programs in the required time.
- The software crisis was due to using the same workforce, same methods, and same tools even though rapidly increasing software demand, the complexity of software, and [software challenges](#).

- Suppose we use the same workforce, same methods, and same tools after the fast increase in software demand, software complexity, and software challenges.
- In that case, there arise some issues like software budget problems, software efficiency problems, software quality problems, software management, and delivery problems, etc. This

Causes of Software Crisis

- The cost of owning and maintaining software was as expensive as developing the software.
- At that time Projects were running overtime.
- At that time Software was very inefficient.
- The quality of the software was low quality.
- Software often did not meet user requirements.
- At that time Software was never delivered

- **Factors Contributing to Software Crisis**

- Factor Contributing to Software Crisis are:
- Poor project management.
- Lack of adequate training in software engineering.
- Less skilled project members.
- Low productivity improvements.

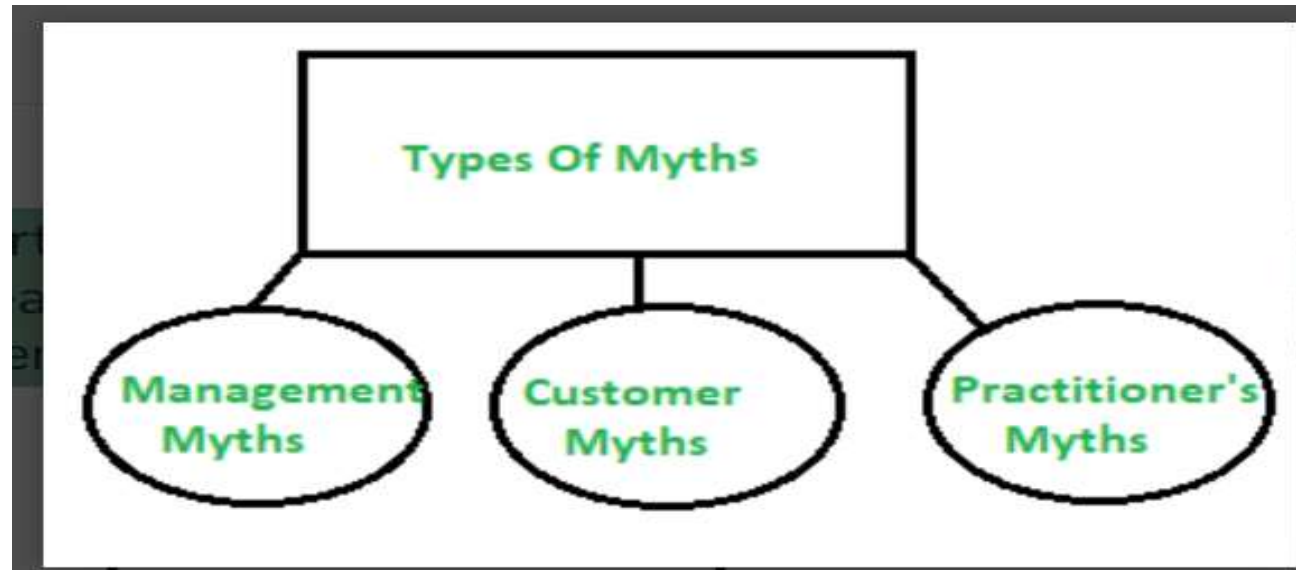
- **Solution of Software Crisis:**

- There is no single solution to the crisis. One possible solution to a software crisis is Software Engineering because software engineering is a systematic, disciplined, and quantifiable approach. For preventing software crises, there are some guidelines:

- Reduction in software over budget.
- The quality of the software must be high.
- Less time is needed for a software project.
- Experienced and skilled people working on the software project.
- Software must be delivered.
- Software must meet user requirements.

Software Myths

- Most, experienced experts have seen myths or superstitions (false beliefs or interpretations) or misleading attitudes (naked users) which creates major problems for management and technical people (leading to poor planning, communication issues, increased costs, delays, and lower software quality).



(i) Management Myths:

- **Myth 1:**

- We have all the standards and procedures available for software development.
- Management may believe that **having standards, guidelines, and procedures alone is enough to ensure a successful software project.**

Fact:-

- Standards and procedures provide only a **framework** for development
- Success also depends on:
 - Skilled and experienced developers
 - Proper project planning
 - Good communication among team members
 - Effective project management
 - Regular testing and quality assurance
 - Continuous monitoring and risk management

- **Myth 2:**

- The addition of the latest hardware programs will improve the software development.
- Fact:-
- New hardware and tools **can help**, but they **do not guarantee** better software.
- The success of software development depends mainly on:

Skilled and experienced developers

Well-defined requirements

Proper planning and project management

Effective communication

Good software engineering practices

- **Myth 3:**

- With the addition of more people and program planners to Software development can help meet project deadlines (If lagging behind)
- Fact:-
- New team members need time to understand the project.
- Existing developers must spend time training and coordinating with them.
- Communication and coordination become more complex as the team grows. As a result, the project may become **even more delayed.**

- **(ii) Customer Myths:**

- The customer can be the direct users of the software, the technical team, marketing / sales department, or other company.

- **Myth 1:**

- A general statement of intent is enough to start writing plans (software development) and details of objectives can be done over time.
- Fact:-
- Software development requires **clear, complete, and detailed requirements** before starting.

- If requirements are incomplete or keep changing:

Developers may misunderstand customer needs.

More time and effort are required to make changes.

Development cost increases.

The project may be delayed.

Software quality may be affected.

- **Myth 2:**

- Software requirements continually change, but change can be easily accommodated because software is flexible.
- Fact:-
- Although software is flexible, **every change affects the project.**
- Requirement changes may require:
 - Modifying the design
 - Changing the code
 - Updating documentation
 - Retesting the software

- As development progresses, the **cost and effort of making changes increase**
- Frequent changes can cause:
 - Project delays
 - Increased development cost
 - More defects (bugs)
 - Reduced software quality

- **iii) Practitioner's Myths:**

- **Myths 1:**

- They believe that their work has been completed with the writing of the plan.
- Fact:-
- software development does not end with coding. After coding, several important activities are still required:
- Testing the software to find and fix bugs.

- Preparing documentation.
- Integrating the software with other modules.
- Deploying the software.
- Maintaining and updating the software after delivery.

- Myths 2:

There is no other way to achieve system quality, until it is "running".

Fact:-

- software quality can be ensured **throughout the development process**, not just after execution.
- Quality can be improved by:
- Requirement reviews

- Design reviews
- Code inspections
- Unit testing
- Integration testing
- Static analysis
- Verification and validation

- **Myth 3:**

- An operating system is the only product that can be successfully exported project.
- Some developers believe that **once the software runs correctly, the project is complete and nothing else needs to be delivered.**

- **Fact: -**

- successful software project includes many deliverables besides the working software, such as:
- Software documentation
- Design documents
- Test cases and test reports
- User manuals
- Installation and maintenance guides
- Training materials (if required)

- **Myth 4:**

- Engineering software will enable us to build powerful and unnecessary document & always delay us.
- Documentation is an important part of software engineering. Good documentation helps in:

- Understanding the system

- Team communication

- Software maintenance

- Future enhancements

- Software engineering promotes **necessary and useful documentation**, not unnecessary paperwork.

Software Quality Attributes,

- Software Quality Attributes are the characteristics that determine how well a software system performs and satisfies user requirements. They define the quality, reliability, efficiency, and usability of software.
- Definition
- Software Quality Attributes are measurable properties that describe the quality of a software system and determine whether it meets

- Major Software Quality Attributes

1. Correctness

- The software performs the required functions correctly.
- It satisfies all specified user requirements.

2. Reliability

- The software works consistently without failures.
- It performs accurately under specified conditions for a given period.

3. Efficiency

- The software uses system resources (CPU, memory, storage, network) effectively.
- It provides fast response and good performance.

4. Integrity (Security)

- Protects data from unauthorized access or modification.
- Ensures confidentiality and data security

5. Usability

- The software is easy to learn, understand, and use.
- It provides a user-friendly interface

6. Maintainability

- The software can be easily modified, corrected, or upgraded.
- Makes fixing bugs and adding new features easier.

7. Flexibility

- The software can be adapted to changing user requirements.
- New features can be added with minimal effort.

8. Testability

- The software can be easily tested to detect defects.
- Supports effective verification and validation.

9. Portability

- The software can run on different operating systems or hardware platforms with little modification.

10. Reusability

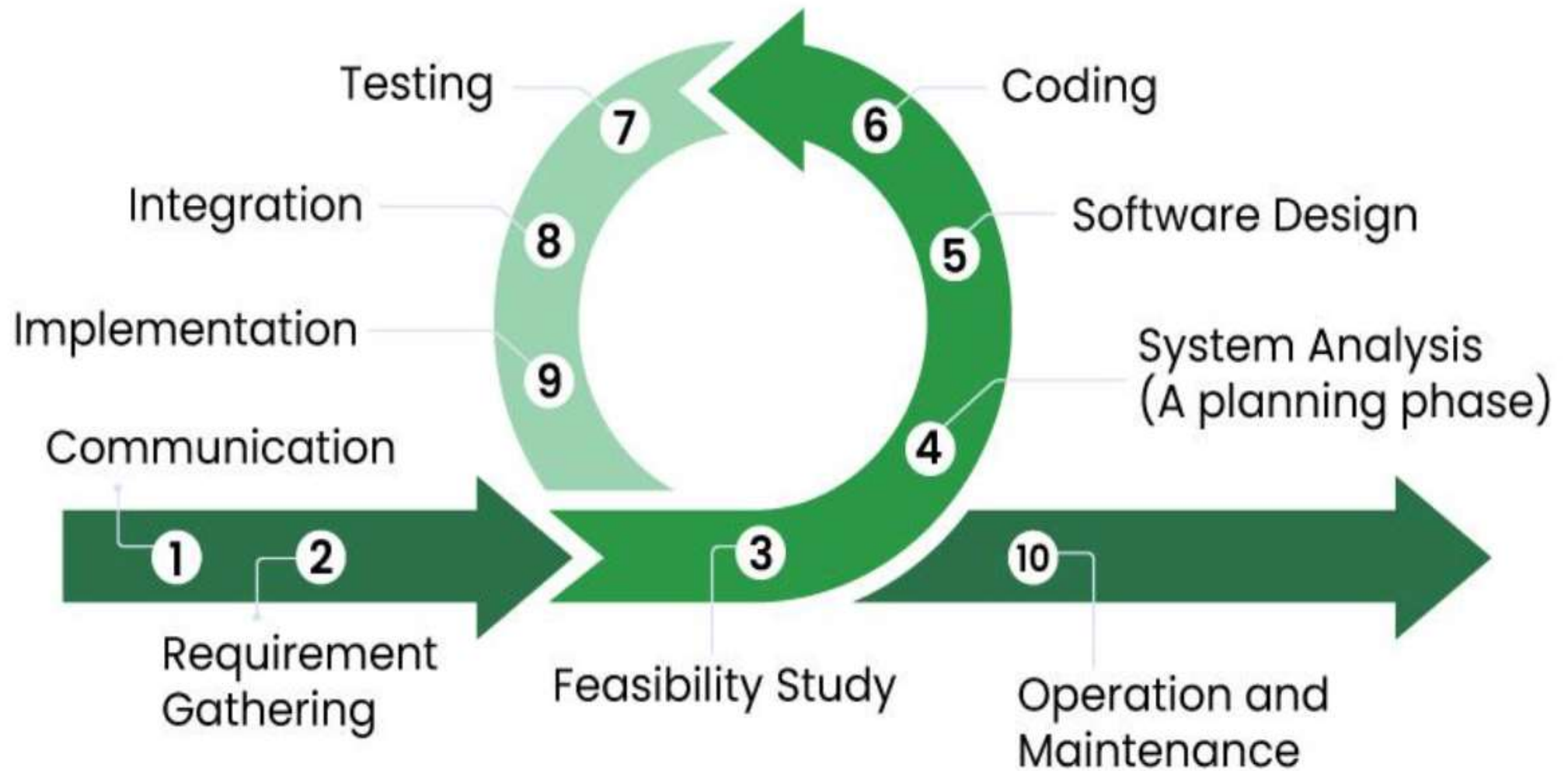
- Software components can be reused in other applications.
- Reduces development time and cost.

11. Interoperability

- The software can communicate and exchange data with other systems.

Software Process and development

- **What is Software Development Process?**
- The software development process is the approach to developing, and delivering software applications. This process might include improving design and product management by splitting the work into smaller steps or processes.



Software development process

1. Communication

- Developers meet with the client or stakeholders.
- They discuss the problem and understand what the software should do.
- **Goal:** Understand the client's needs.

2. Requirement Gathering

- Collect all functional and non-functional requirements.
- Document them clearly.
- **Output:** Software Requirements Specification (SRS) .

3. Feasibility Study

- Analyse whether the project is possible.
- Check:
 - **Technical feasibility:** Can it be built?
 - **Economic feasibility:** Is it within budget?
 - **Operational feasibility:** Will users accept it?
 - **Time feasibility:** Can it be completed on time?
- **Goal:** Decide whether to proceed.

4. System Analysis (Planning Phase)

- Study the requirements in detail.
- Break the project into modules.
- Plan resources, schedule, risks, and architecture.
- **Goal:** Create a clear development plan.

5. Software Design

- Design how the software will work.
- Create:
 - Database design
 - User interface (UI)
 - System architecture
 - Algorithms
- **Output:** Design documents.

6. Coding

- Developers write the actual program using programming languages such as Java, Python, C++, or JavaScript.
- **Goal:** Convert the design into working software.

7. Testing

- Testing is the process of checking whether the software works correctly and meets the user's requirements.
- It helps identify and fix errors (bugs) before the software is delivered to users.

- Common testing types:
 - Unit Testing
 - Integration Testing
 - System Testing

8. Integration

Integration is the process of combining all the individual software modules (frontend, backend, database, APIs, and libraries) into one complete system so they work together correctly.

9. Implementation (Deployment)

Implementation is the stage where the completed software is installed and made available for users to use in the real environment.

10. Operation and Maintenance

Operation and Maintenance is the final phase of the Software Development Process (SDP). In this phase, the software is used by end users, monitored for performance, and regularly updated to fix bugs, improve features, and adapt to changing technologies or user requirements.

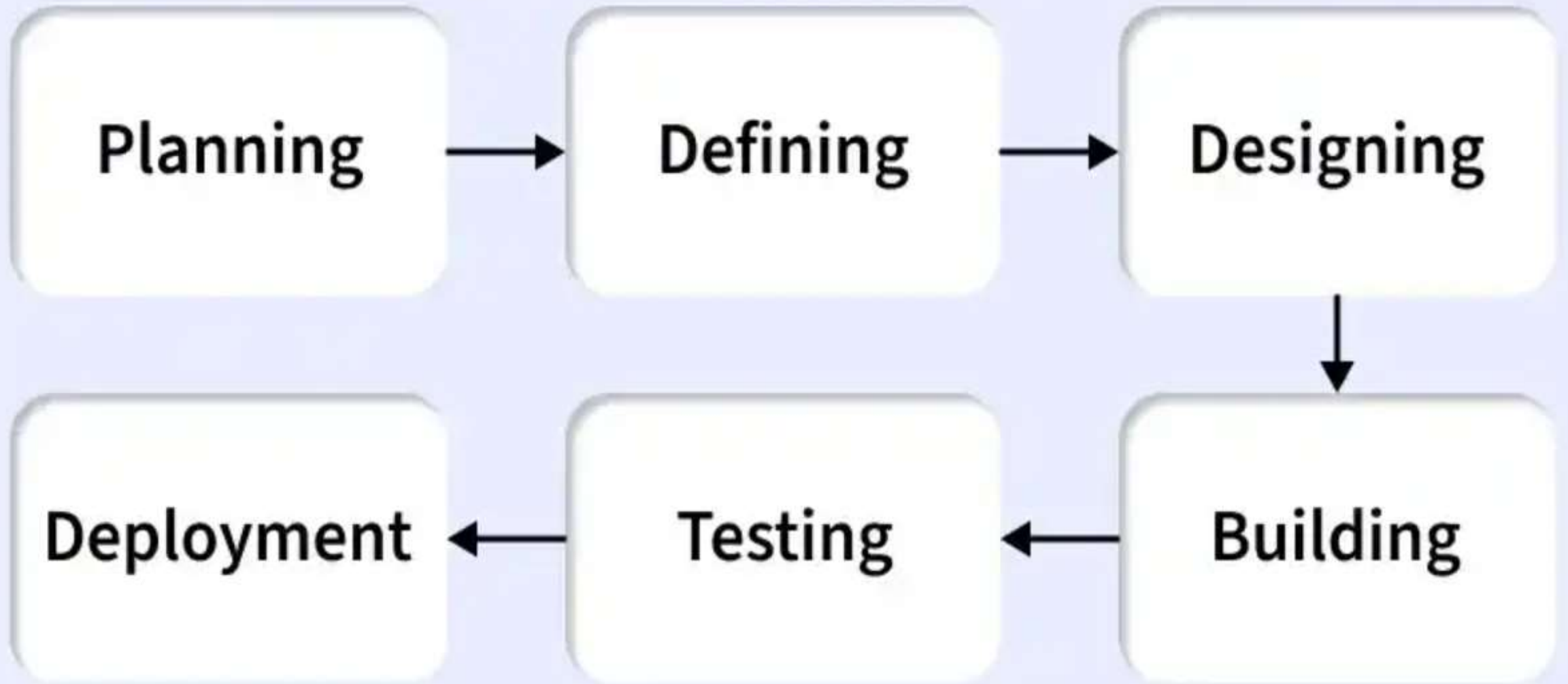
- **Activities in Operation and Maintenance**

- Monitor software performance.
- Fix bugs and errors.
- Provide user support and documentation.
- Update the software with new features.
- Improve performance and security.
- Ensure compatibility with new technologies and operating systems.

Software life cycle and Models

- The **Software Development Life Cycle (SDLC)** is a structured process used to **plan, design, develop, test, deploy, and maintain software**. It provides a systematic approach to building software that meets business goals and user requirements.
- **Definition**
- **Software Development Life Cycle (SDLC)** is a step-by-step process followed by software developers to create high-quality software efficiently and systematically

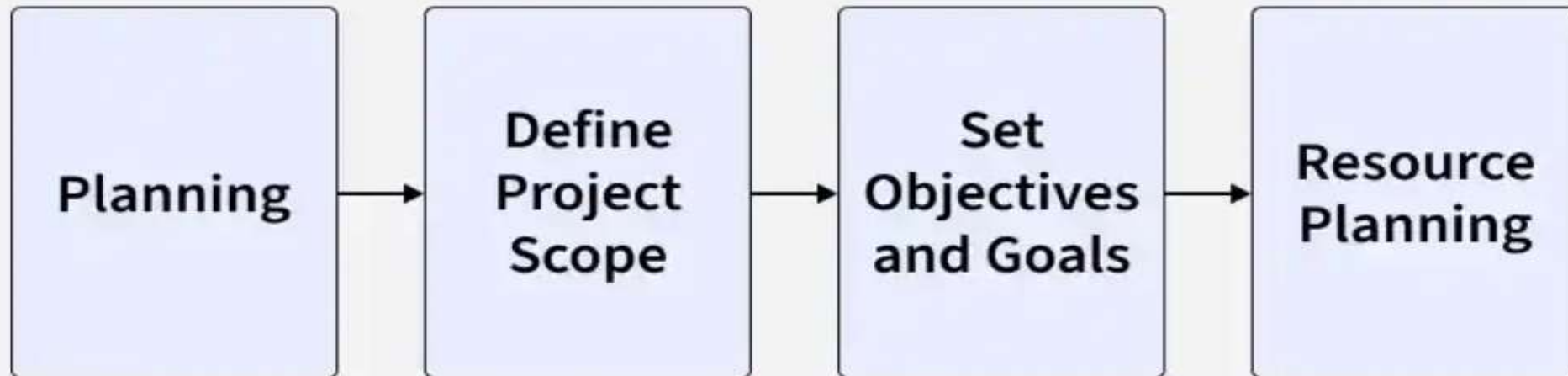
Software Development Life Cycle



Stage 1: Planning & Feasibility Analysis

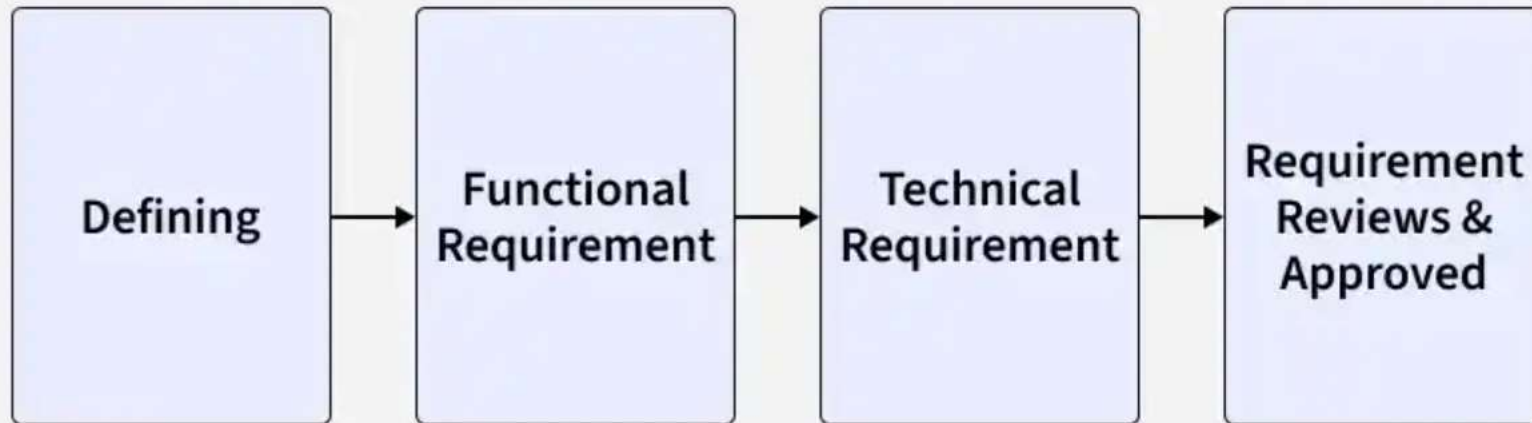
- This is the **first stage of the Software Development Life Cycle (SDLC)**. In this stage, the organization decides whether the proposed software project is practical and worth developing.
- **Definition**
- **Planning and Feasibility Analysis** is the process of identifying project goals, estimating resources, cost, time, and analyzing whether the project is **technically, financially, and operationally feasible**.

Stage-1: Planning and Requirement Analysis



- **Stage 2: Requirement Specification (SRS)**
- This is the **second stage of the Software Development Life Cycle (SDLC)**. In this stage, all the software requirements are collected, analyzed, documented, and approved by the stakeholders.
- **Definition**
- **Requirement Specification (SRS)** is the process of identifying, documenting, and validating all **functional and non-functional requirements** of the software before development begins.

Stage-2: Defining Requirements



Stage 3: System Design

- The **System Design** phase is the **third stage of the Software Development Life Cycle (SDLC)**. In this stage, the approved requirements from the **SRS document** are converted into a **technical blueprint** for software development.
- Definition:-
- **System Design** is the process of creating the architecture and detailed design of the software based on the approved requirements. It defines how the system will be built.

Stage-3: Designing Architecture



1. High-Level Design (HLD)

- High-Level Design describes the overall structure of the system.
- **It includes:**
 - System architecture
 - Technology stack (programming languages, frameworks, tools)
 - Database design
 - Major modules and their interactions

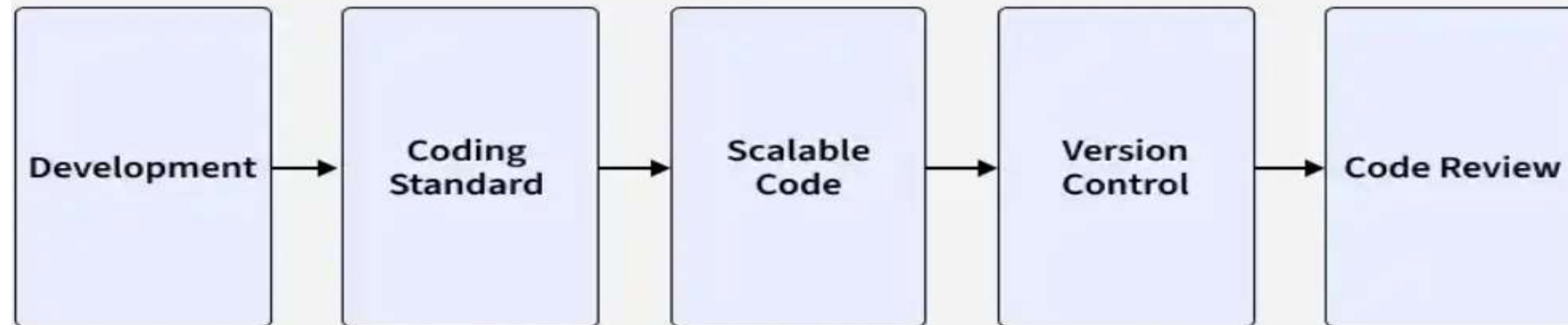
2. Low-Level Design (LLD)

- Low-Level Design provides detailed information about each module.
- **It includes:**
 - Component logic
 - APIs (Application Programming Interfaces)
 - Data structures
 - Algorithms
 - Workflows and process flow
 - Class and sequence diagrams

Stage 4: Development (Coding)

- The **Development (Coding)** phase is the **fourth stage of the Software Development Life Cycle (SDLC)**. In this stage, developers write the actual program code based on the **Design Document Specification (DDS)** created during the design phase.
- **Definition**
- **Development (Coding)** is the process of converting the software design into executable source code using programming languages and development tools.

Stage-4: Developing Product



Stage 5: Testing

- The **Testing** phase is the **fifth stage of the Software Development Life Cycle (SDLC)**. In this stage, the software is tested to ensure it **meets the specified requirements and is free from defects (bugs)** before being released to users.

Definition

- **Testing** is the process of verifying and validating the software to ensure it works correctly, meets user requirements, and

Types of Testing include:

- **Unit Testing:** Verifies individual components
- **Integration Testing:** Ensures modules work together
- **System Testing:** Validates the complete system
- **User Acceptance Testing (UAT):** Confirms business requirements are met.

Stage-5: Product Testing and Integration

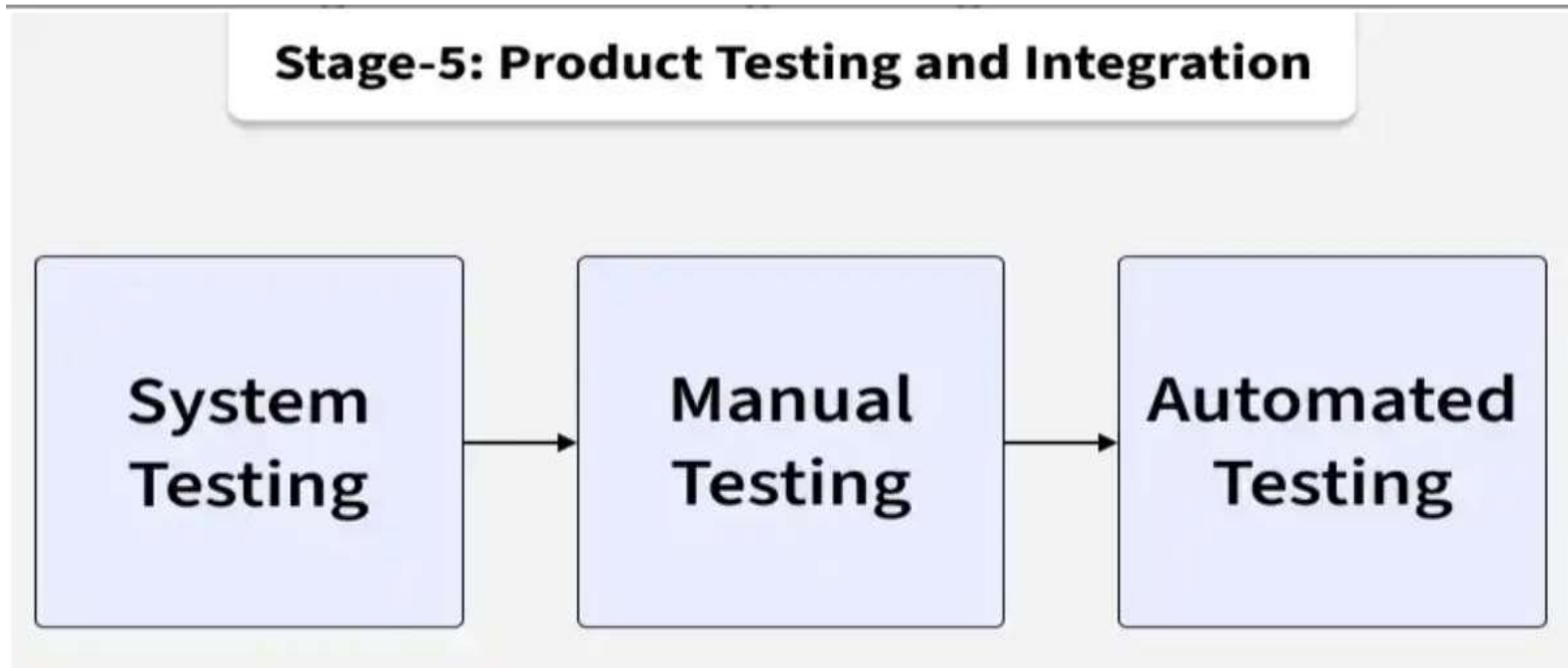
**System
Testing**



**Manual
Testing**



**Automated
Testing**



Stage 6: Deployment

- The **Deployment** phase is the **sixth stage of the Software Development Life Cycle (SDLC)**. In this stage, the **tested software is released to the production environment** so that end users can use it.
- **Definition**
- **Deployment** is the process of installing, configuring, and releasing the software into the production environment for users.

Stage 7: Maintenance

- The **Maintenance** phase is the **final stage of the Software Development Life Cycle (SDLC)**. After the software is deployed, it is continuously monitored, updated, and improved to ensure it remains reliable, secure, and useful for users.

- **Importance of SDLC**

- Provides a clear and organized development framework.
- Improves planning, cost control, and project management.
- Ensures better quality through defined testing phases.
- Helps deliver software that meets user and business needs

Example: Online Library Management System

- **1. Planning & Feasibility Analysis**
- The college decides to develop an **Online Library Management System**.
- The team checks whether the project is technically, financially, and operationally feasible.
- **Output:** Project plan and feasibility report.

2. Requirement Specification (SRS)

- Requirements are collected from the librarian, students, and college management.
- **Functional Requirements:**
 - User login
 - Search books
 - Issue and return books
 - Fine calculation
- **Non-Functional Requirements:**
 - Fast response time
 - Secure login
 - Easy-to-use interface

3. System Design

- The system architecture, database, and modules are designed.
- **HLD**: User Management, Book Management, Issue/Return, Reports.
- **LLD**: Login logic, database tables, APIs, workflows.

4. Development (Coding)

- Developers write the source code.
- Frontend developers create the user interface.
- Backend developers develop the database and APIs.

5. Testing

- The software is tested for defects.
- **Unit Testing:** Test the login module.
- **Integration Testing:** Check communication between modules.
- **System Testing:** Test the complete library system.
- **UAT:** Librarians and students verify the software.

6. Deployment

- The application is installed on the college server.
- Smoke testing ensures the main features work correctly.

7. Maintenance

- Bugs are fixed after deployment.
- New features such as **online book reservation** are added.
- Security updates and performance improvements are provided.

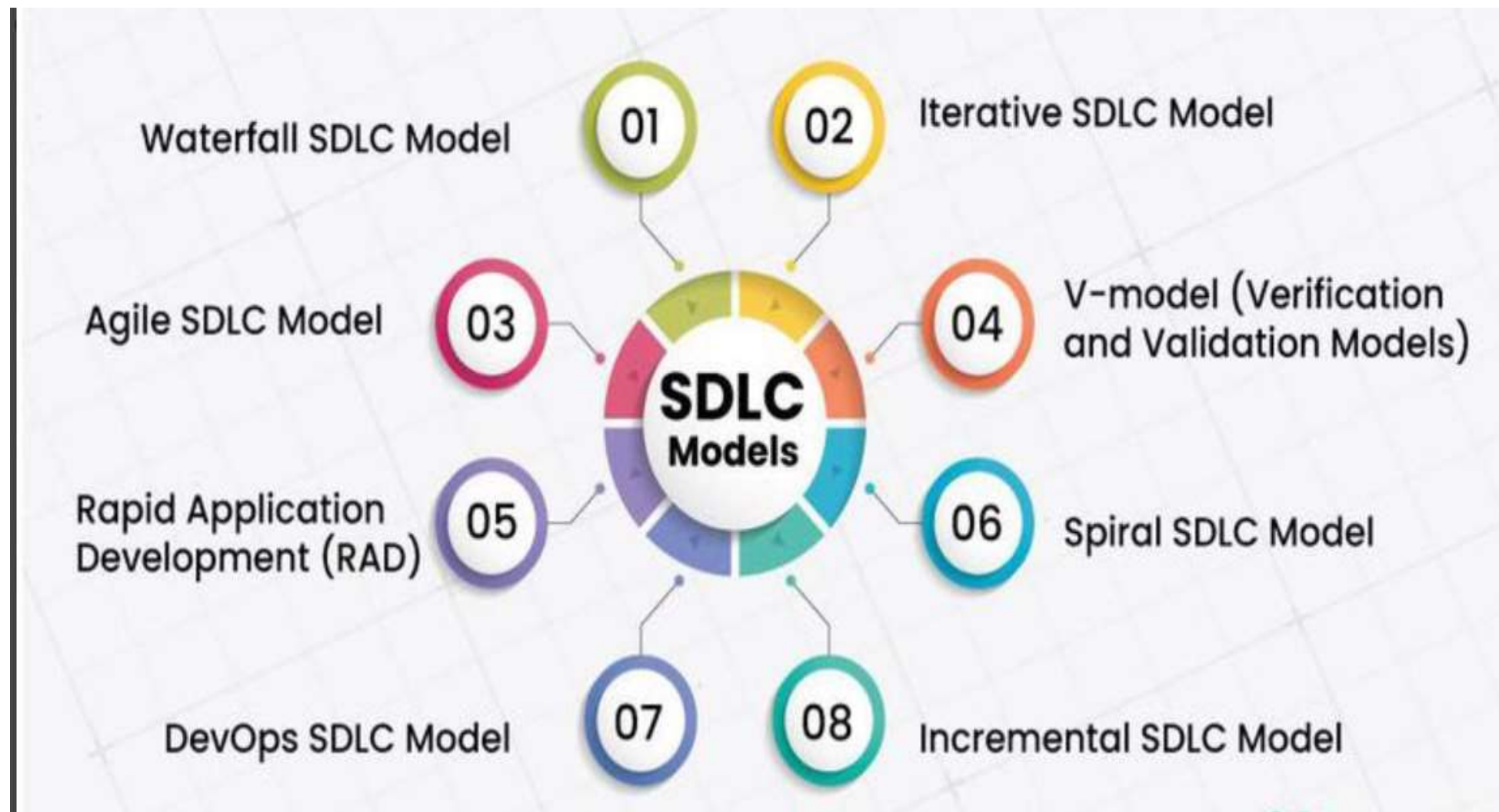
Common Models

- Waterfall Model
- Agile Model
- V-Model
- Spiral Model
- Incremental Model
- RAD Model

Software Development Models - SDLC Models

- **Definition:**

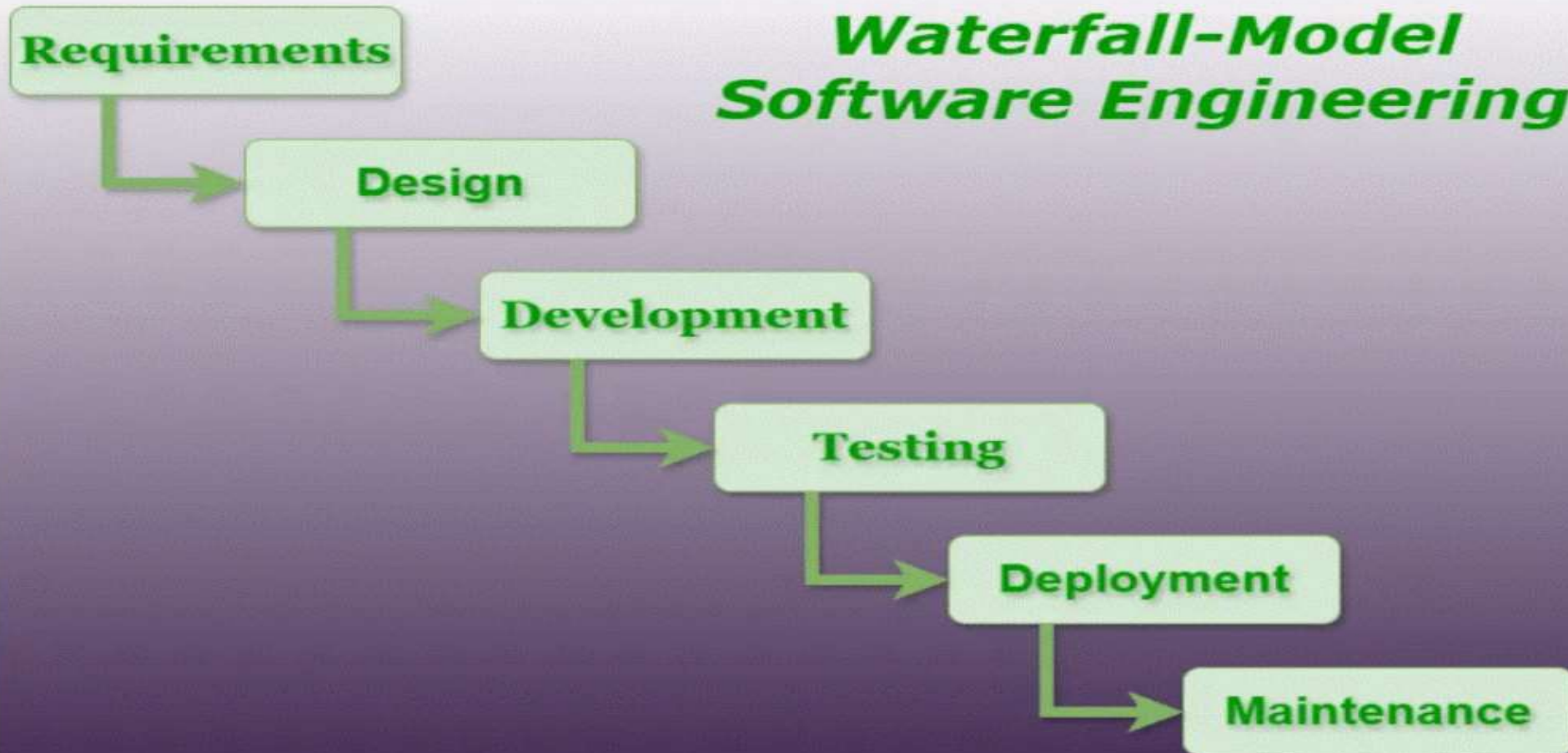
- Software Development Life Cycle (SDLC) models are **frameworks or methodologies** that guide the process of developing software applications from **planning and requirements gathering to deployment and maintenance.**
- Each SDLC model follows a different approach to organizing and managing the phases of software development.



1. Waterfall model

- The Waterfall model is one of the oldest and most straightforward approaches to software development.
- The **Waterfall Model** is a **linear and sequential Software Development Life Cycle (SDLC)** model.
- In this model, each phase of software development is completed before moving to the next phase. The process flows in one direction, like a **waterfall**, making it simple, structured, and easy to understand.

Waterfall-Model Software Engineering



Phases of Waterfall SDLC Models

1. Requirement Analysis

- This is the **first phase** of the Waterfall Model. The development team meets with the client or stakeholders to gather and analyze all the software requirements. The goal is to clearly understand what the customer wants and define the project's scope and objectives.

Activities:

- Gather user requirements.
- Analyze project feasibility.
- Prepare the Software Requirement Specification (SRS) document.

2. System Design

- After the requirements are finalized, the **design phase** begins. In this phase, the development team creates a detailed plan for building the software.

Activities:

- Design the software architecture.
- Design the database and user interface (UI).
- Define system components and modules

3. Development (Implementation)

- In this phase, developers write the source code according to the design specifications. Each module is developed and tested individually to ensure it works correctly.

Activities:

- Write the program code.
- Develop software modules.
- Perform **Unit Testing** on each module.

4. Testing

- After the software is developed, it is tested to ensure that it meets all the specified requirements and is free from defects or errors.

Activities:

- Perform integration testing.
- Conduct system testing.
- Carry out user acceptance testing (UAT).
- Identify and fix bugs.

5. Deployment

- Once the software has been successfully tested and approved, it is released to the users or installed in the production environment for actual use.

Activities:

- Install the software.
- Configure the production environment.
- Release the application to users.
- Perform final verification (smoke testing).

6. Maintenance

- Maintenance is the final phase of the Waterfall Model. After deployment, the software is monitored and maintained to ensure it continues to work correctly. Any issues are fixed, and updates or enhancements are made as needed.

Activities:

- Fix bugs and errors.
- Improve software performance.
- Update the software based on user feedback.
- Add minor enhancements and security updates

Advantages of the Waterfall Model:

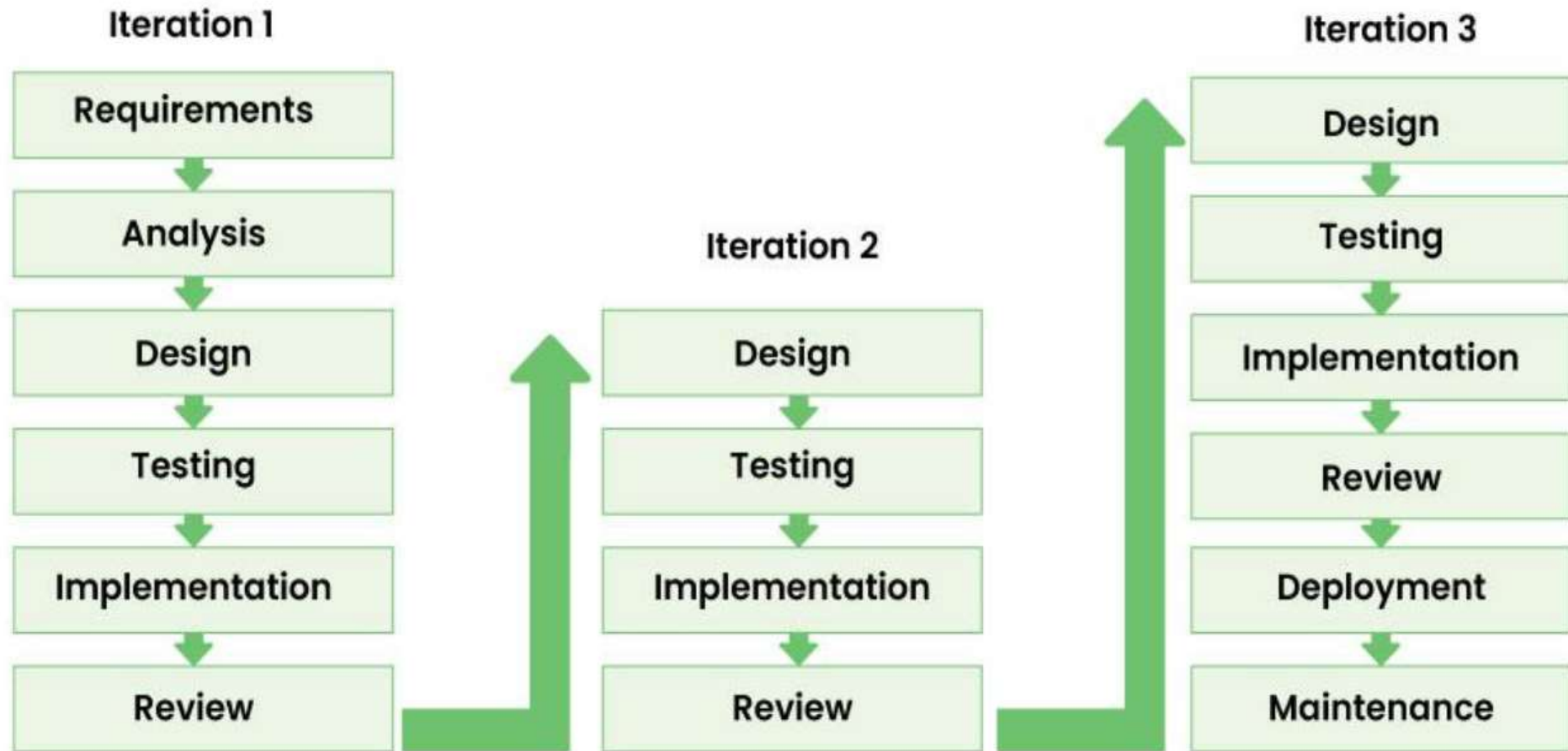
- Simple and easy to understand.
- Provides clear documentation at every phase.
- Suitable for projects with stable and fixed requirements.
- Offers better predictability of project schedule, cost, and deliverables.

Disadvantages of the Waterfall Model:

- Difficult to accommodate changes once development begins.
- Testing is performed only after coding, so defects are identified late.
- Limited customer involvement after the requirements phase.
- Does not support prototyping or early user feedback.

2. Iterative SDLC Models

- The **Iterative SDLC Model** is a software development methodology in which the software is developed **through repeated cycles (iterations)**. Instead of building the entire system at once, the project is developed in **small parts**, and each iteration improves the software based on testing and user feedback.
- The Iterative Model is **flexible and efficient** because it allows continuous improvement, early testing, and easy adaptation to changing requirements.



Iteration 1 – Initial Version

- This is the first development cycle where the basic version of the software is created.

Steps:

- **Requirements** – Gather the initial requirements from the client.
- **Analysis** – Analyze the requirements and determine feasibility.
- **Design** – Create the system design and architecture.
- **Testing** – Test the developed features for defects.
- **Implementation** – Develop (code) the first version of the software.

Iteration 2 – Improved Version

- Based on the feedback from Iteration 1, improvements and additional features are added.

Steps:

- Design
- Testing
- Implementation
- Review

Iteration 3 – Final Version

- The software is further refined and prepared for release.

Steps:

- **Design** – Design the remaining features.
- **Testing** – Test the new features and overall system.
- **Implementation** – Complete the remaining development.
- **Review** – Final evaluation and customer approval.
- **Deployment** – Release the software to users.

Advantages of the Iterative SDLC Model

- Faster delivery of working software through incremental development.
- Flexible enough to accommodate changing requirements.
- Better software quality due to continuous testing and improvements.
- Regular stakeholder feedback ensures the final product meets user and business requirements.

Disadvantage

Higher Cost

Repeated development and testing increase project cost.

Time-Consuming

Multiple iterations may take longer to complete.

Requires Skilled Team

Needs experienced developers and managers.

Scope Creep

Frequent changes may expand the project scope.

Complex Management

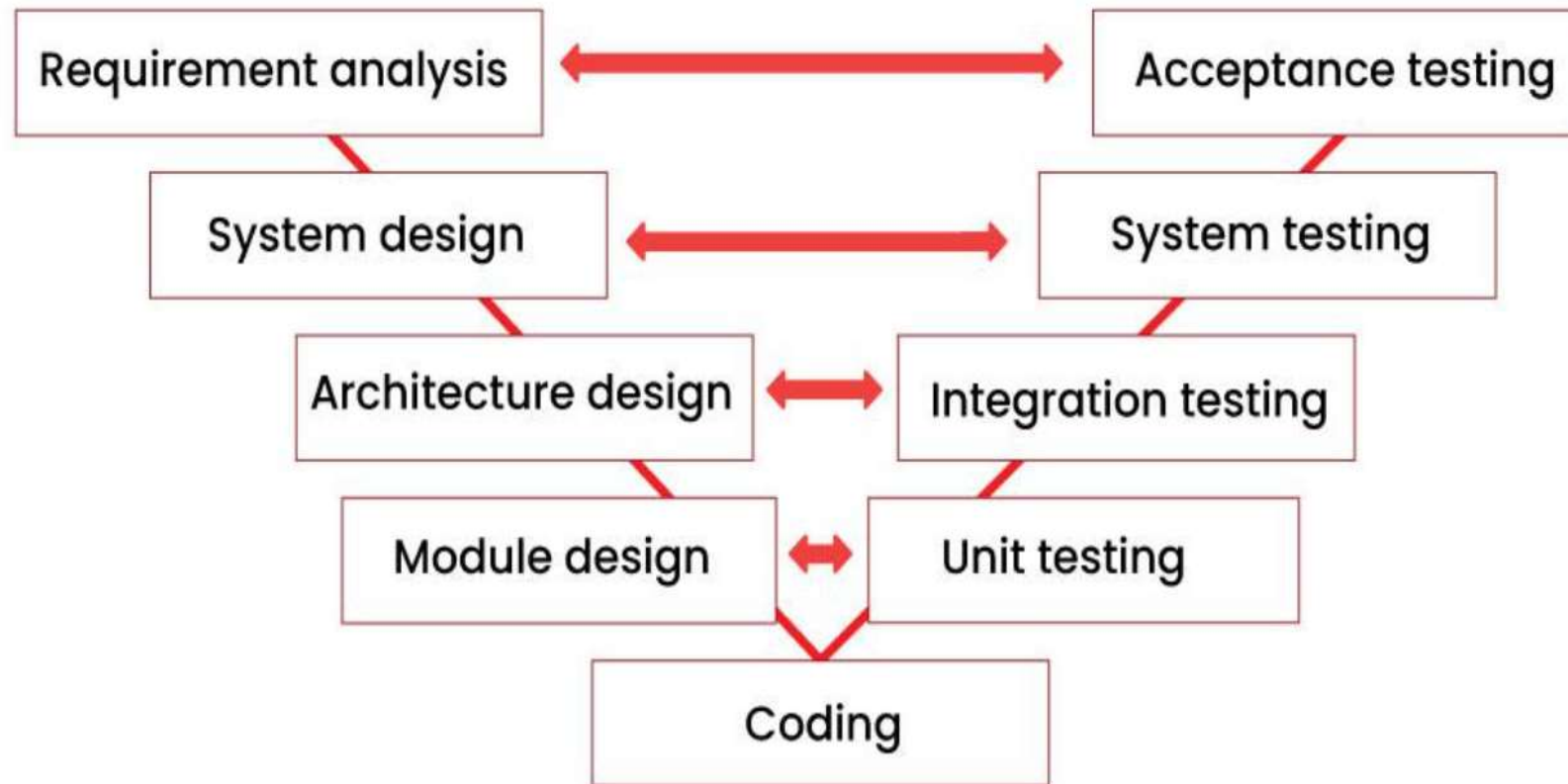
Managing multiple iterations is more difficult.

More Documentation

Documentation is needed after each iteration.

V-Model (Verification and Validation Model) in SDLC

- The **V-Model (Verification and Validation Model)** is an **extension of the Waterfall Model**. It follows a **sequential development process**, but unlike the Waterfall Model, **each development phase has a corresponding testing phase**.
- The development activities are on the **left side** of the V, and the testing activities are on the **right side**, forming a **V-shaped structure**.



Key Principles of the V-Model

1. Parallel Development and Testing

- Unlike the Waterfall Model, the **V-Model performs development and testing in parallel.** Every development phase has a corresponding testing phase, which helps detect defects early and improves software quality.

Example:

- Requirement Analysis ↔ Acceptance Testing
- System Design ↔ System Testing
- High-Level Design ↔ Integration Testing
- Low-Level Design ↔ Unit Testing

2. Verification and Validation

- The V-Model gives equal importance to **Verification** and **Validation**.
- **Verification** checks whether the software is being developed **correctly** according to the requirements and design.
 - **Activities:** Requirement review, design review, code review.
- **Validation** checks whether the final software **meets the customer's needs**.
 - **Activities:** Unit Testing, Integration Testing, System Testing, Acceptance Testing.

3. Traceability

- The V-Model provides **traceability** by creating a direct link between every development phase and its corresponding testing phase. This ensures that all requirements are properly designed, implemented, and tested.

Benefits:

- Better documentation.
- Easy tracking of requirements.
- Ensures complete test coverage.
- Helps identify missing or untested requirements.

4. Early Defect Detection

- Since testing activities are planned from the beginning and performed alongside development, defects are identified and corrected **early**. This reduces the cost and effort required to fix errors later.

Benefits:

- Improves software quality.
- Reduces development cost.
- Saves time.
- Produces a more reliable software product.

Advantages of the V-Model

1. Clear Design and Planning

- The V-Model follows a **well-structured framework** with clearly defined phases. Each stage has specific tasks and deliverables, making project planning and management easier.

2. Early Issue Identification

- Testing activities begin **from the early stages of development**. This helps identify and fix defects before they become major problems, reducing development time and cost.

3. Traceability and Documentation

- The V-Model maintains a **direct relationship between development phases and testing phases**. This ensures complete documentation and makes it easier to track project progress and verify that all requirements are tested.

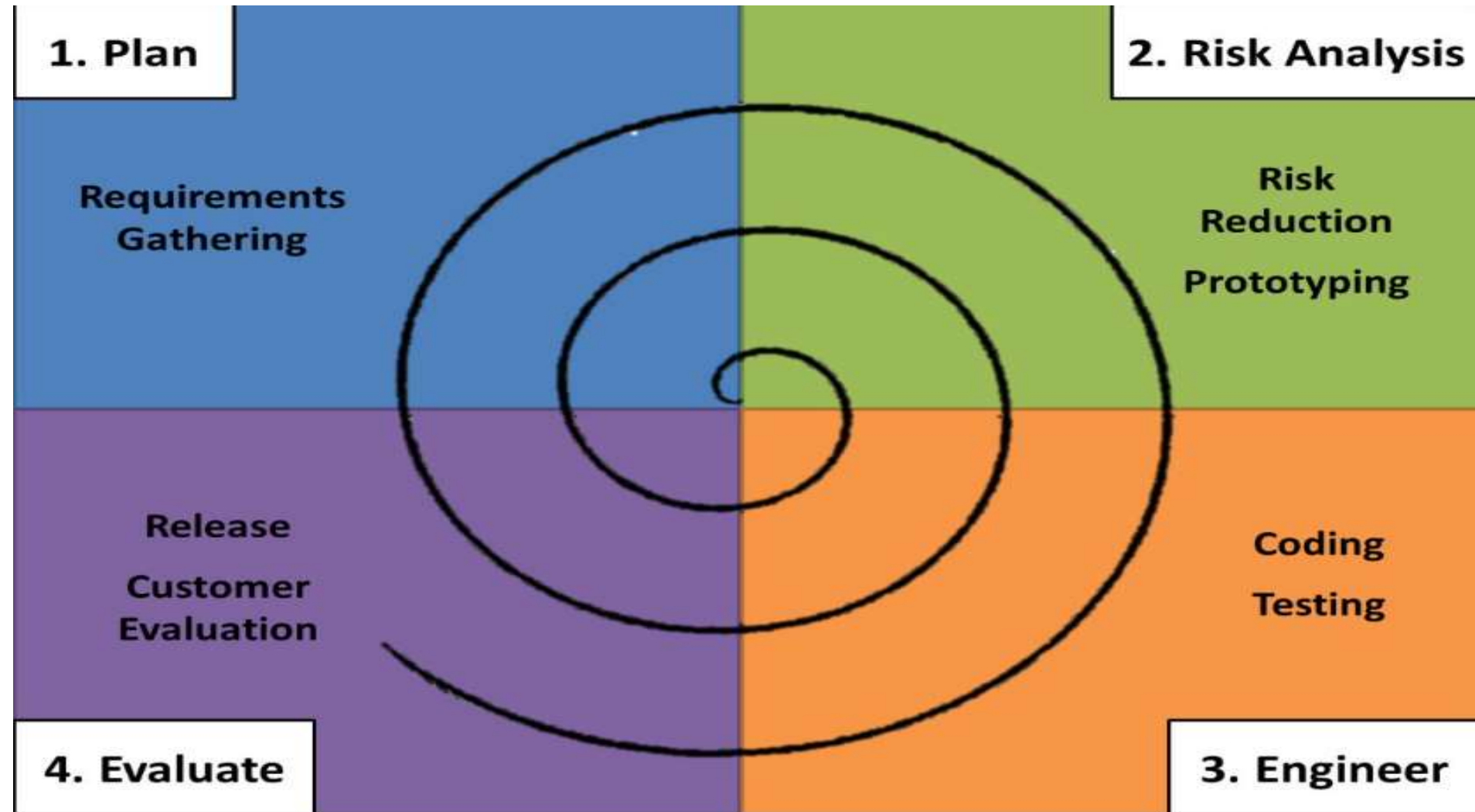
4. Predictability and Control

- The systematic and sequential nature of the V-Model makes the project **more predictable and easier to control**. Stakeholders have a clear understanding of project milestones, timelines, and deliverables.

4. Spiral SDLC Model

- The **Spiral SDLC Model** is a software development model that **combines the iterative approach of the Iterative Model with the systematic phases of the Waterfall Model.**
- Development progresses through a series of **spiral loops**, where each loop represents one phase or iteration of the project.

- The Spiral Model is **risk-driven**, meaning that **risks are identified, analyzed, and addressed during every iteration**. This makes it highly suitable for **large, complex, and high-risk projects**.



Phases of the Spiral Model

- Each loop of the spiral consists of **four phases**:

1. Planning

- Gather customer requirements.
- Define project objectives and constraints.
- Plan the activities for the current iteration.

2. Risk Analysis

- Identify possible risks.
- Analyze technical, financial, and schedule risks.
- Develop strategies to reduce or eliminate

3. Engineering (Development)

- Design, code, and test the software.
- Develop a working version of the product.

4. Evaluation

- the developed software to the customer.
- Collect feedback.
- Decide whether another iteration is needed.
- The cycle continues until the complete software is developed.

Advantages

- Excellent risk management.
- Flexible to changing requirements.
- Continuous customer feedback.
- Suitable for large and complex projects.
- Early detection and resolution of risks.
- Delivers improved software after every iteration.

Disadvantages

- Expensive due to repeated risk analysis.
- Requires experienced project managers.
- Complex to implement and manage.
- Not suitable for small or low-budget projects.
- Development time may increase because of multiple iterations.

5. Incremental Process Model

- The **Incremental Process Model** is a software development approach in which the software is developed and delivered in **small parts called increments**.
- Each increment adds new functionality to the previous version until the complete system is finished.

Phases of Incremental model

01

Requirement Analysis

02

Design & Development

03

Testing

04

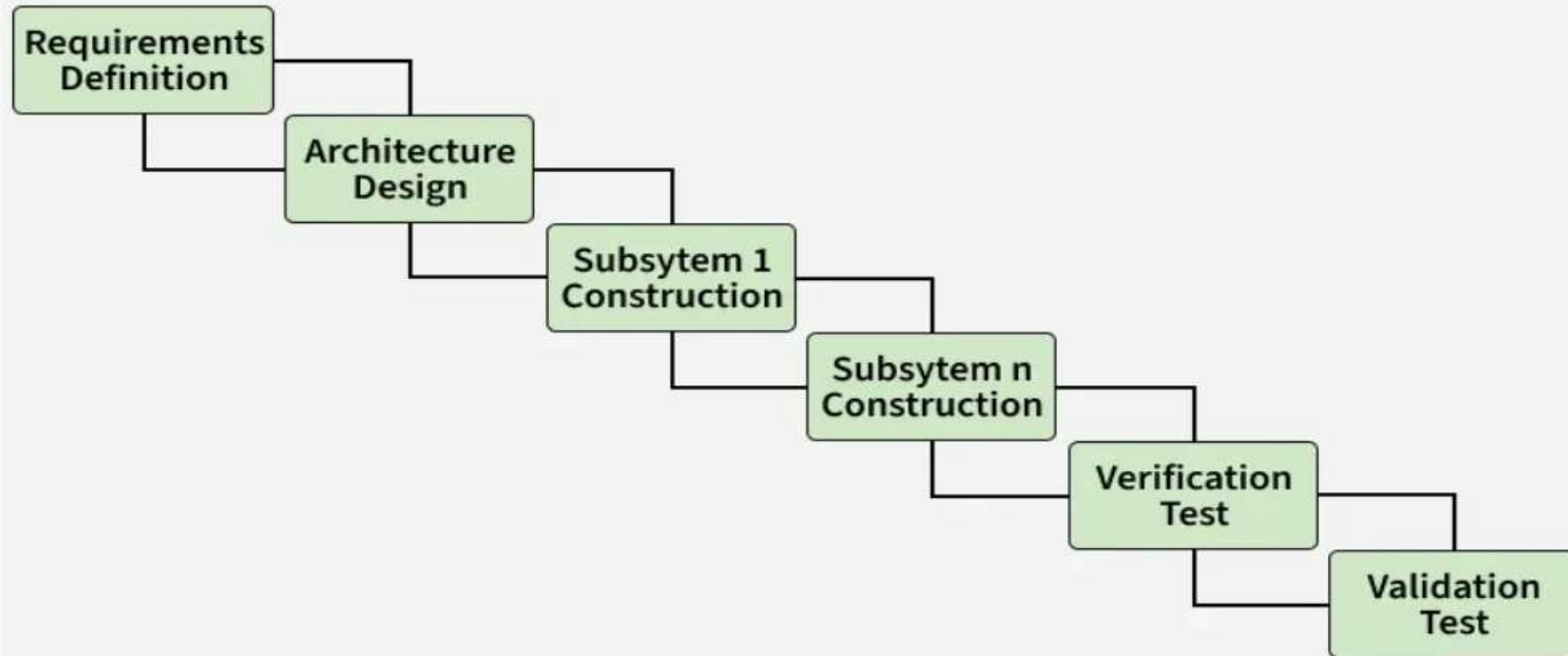
Implementation

- **Requirement Analysis:** Requirements for the current increment are gathered and clearly defined.
- **Design & Development:** The system design is prepared, and new features are developed.
- **Testing:** Each increment is tested to ensure new and existing features work correctly.
- **Implementation:** The tested increment is delivered and integrated with previous versions.

- **Types of Incremental Model**
- The **Incremental Model** has **two main types**, each offering a different approach to developing software in small parts (increments).

1. Staged Delivery Model

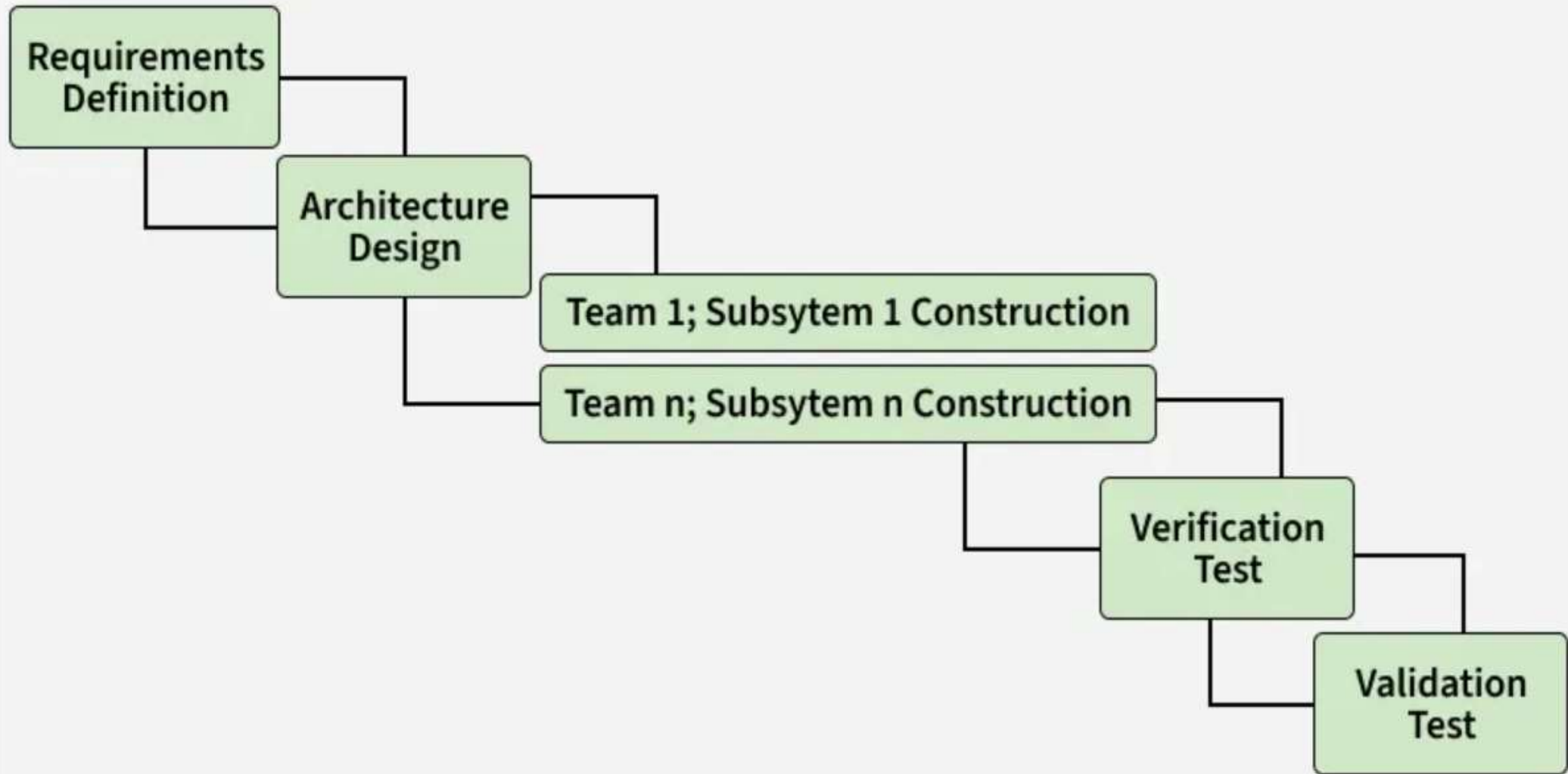
- The **Staged Delivery Model** is an incremental software development approach in which the software is built and delivered in **planned stages**.
- Each stage provides a **working and functional part** of the system. With every new release, more features are added until the complete software is finished.
- This approach allows users to see progress regularly, provide feedback, and identify issues



Staged Delivery Model

- **Parallel Development Model**

- The **Parallel Development Model** is an incremental software development approach in which the system is **divided into multiple modules**.
- After the overall system design is completed, different modules are **developed simultaneously (in parallel)** by different development teams.
- Since multiple teams work at the same time, the overall development time is reduced, making the process faster and more efficient. Once all modules are completed, they are integrated to form the final software system



- **Advantages**

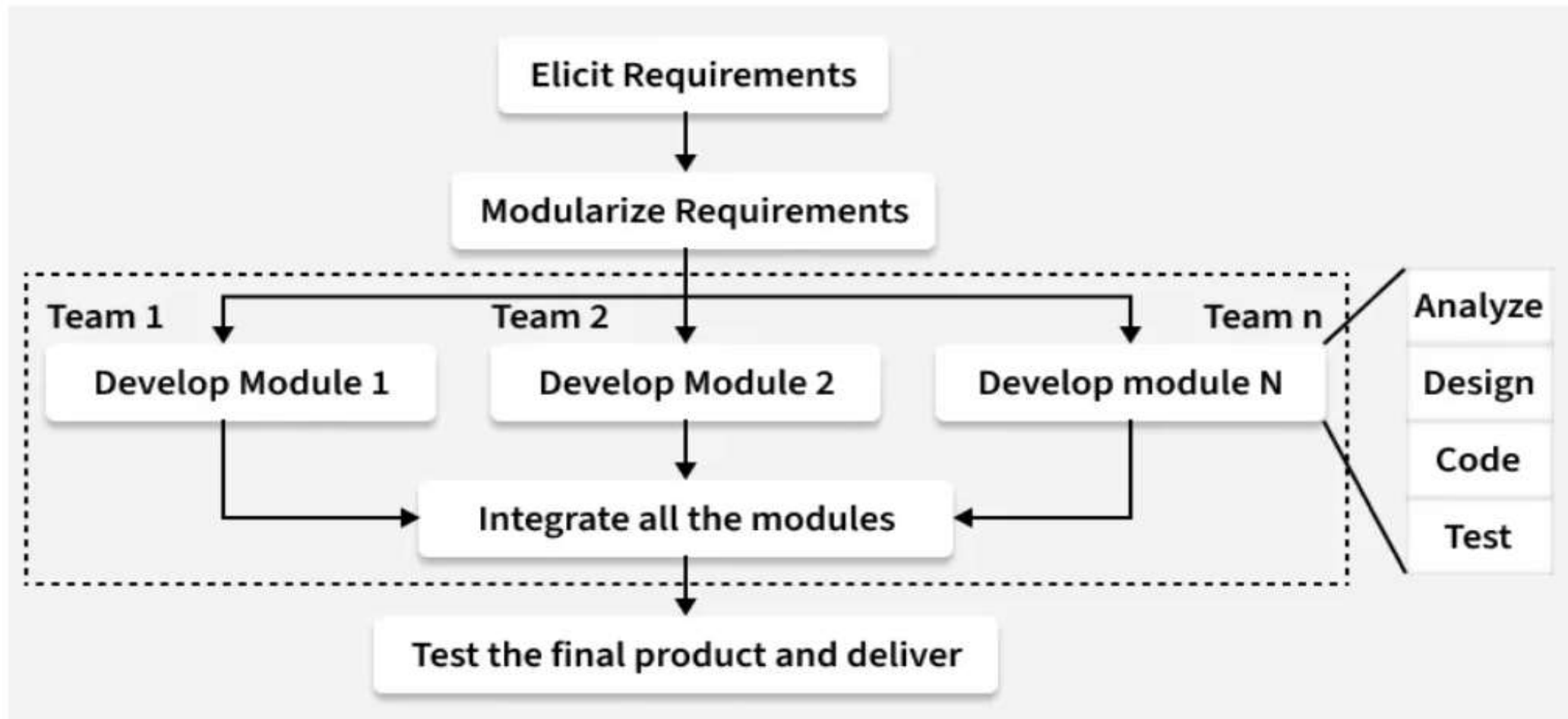
- Early Software Delivery, improves customer satisfaction
- Easy to Manage Changes between increments
- Better Risk Management through early testing
- Cost Control by spreading budget over stages
- Simpler Debugging due to modular development
- High Software Quality through continuous testing

Disadvantages

- Requires Skilled Team and proper planning
- Cost May Increase due to repeated testing and integration
- Design Issues if future requirements are unclear
- Integration Challenges between increments
- Repeated Bug Fixing across multiple increments

6. Rapid Application Development (RAD) model

- The **Rapid Application Development (RAD) Model** is an **iterative and incremental software development model** that focuses on **rapid prototyping, quick development, and continuous user feedback** instead of extensive planning.
- It divides the project into small modules that are developed in parallel within **short development cycles (60–90 days)**.
- Users review each prototype and provide feedback, allowing improvements throughout the development process.



RAD Model

Advantages

- **Fast development:** Reusable components reduce development time.
- **Early customer feedback:** Users see prototypes early and give feedback.
- **Lower development effort:** Fewer developers are needed due to automation.
- **Better quality:** Powerful tools help build quality software quickly.
- **Easy progress tracking:** Development progress is visible in each phase.
- **Flexible to changes:** Changing requirements are easy to handle due to short cycles.
- **High productivity:** Small teams can deliver results faster.

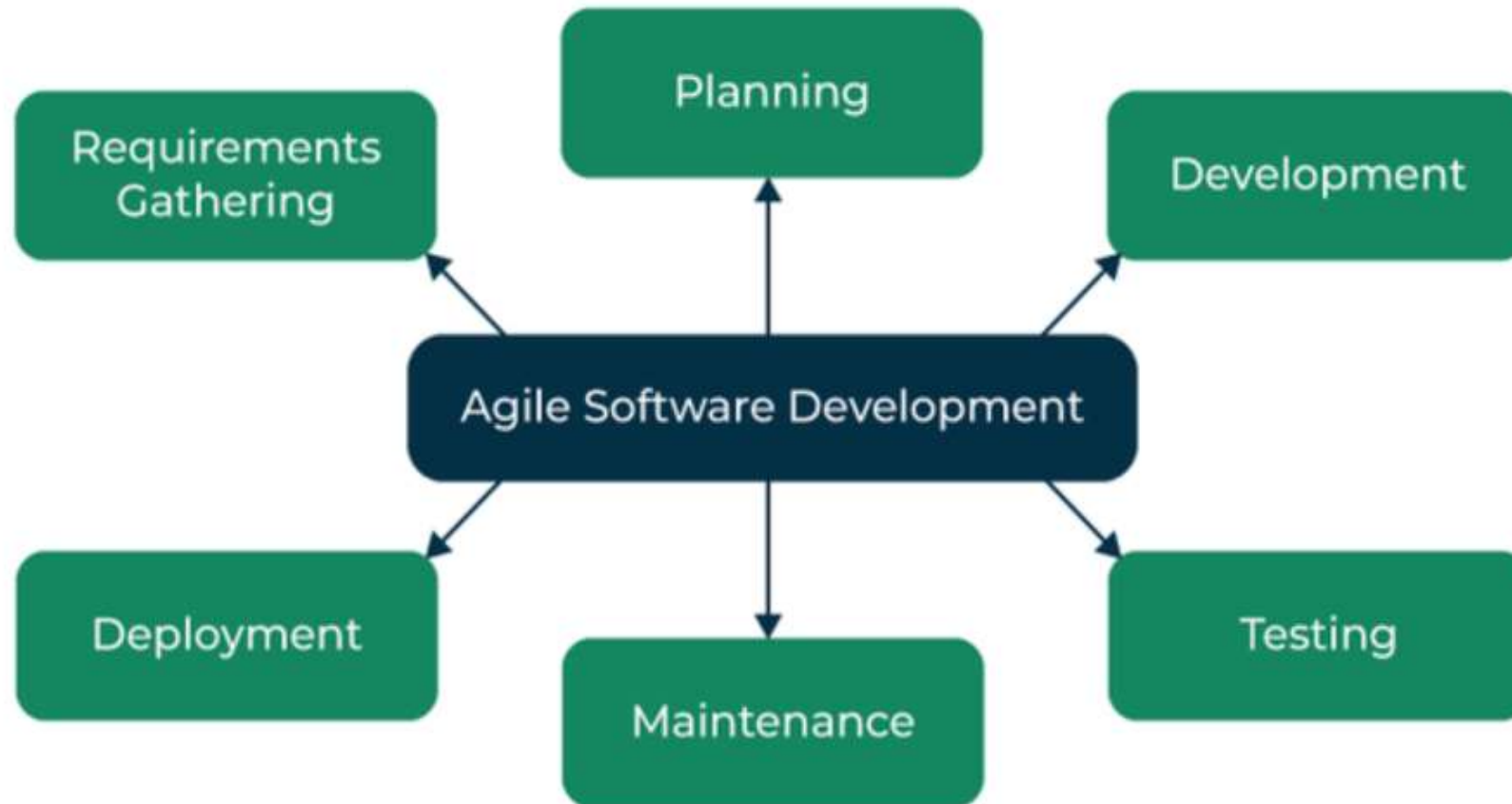
Disadvantages

- **Needs skilled professionals:** Developers must be experienced with advanced tools.
- **Depends on reusable components:** Lack of reusable parts can cause failure.
- **High coordination needed:** Team leader must closely manage developers and customers.
- **Not suitable for all systems:** Systems that cannot be modularized cannot use RAD.

- **Continuous customer involvement required:** Lack of user commitment can fail the project.
- **Costly for small projects:** Tool and automation costs may exceed the budget.
- **Not suitable for every application:** Some projects do not fit RAD structure

7. Agile Software Development

- The **Agile Software Development Model** is a software development methodology that emphasizes **flexibility, collaboration, and customer satisfaction**.
- It follows an **iterative and incremental approach**, where software is developed in small parts called **iterations (or sprints)**.
- Each iteration delivers a **working product quickly**, allowing customers to provide continuous feedback.
- The development team and customer work closely together to ensure the software meets changing requirements and user expectations.



Agile Software Development

Characteristics of Agile Software Development

1. Agile as a Mindset

- Agile is not just a development method; it is a way of thinking and working.
- It focuses on adaptability, teamwork, continuous improvement, and customer satisfaction.
- **Example:** If a customer requests a new feature, the team is ready to adapt instead of following a fixed plan.

2. Quick Response to Change

- Agile welcomes changing requirements, even during development.
- Teams can quickly modify the software based on customer feedback or new business needs.
- This makes Agile suitable for projects where requirements change frequently.
- **Example:** An online shopping app adds a new payment method after customer feedback without waiting for the entire project to finish.

3. Regular Demonstrations

- At the end of every **Sprint (2–4 weeks)**, the team demonstrates the working software to the customer.
- Customers review the progress and provide feedback.
- This ensures that the project is moving in the right direction and meets customer expectations.
- **Benefit:** Problems are identified early, reducing the cost of fixing them later.

4. Cross-Functional Teams

- Agile teams consist of members with different skills, such as **developers, testers, designers, business analysts, and product owners**.
- Team members work together, share knowledge, and communicate regularly.
- The team is **self-organizing**, meaning members decide how to complete the work without constant supervision.
- **Example:** While developers write code, testers test new features, and designers improve the user interface—all working together during the same sprint.

12 Principles of Agile Methodology

01

Customer Satisfaction

02

Changing Requirement

03

Frequent Delivery

04

Promoting Collaboration

05

Motivated Individuals

06

Face to Face Communication

07

Maintain a Constant pace

08

Measure Progress

09

Technical Excellence

10

Simplicity

11

Self organized Teams

12

Continues Improvements

Advantages :

1. Improved Collaboration and Communication
2. Flexibility and Adaptability
3. Better Quality and Reliability
4. Higher Customer Satisfaction

Disadvantages

1. Lack of Predictability
2. Limited Scope Control (Scope Creep)
3. Less Focus on Testing
4. Risk of Team Burnout
5. Lack of Structure

Requirement Engineering

What is Software Requirement?

- **Definition:**
- According IEEE Standard, Software Requirement is a condition needed by a user to solve a problem or achieve an objective.
- Requirement should be clear, correct and well-defined.
- **Requirements engineering (RE)** refers to the process of defining, documenting, and maintaining requirements in the engineering design process. Requirement engineering provides the appropriate mechanism to understand what the customer desires, analyzing the need, and assessing feasibility, negotiating a reasonable solution, specifying the solution clearly, validating the specifications and managing the requirements as they are transformed into a working system.
- Thus, requirement engineering is the disciplined application of proven principles, methods, tools, and notation to describe a proposed system's intended behavior and its associated constraints.

Types of Requirements

There are two types of requirements:

1. Functional Requirements:
2. Non-Functional Requirements



Functional Requirements

- Functional requirements define a function that a system or system element must be qualified to perform and must be documented in different forms. The functional requirements describe the behavior of the system as it correlates to the system's functionality.
- Functional requirements should be written in a simple language, so that it is easily understandable. The examples of functional requirements are authentication, business rules, audit tracking, certification requirements, transaction corrections, etc.
- These requirements allow us to verify whether the application provides all functionalities mentioned in the application's functional requirements. They support tasks, activities, user goals for easier project management.
- There are a number of ways to prepare functional requirements. The most common way is that they are documented in the text form. Other formats of preparing the functional requirements are use cases, models, prototypes, user stories, and diagrams.

Non-functional requirements

- Non-functional requirements are not related to the software's functional aspect. They can be the necessities that specify the criteria that can be used to decide the operation instead of specific behaviors of the system. Basic non-functional requirements are - usability, reliability, security, storage, cost, flexibility, configuration, performance, legal or regulatory requirements, etc.
- They are divided into two main categories:
- **Execution qualities** like security and usability, which are observable at run time.
- **Evolution qualities** like testability, maintainability, extensibility, and scalability that embodied in the static structure of the software system.
- Non-functional requirements specify the software's quality attribute. These requirements define the general characteristics, behavior of the system, and features that affect the experience of the user. They ensure a better user experience, minimizes the cost factor. Non-functional requirements ensure that the software system must follow the legal and adherence rules. The impact of the non-functional requirements is not on the functionality of the system, but they impact how it will perform. For a well-performing product, atleast some of the non-functional requirements should be met.

Functional requirements v/s Non-functional requirements

Functional Requirements	Non-functional requirements
Functional requirements help to understand the functions of the system.	They help to understand the system's performance.
Functional requirements are mandatory.	While non-functional requirements are not mandatory.
They are easy to define.	They are hard to define.
They describe what the product does.	They describe the working of product.
It concentrates on the user's requirement.	It concentrates on the expectation and experience of the user.
It helps us to verify the software's functionality.	It helps us to verify the software's performance.
These requirements are specified by the user.	These requirements are specified by the software developers, architects, and technical persons.
There is functional testing such as API testing, system, integration, etc.	There is non-functional testing such as usability, performance, stress, security, etc.
Examples of the functional requirements are - Authentication of a user on trying to log in to the system.	Examples of the non-functional requirements are - The background color of the screens should be light blue.
These requirements are important to system operation.	These are not always the important requirements, they may be desirable.
Completion of Functional requirements allows the system to perform, irrespective of meeting the non-functional requirements.	While system will not work only with non-functional requirements.

Examples

► Examples of Functional Requirements:

1. In hospital management system, a doctor should be able to retrieve the information of his patients.
2. Search option given to user to search from various invoices.
3. User should be able to mail any report to management.
4. Only Managerial level employees have the right to view banking revenue data.

► Examples of Non-Functional Requirements:

1. Security, Logging, Storage, Configuration, Performance, Cost, Interoperability, Flexibility, Disaster recovery, Accessibility
2. Employees never allowed to update their salary information. Reported to the administrator.
3. A website should be capable enough to handle 20 million users without affecting its performance.
4. The software should be portable. Moving from one OS to other OS does not create any issue.

What is Requirement Engineering?

The process to gather the software requirements from customers, analyze and document them is known as requirement engineering.

Requirement engineering provides the appropriate mechanism to understand:

- Customer desires / wishes.
- Analyzing Customer need
- Assessing feasibility
- Negotiating a reasonable solution
- Specifying the solution clearly
- Validating the specifications and managing the requirements

as they are transformed into a working system

Users VS System Requirements

No.	Users Requirements	System Requirements
1.	Written for Customers.	Written for Implementation Team.
2.	In Natural language.	In Technical Language.
3.	Describe the services & features provide by system.	Describe the detail description of services, features & complete operations of system.
4.	May include diagrams & tables.	May include system models.
5.	Understandable by system users who don't have technical knowledge.	Understandable by implementation team who have technical knowledge.
6.	For Client Managers, System Users, Contractor Managers, System Architects	For System Architects, Software Developers. Client Engineers, System Users

Tools in Requirement Engineering

1. Observation report
2. Questionnaire (survey, poll)
3. Use cases
4. User stories
5. Requirement workshop
6. Mind mapping
7. Role playing
8. Prototyping

Available Softwares:

1. Xebrio Requirements Management Software
2. Jira for requirements engineering
3. Doc Sheets
4. RequirementsHub
5. IBM Engineering Requirements Management DOORS
6. Modern Requirements
7. SpiraTeam

Establishing Ground Work

1. Identifying the stakeholders:

- Any person who benefits directly or indirectly from the system being developed is a stakeholder.
- **Examples:** Business operations managers, product managers, marketing people, internal and external customers, end-users, consultants, product engineers, software engineers, developers, testers and support/maintenance engineers are the usual stakeholders.
- Each stakeholder sees the system differently, gains different benefits when the system is successfully developed.
- They also faces different risks if the development effort fails.

Establishing Ground Work

2. Recognizing Multiple Viewpoints:

- Because there are so many different stakeholders, the system's requirements will be examined from various perspectives.
- Each of these stakeholders will contribute data to the requirements engineering process.
- As information is gathered from multiple viewpoints, collecting requirements may be inconsistent or contradictory.
- You should categorize all stakeholder information so that decision-makers can select a consistent set of system requirements for the system.

Establishing Ground Work

3. Working toward Collaboration:

- If there are five stakeholders involved in a software project, there may be five different opinions on the set of requirements.
- Customers must work together with software development team to create a successful system.
- A requirements engineer's job is to identify areas of commonality requirement as well as areas of conflict requirements.
- At the end project head & requirement engineer may decide on which requirements are accepted.

Establishing Ground Work

4. Asking the first questions: (For identifying stakeholders)

The first set of questions are focuses on the customers, stakeholders as well as the overall project goals and benefits.

1. Who is the person or organization behind the request for this work?
2. Who will make use of the solution?
3. What is the economic value of a successful solution?
4. Is there a different source for the solution you require?

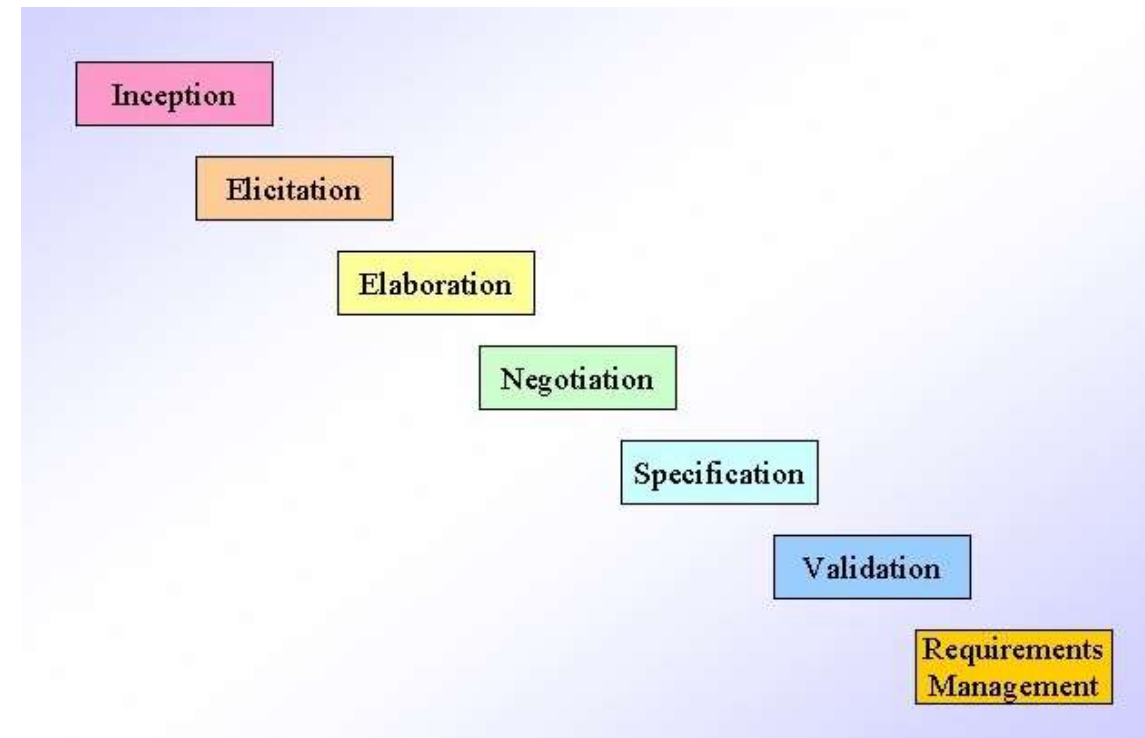
Establishing Ground Work

4. Asking the first questions: (Understanding Problems & Solution)

- Following set of questions helps in gaining a better understanding of the problem and allows the customer to express his or her thoughts on a solution:
 1. How would you characterize "good" output produced by a successful solution?
 2. To what problem(s) will this solution be applied?
 3. Can you describe (or show me) the business environment in which the solution will be used?
 4. Will there be any special performance issues or constraints that will influence how the solution is approached?

Requirement Engineering Tasks

- The process of collecting the software requirements from the client and then understanding, analysis, and documenting it is called requirement engineering.
- Requirement engineering constructs a bridge for design and construction.
- Requirement Engineering consists of seven different tasks:
 1. Inception
 2. Elicitation
 3. Elaboration
 4. Negotiation
 5. Specification
 6. Validation
 7. Management



1. Inception

- This is the first phase of the requirements analysis process or starting of project.
- Requirement engineering asks a set of questions to establish a software process.
 1. They understand basic details, aim, goal of the project & find out the solution.
 2. They identify stakeholders & who want the solution.
 3. They understand nature of solution.
 4. Enhance collaboration between customer & developer.



2. Elicitation

- This phase focuses on gathering the requirements from the stakeholders.
 - The right people must be involved in this phase, no space for mistake.
- The following problems can occur in the elicitation phase:
- 1. Problem of scope:** The customer give the unnecessary technical details rather than clarity of the overall system objective. Not possible to implement details have given.
 - 2. Problem of Understanding:** Not having a clear understanding between the developer and customer. Sometimes the customer might not know what they want or the developer might misunderstand one requirement for another.
 - 3. Problem of Volatility:** Requirements changing over time can cause difficulty in developing project. It can lead to loss and wastage of resources and time.

3. Elaboration & 4. Negotiation

3. Elaboration:

- It takes the requirements that have been gathered in the first two Inception & Elicitation phases and expand them.
- Its main task is developing model & prototype of software using functions, feature and constraints of a software using different tools.

4. Negotiation:

- Negotiation is between the developer and the customer about limited availability of resources, delivery time, project cost & overall estimation of project.
- Customers are asked to prioritize the requirements for development & also analyze any conflict has occur.

5. Specification & 6. Validation



5. Specification:

- In this task, the requirement engineer constructs a final work product.
- The work product is in the form of software requirement specification document.
- It collect all user, system, functional & non-functional requirements.
- ER. DFD, Data dictionary model used in this phase.
- A software specification document is submitted to the customer in a language that he/she will understand, to give a glimpse of the working model.

6. Validation:

- It checking for errors and debugging in final work product or SRS document.
- It check all the requirements have been stated and met correctly as per stakeholders.
- It check any missing information or want to add any additional information.



7. Requirement Management

- Requirements management is a set of activities where the entire team takes part in identifying, controlling, tracking, and establishing the requirements for the successful and smooth implementation of the project.
- It track on new additional requirement implementation which does not affect on overall system.
- Based on this phase, the working model will be analyzed carefully and ready to be delivered to the customer.



Summary

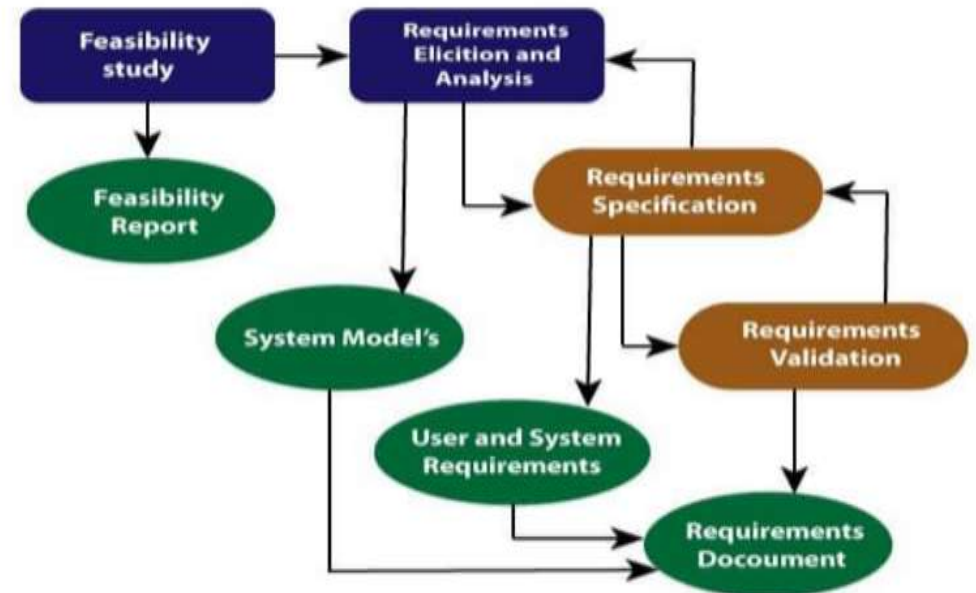
- Requirement Engineering consists of seven different tasks:
 1. **Inception** (Better Understanding)
 2. **Elicitation** (Clear Understanding of Requirements)
 3. **Elaboration** (Refinement & Modelling)
 4. **Negotiation** (Settlement & Conflicts)
 5. **Specification** (SRS Document)
 6. **Validation** (Validates Requirements)
 7. **Management** (Manage changing requirement)

Requirement Engineering Process

- The process to gather the software requirements from customers, analyze and document them is known as requirement engineering.
- ➤ Following four steps involve in Requirement Engineering Process:

Feasibility Study

- 1. Requirement Elicitation and Analysis
- 2. Software Requirement Specification
- 3. Software Requirement Validation
- 4. Software Requirement Management



Requirement Engineering Process

Feasibility Study

- To create & analyze the reasons for developing the software that is acceptable to users.
- To check aim & goal of the customer & organization behind implementation of software.

Types of Feasibility Study:

- 1. Technical Feasibility:** Evaluates the current technologies, which are needed to achieve customer requirements within the time and budget.
- 2. Operational Feasibility:** Required software solve business problems and customer requirements.
- 3. Economic Feasibility:** It decides whether the necessary software can generate financial profits for an organization.

The output is to decide whether or not the project should be undertaken

Step 1: Requirement Elicitation & Analysis

- Requirements elicitation is the process of gathering actual requirements about the needs and expectations of stakeholders for a software system.
- To understand customers, business manuals, the existing software of same type, standards and other stakeholders of the project.

Techniques of Requirements Elicitation:

- 1. Interviews:** One-on-one conversations with stakeholders to gather information about their needs and expectations.
- 2. Surveys:** These are questionnaires that are distributed to stakeholders to gather information.
- 3. Focus Groups:** Small groups of stakeholders who are brought together to discuss their needs.
- 4. Observation:** Observing the stakeholders in their work environment to gather information.
- 5. Prototyping:** Creating a working model of the software system, which can be used to gather feedback from stakeholders and to validate requirements.

Step 2: Software Requirement Specification

- Requirements received from client are written in natural language.
- After collecting, System analyst to document the requirements in technical language.
- SRS include Data Flow Diagram. Data Dictionary, Entity Relationship diagram.

► SRS defines

- ☐ How the intended software will interact with hardware.
- ☐ External interface / GUI
- ☐ Speed of operation, Response time of system
- ☐ Portability of software across various platforms
- ☐ Maintainability
- ☐ Speed of recovery after crashing
- ☐ Security, Quality. Limitations etc.



Step 3: Software Requirement Validation

- After SRS developed, the requirements discussed in this document are validated or tested
- Requirement validation done through Requirement reviews taken by customers. After developing prototyping check with customers & Test case generations.
- **Requirements can be the check against the following conditions -**
 - If they can practically implement.
 - If they are correct and as per the functionality and specially of software.
 - If there are any ambiguities.
 - If they can describe
 - Any changing or additional requirement



Step 4: Software Requirement Management

- Requirement management is the process of managing changing requirements during the requirements engineering process.

►Activities involved in Requirement Management:

1. **Tracking & Controlling Changes:** Handling changing requirements from customers.
2. **Version Control:** Keeping track of different versions of system.
3. **Traceability:** Trace to software is design, develop or test as per requirements or not.
4. **Communication:** If changing requirements has any issues contact with clients within time.
5. **Monitoring & Reporting:** Monitoring development process & report the status.

What is Software Requirement Specification?

- A software requirements specification (SRS) is a document that mentioned complete description about how the system is expected to perform, Behavior of system, Functional & Non Functional requirement of system.
- SRS is a formal report, that enables the customers to review whether it (SRS) is according to their requirements.
- It is usually signed off at the end of requirements engineering phase.
- **Users of SRS:**
 - 1. Client
 - 2. Development Team
 - 3. Maintenance Team
 - 4. Technical Writers

Need of SRS Document

1. It structures and formalizes all project requirements.
2. It helps the development team build a product that exactly meets customer and target audience needs.
3. It provides all team members with the necessary information while working on the project.
4. It minimizes the possible misunderstanding between the members of the development team and the customer.
5. It clearly defines the scope of work, and that allows developers to plan particular iterations and release dates.
6. It allows estimating the required development budget.

Structure of SRS

1. Introduction

- 1.1 Purpose
- 1.2 Intended Audience
- 1.3 Scope
- 1.4 Definition
- 1.5 References

2. Overall Description

- 2.1 User Interface
- 2.2 System Interface Structure of SRS
- 2.3 Software & Hardware Requirements
- 2.4 Constraints
- 2.5 User Characteristics

3. System Features & Requirements

- 3.1 Functional Requirements
- 3.2 Use Cases / Sequence Diagrams
- 3.3 External Interface Requirement
- 3.4 Database Requirement
- 3.5 Non-Functional Requirement

4. Deliver for Approval



Characteristics of good SRS

- 1. Correctness:** Provide accurate functional & non functional requirements as discussed with client.
- 2. Completeness:** Should complete all essential features like functionality, performance, features, design, constraints & external interfaces.
- 3. Consistency:** Same abbreviation, tabular format, diagram format, naming format follow.
- 4. Unambiguousness:** Should not be any confusion regarding understanding of the requirements. Everything mention uniquely & clearly.
- 5. Ranking for importance and stability:** Every requirement is important. But some are urgent and must be fulfilled before other requirements and some could be delayed. It's better to classify each requirement according to its importance and stability.
- 6. Modifiability:** It is capable of quickly obtain changes to the system without affecting other.

Characteristics of good SRS

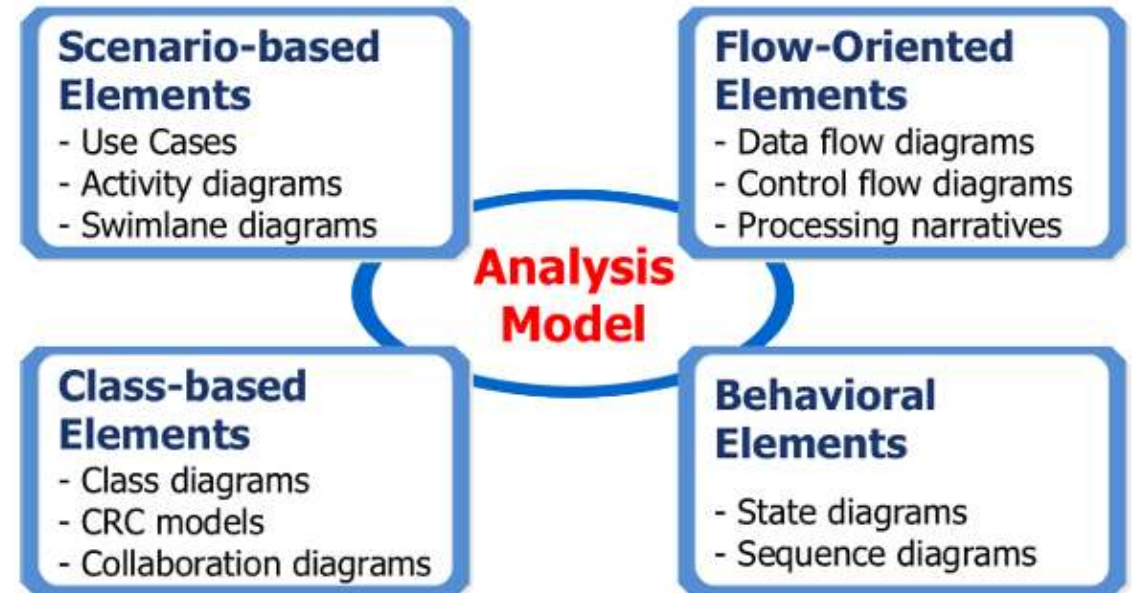
- 7. Verifiability:** The requirements are verified with the help of reviewers & stakeholders.
- 8. Traceability:** Every requirement having unique number that is easy to use in future development.
- 9. Design Independence:** There should be an option to select from multiple design alternatives for the final system. SRS should not contain any implementation details.
- 10. Testability:** An SRS should be written in such a method that it is simple to generate test cases and test plans from the report.
- 11. Understandable by the customer:** The language should be kept simple and clear.
- 12. The right level of abstraction:** If the SRS is written for the requirements stage, the details should be explained explicitly. Whereas, for a feasibility study, fewer analysis can be used. Hence, the level of abstraction modifies according to the objective of the SRS.

Requirement Analysis

- Requirement analysis is significant and essential activity after elicitation.
- This activity reviews all requirements and may provide a graphical view of the entire system.
- After the completion of the analysis, the understandability of the project may improve significantly.

► Requirement Analysis Model / Elements of Requirement Model

- 1. Scenario based Modelling
- 2. Class based Modelling
- 3. Flow oriented Modelling
- 4. Behavior Modelling

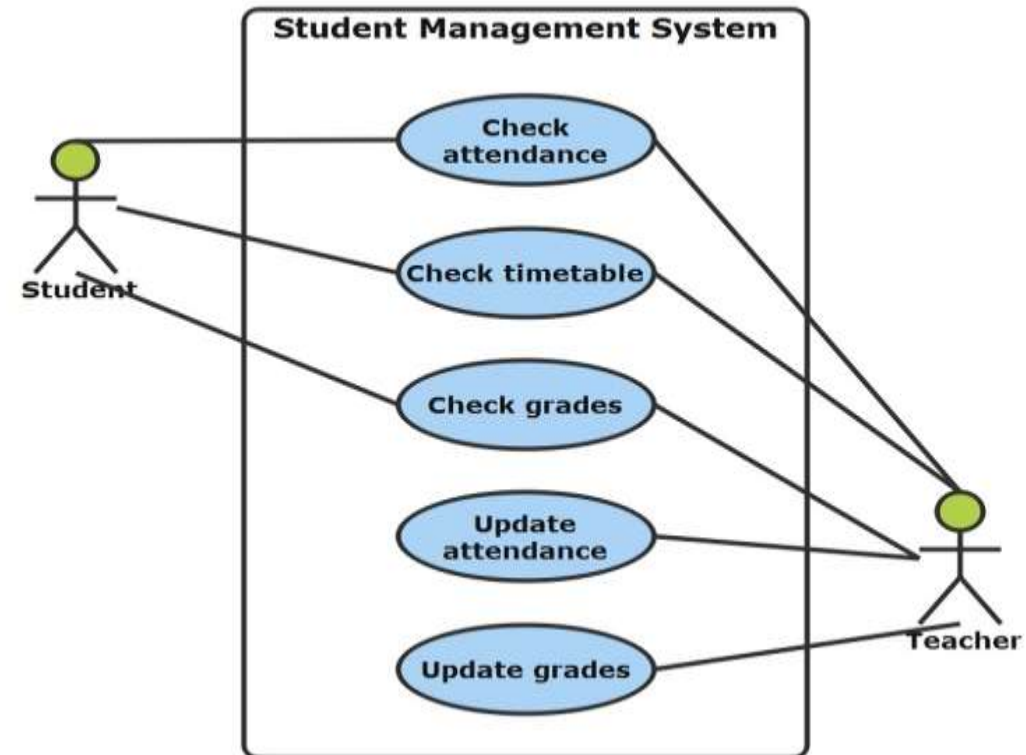
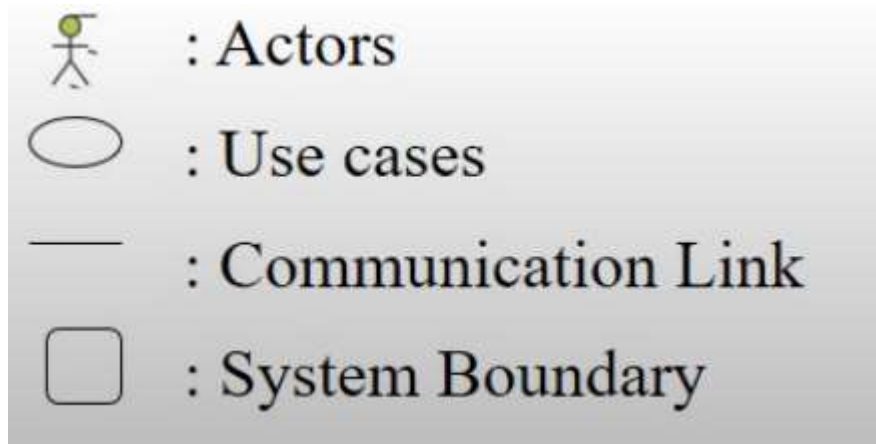


Scenario Based Modelling / Element

- For building the design & analysis model it is important for software engineer to understand how end users & other actors want to interact with the system.
- Here, creation of scenario by using Use case Diagram. Activity Diagram & User Stories.

1. Use case Diagram:

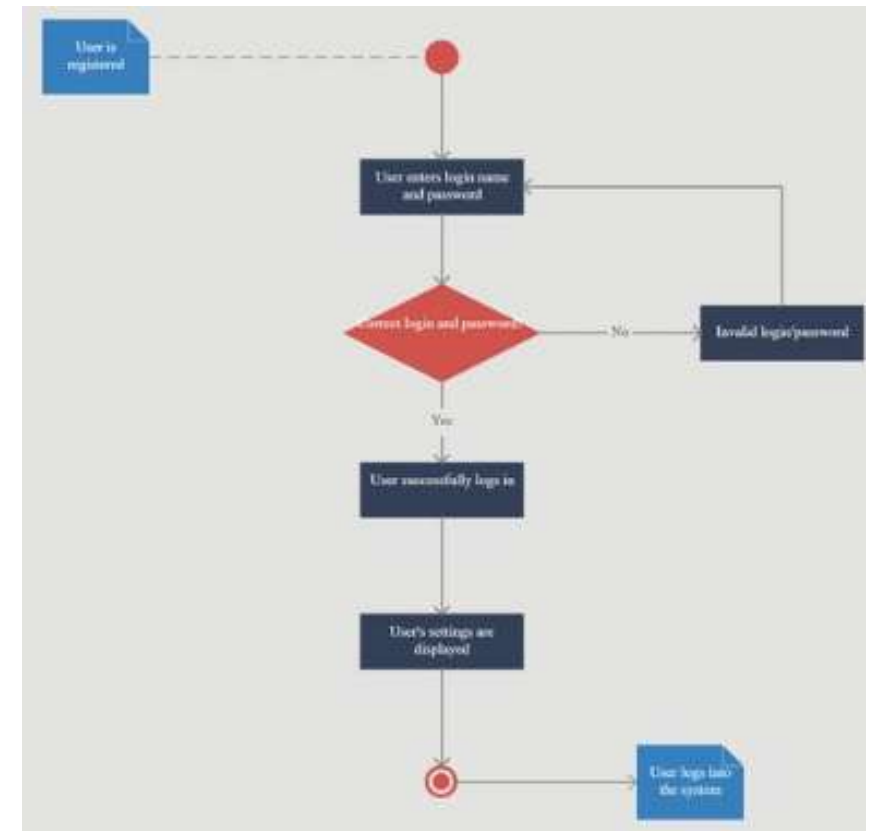
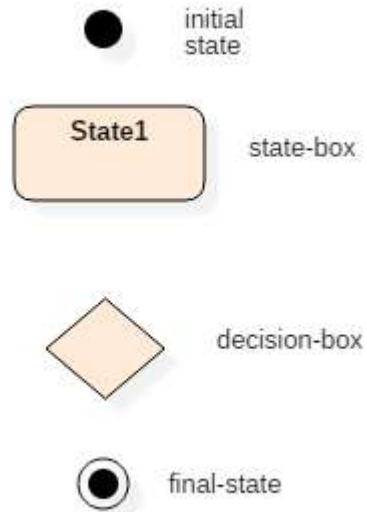
A use case diagram is used to represent the dynamic behavior of a system.



Scenario Based Modelling / Element

2. Activity Diagram:

- The activity diagram helps in predicting the workflow from one activity to another.
- It show condition of flow and the order in which it occurs.

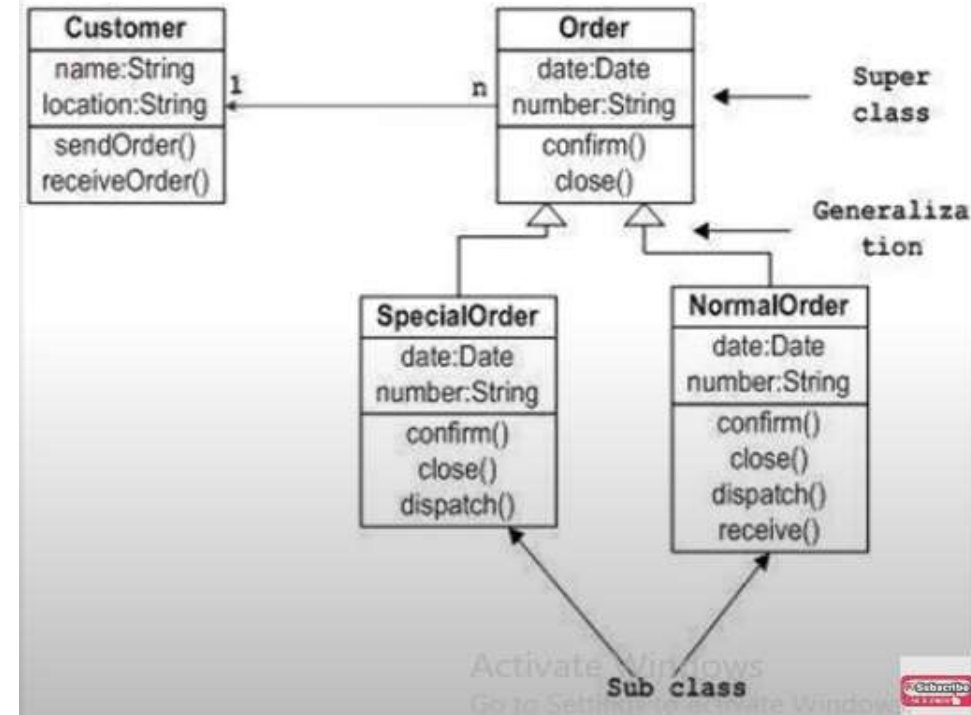
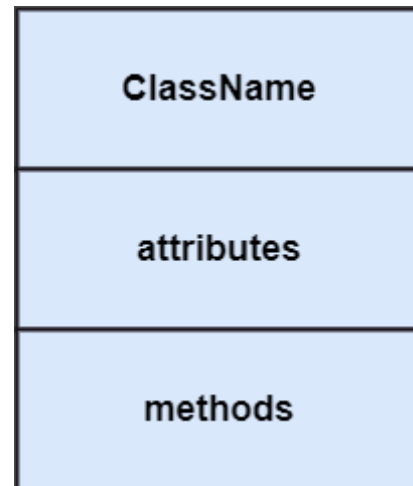


Class Based Modelling / Element

- Class based modeling represent classes, object, attributes & operations of system.

1. Class Diagram:

- The class diagram shows a static view of an application.
- It represents the types of objects residing in the system and the relationships between them.
- It describes the major responsibilities of a system.

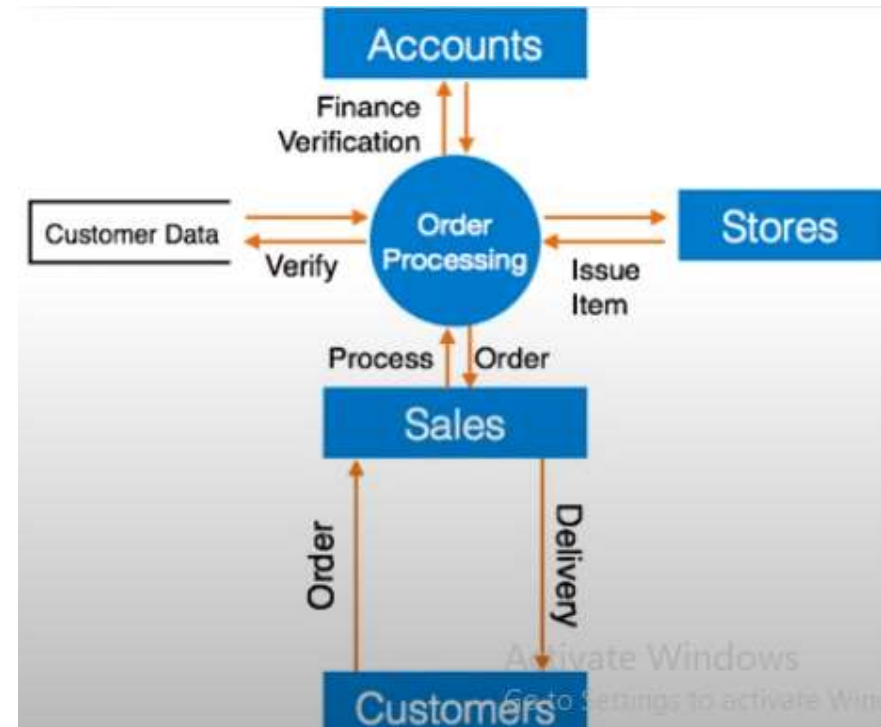


Flow Oriented Modelling / Element

- It shows how data objects are transformed by processing the function.

1. Data Flow Diagram:

- Data flow diagram is graphical representation of flow of data in an information system.
- It is capable of depicting incoming data flow, outgoing data flow and stored data.
- Components of DFD:



Flow Oriented Modelling / Element

2. Control Flow Diagram:

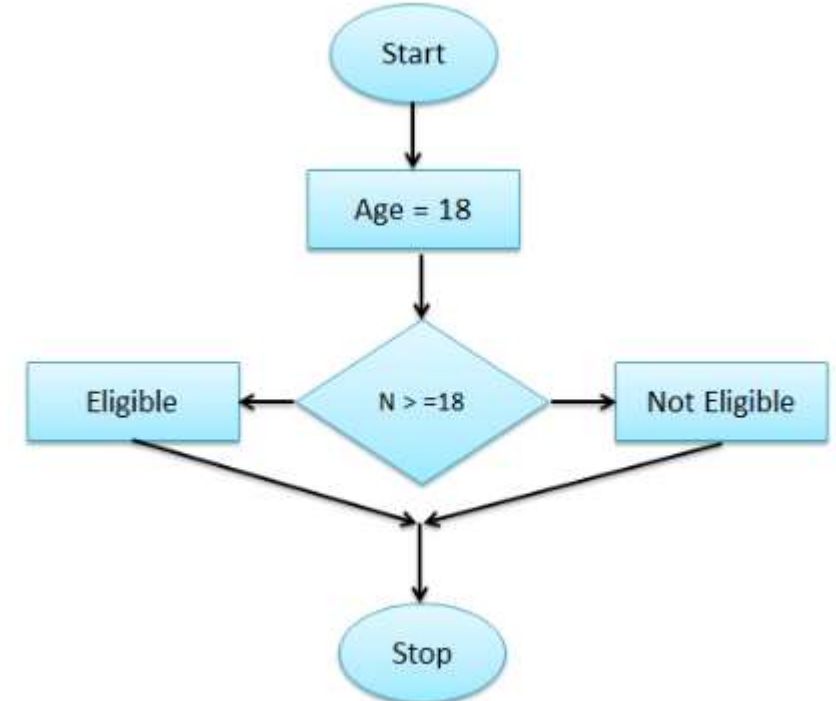
- It is a graphical representation of control flow or computation during the execution of programs or applications.
- Control flow graphs are mostly used in static analysis as well as compiler applications.
- Accurately represent the flow inside of a program unit.
- Used boolean values are true or false, on or off, 1 or 0.

- **Entry Block:**

Entry block allows the control to enter into the control flow.

- **Exit Block:**

Control flow leaves through the exit block.



Behavioral Modelling / Element

- Behavioral Model is specially designed to make us understand overall behavior and factors that influence behavior of a System.

1. State Transition Diagram:

- It usually describes overall states of system.
- And events which are responsible for a change in state of a system.

1. Initial State –



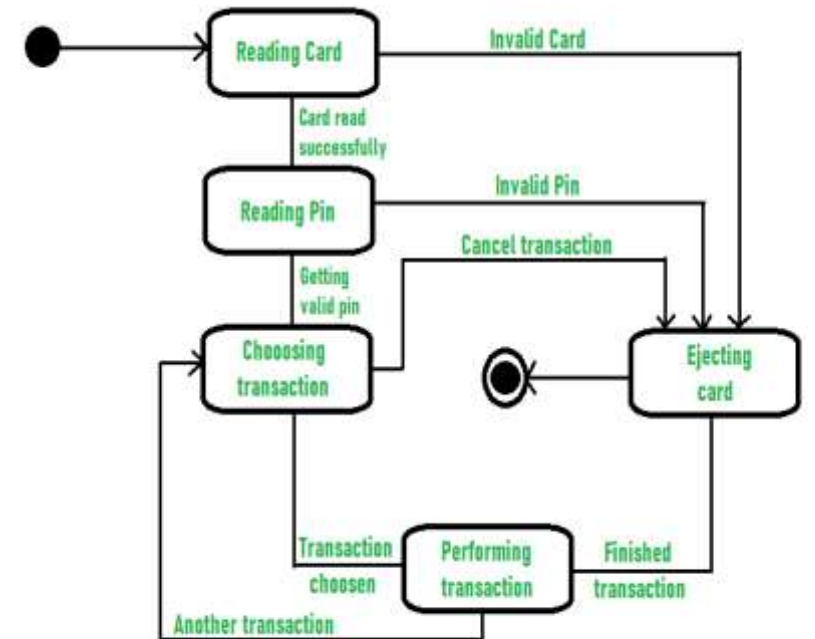
2. Final State –



3. Simple State –



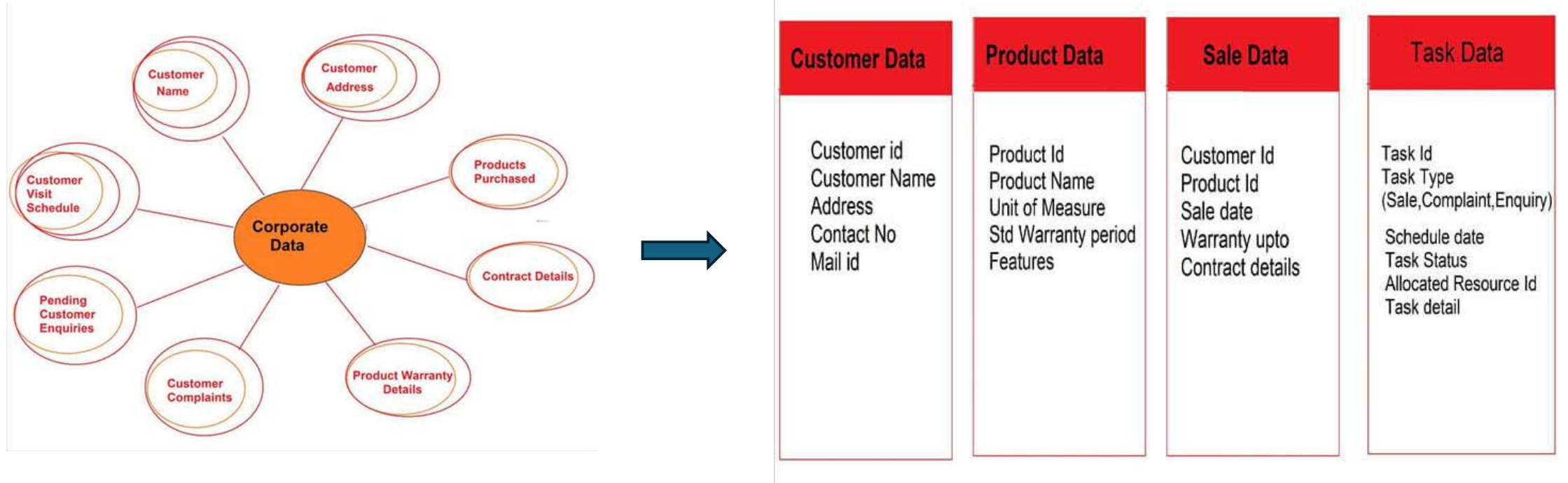
4. Composite State –



State Transition Diagram for ATM System

What is Data Modeling?

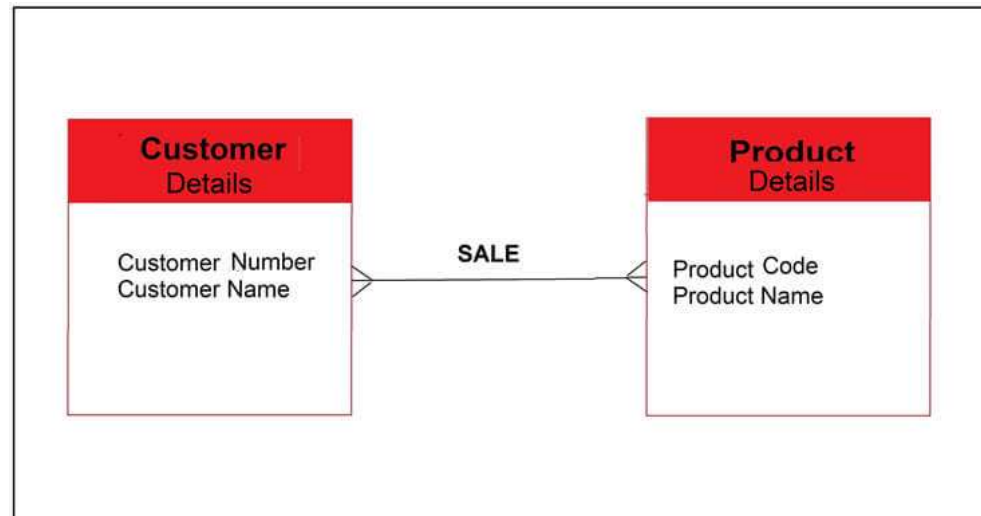
- A data model is an abstract view of the data referred to in the product being developed.
- Data Modeling explaining & visualizing how the data be used by the software and defining data objects that will be stored in a database. Example:



Types of Data Models

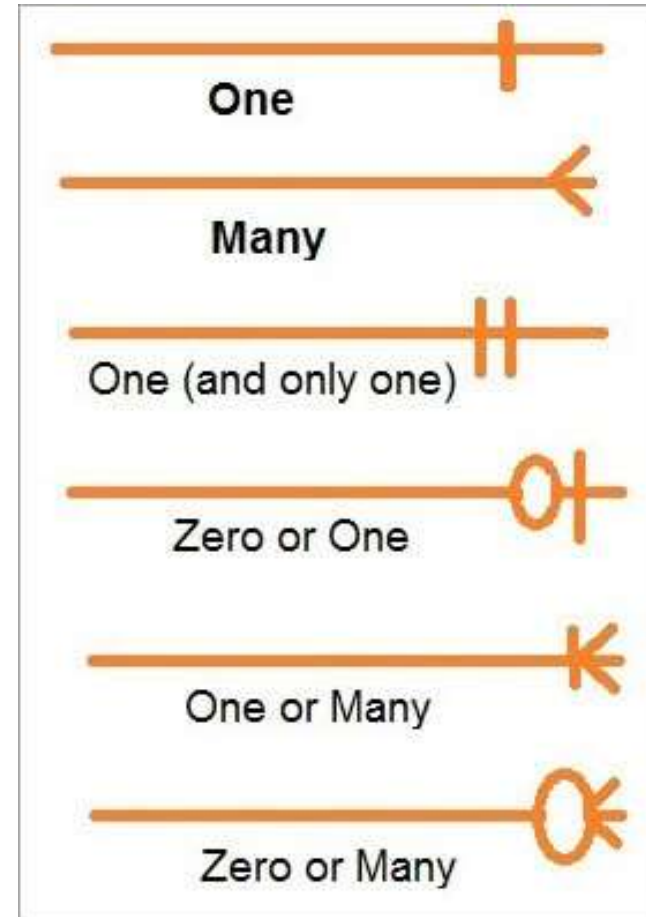
Type 1: Conceptual Data Model

- It gives a front view of the of each data entity and not a technical detail of data.
- It presents all the data entities referred to by the business and their characteristics called attributes and the dependency or the connection between entities called Relationships.
- It makes communication easy and clear as all stakeholders & reducing communication gaps.



Some commonly used Relationship notations are:

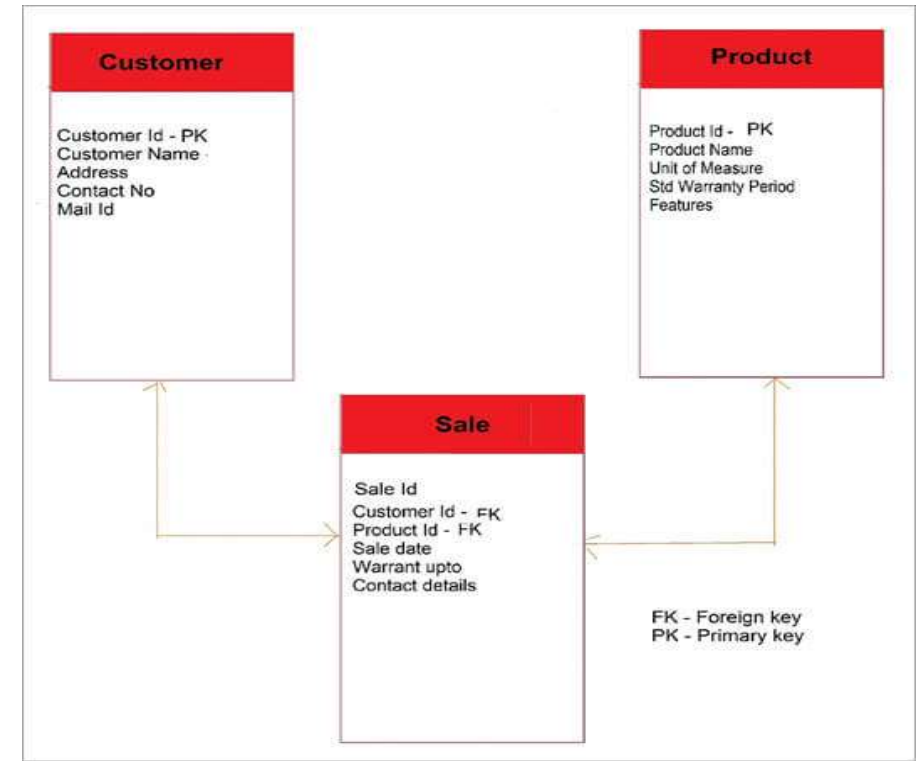
- **One to one:** Employee and his joining details
- **One to many:** Employee and his workplace
- **One and only one:** Employee and his date of birth
- **Zero or one:** Employee and his place of demise
- **Zero or many:** Employee and his subordinates
- **Examples of cardinality:** Relationship Notation



Types of Data Models

Type 2: Logical Data Model

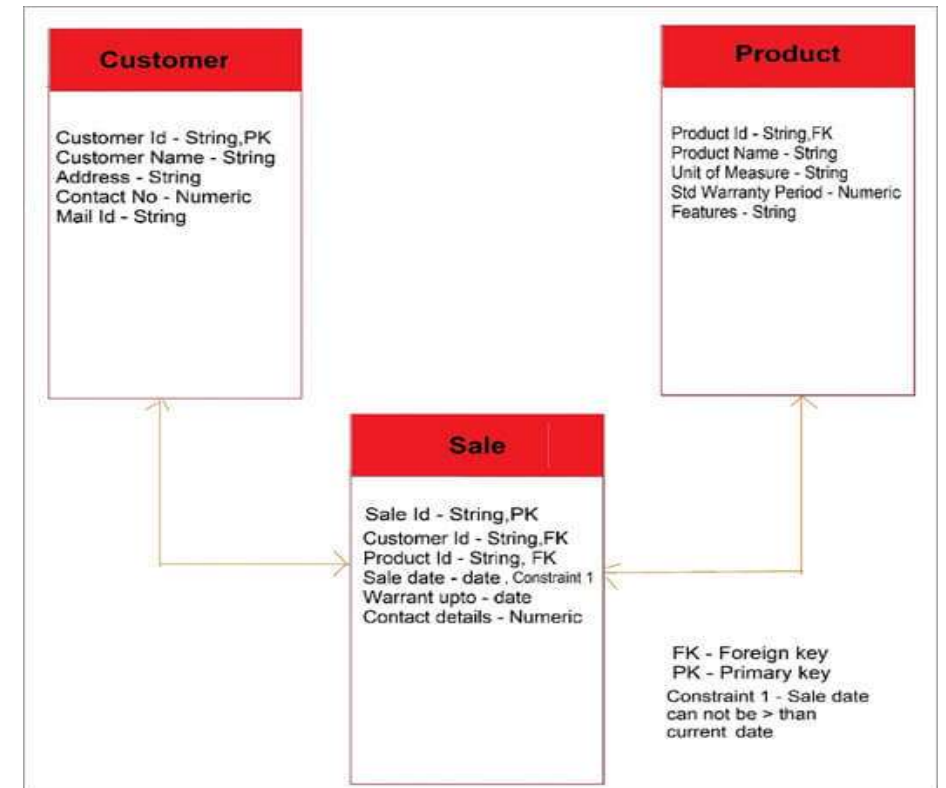
- It gives a detailed description of each data entity their attributes & relationship between two entities giving business purpose to each data.
- The relationship is presented more explicitly with details like Primary key, Foreign key & Parent-child dependent entity type.



Types of Data Models

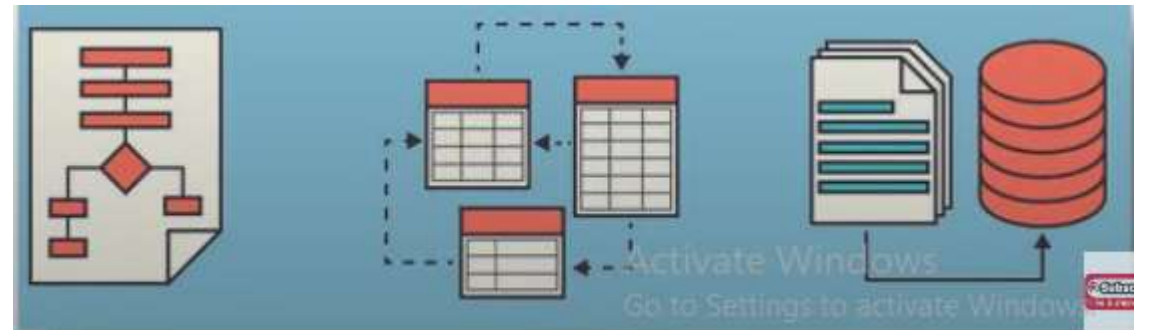
Type 3: Physical Data Model

- A physical data model is the layout of the actual database with all its components.
- It gives a technical view of the data i.e., the table name, column name, data type, constraints, indexes, primary key, triggers, stored procedures, etc.
- It is developed by using databases SQL, ORACLE etc.

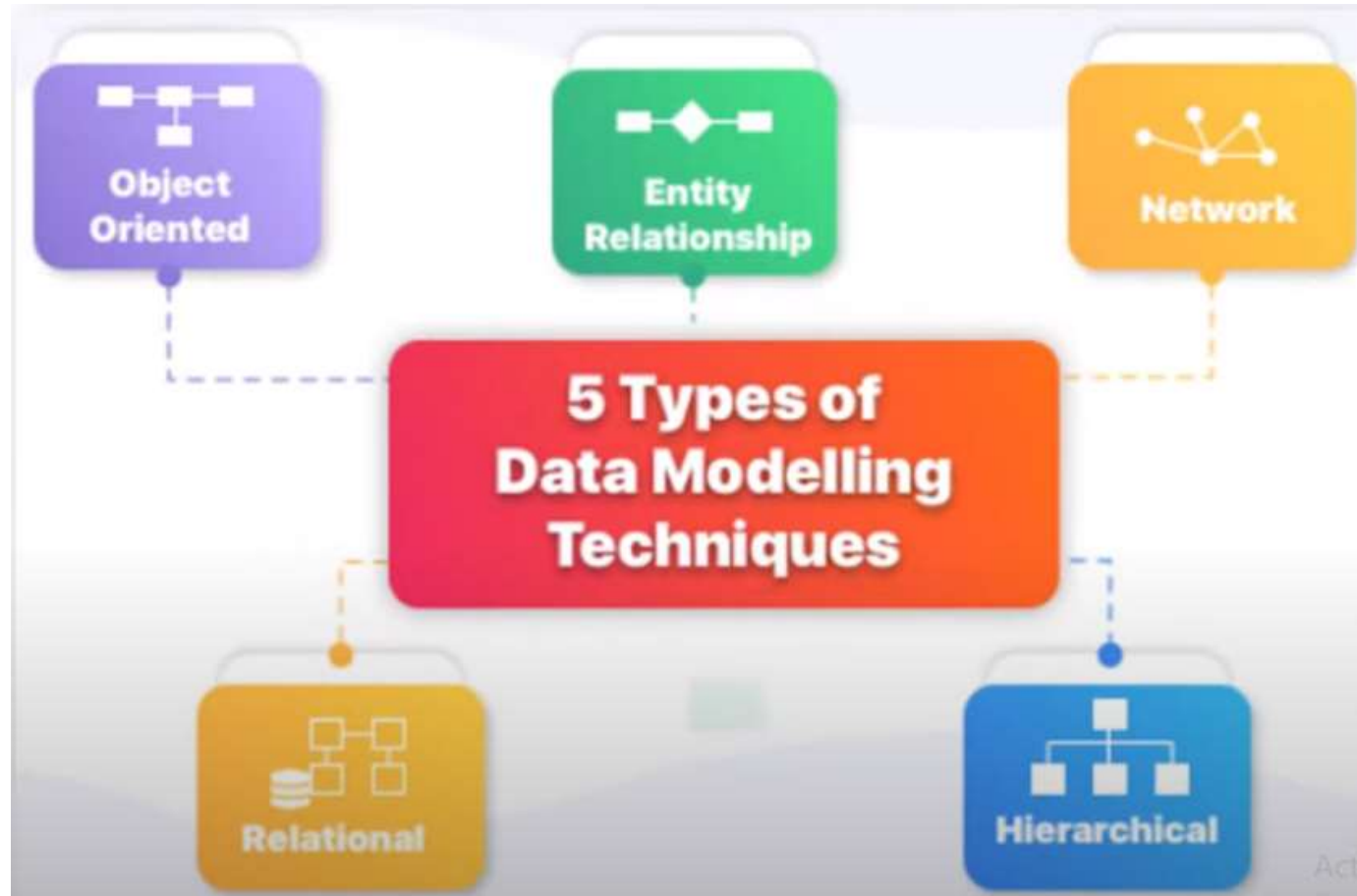


Why Data Modeling?

1. That helps in building the information system & database.
2. Facilitates easy access to the business data.
3. Helps management to take decisions.
4. Facilitate business to function efficiently.
5. All the stakeholders to get a clear and easy understanding of what information is going to be stored
6. The core rules and policies that will govern each entity

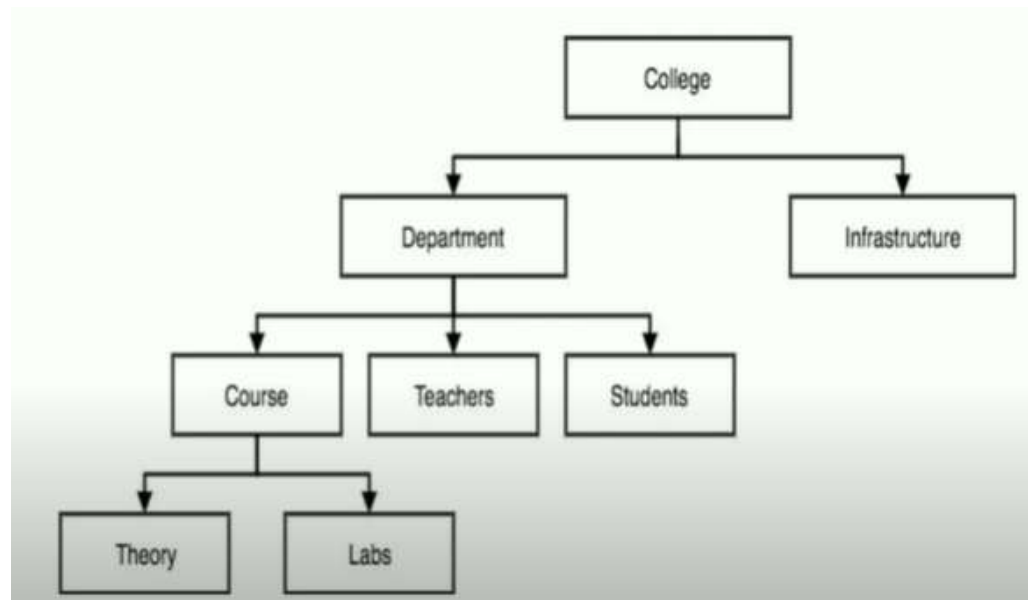


Data Modeling Techniques



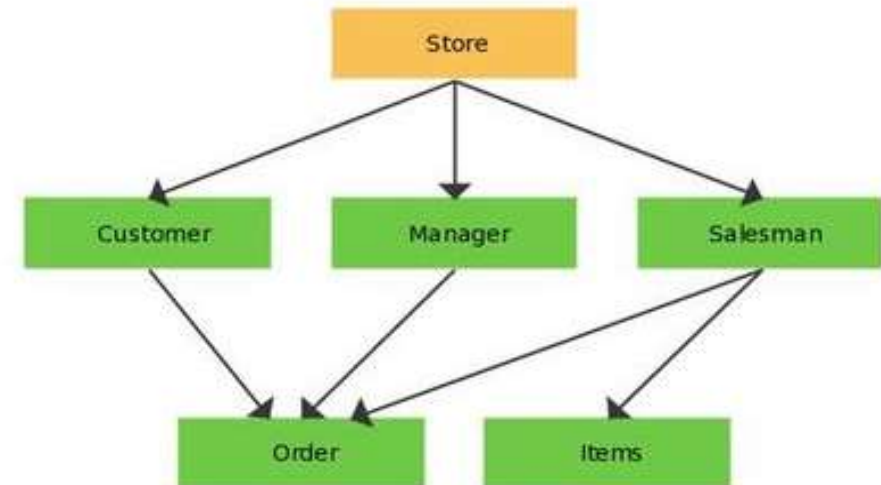
1. Hierarchical Model

- It has a root or a parent node and then follows a tree-like structure with other child nodes.
- Only one parent is allowed for a child node, and a parent node can have multiple child nodes.
- It is very easy and simple to understand.
- Simplex data models can be represented using this technique.



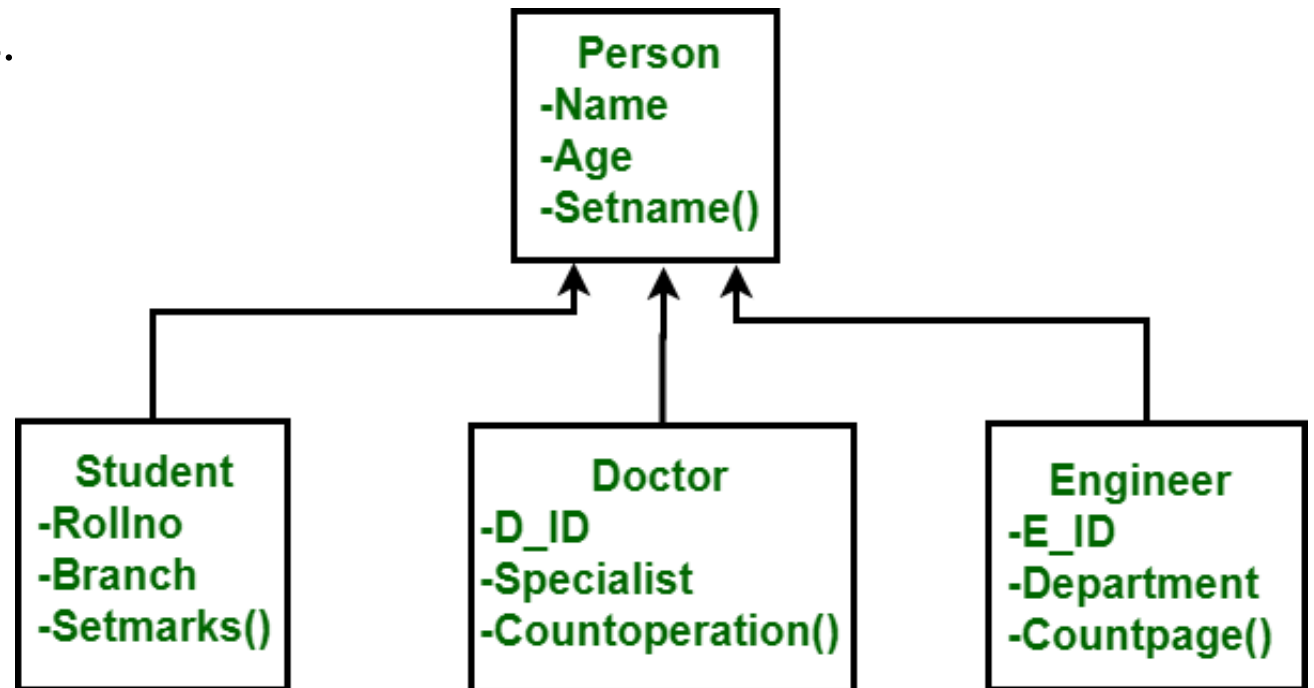
2. Network Model

- This model advance of hierarchical model. Show the Graph of system.
- Here, a child node could have multiple parent nodes.
- Navigation is faster in this model, as there are multiple paths to reach a child entity.
- This model also gets very complex when data is large.



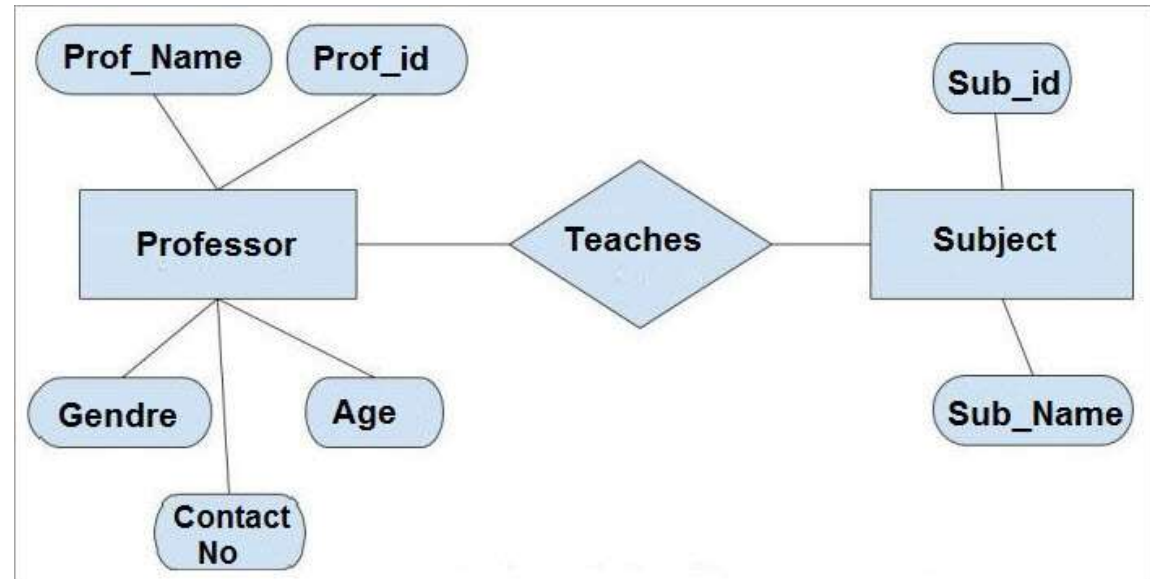
3. Object Oriented Model

- Object-oriented databases gained popularity with object-oriented programming.
- A real-world entity is represented as an object with attributes.
- The model presents a view as a collection of objects.
- Each object has its methods and features.



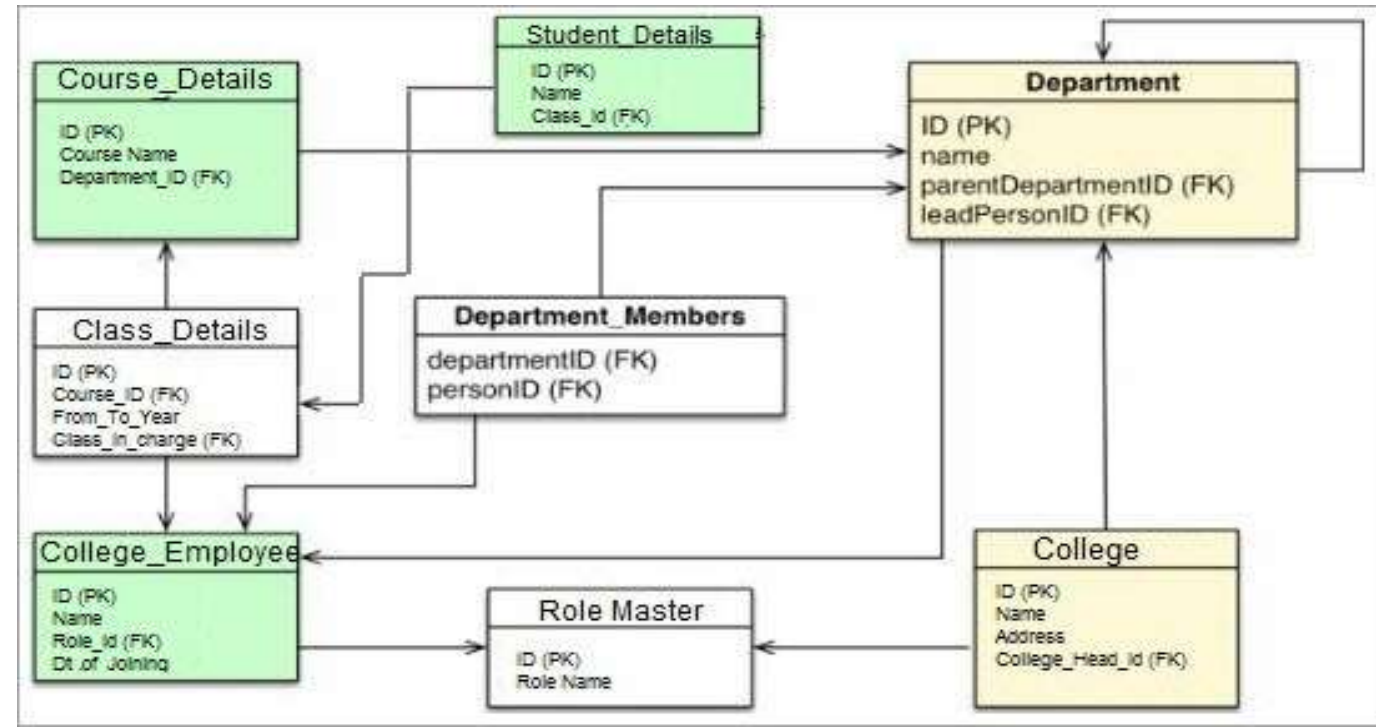
4. Entity Relationship Model

- This model represents the system at a very high level.
- Real-world problems can be represented through this model.
- It is very easy for all technical and non-technical stakeholders to understand.
- It is referred by database architects and developers to build the physical database.



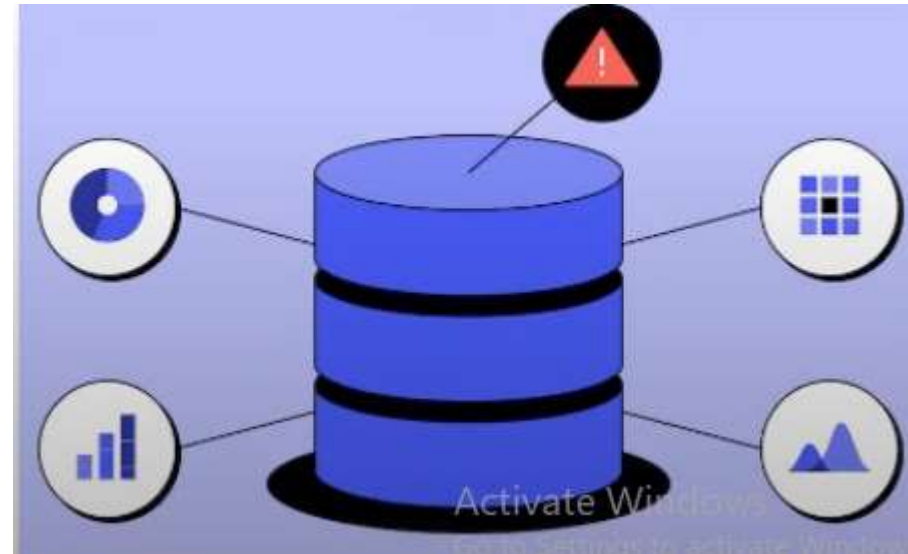
5. Relational Model

- The relational Model is the most popular model.
- In this model, the Entity information is represented as a table.
- All the attributes are represented as columns.
- The primary key and foreign key of each entity are also mentioned.



Data Modeling Tools

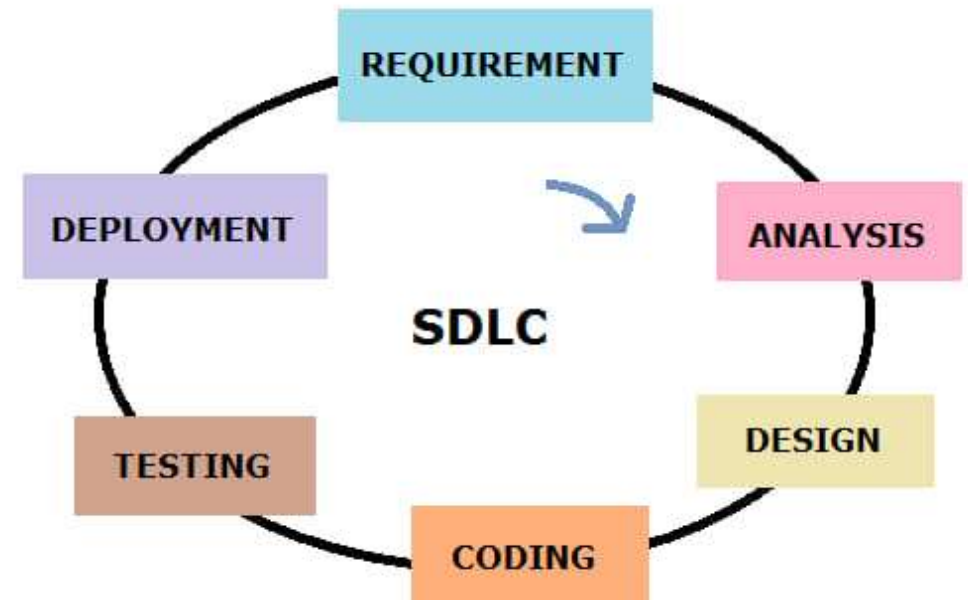
1. MySQL Workbench
2. ER/Studio
3. erwin Data Modeler
4. SQuirreL SQL Client
5. Draw.io
6. Lucidchart
7. Amundsen
8. Postico
9. Navicat
10. Datagrip



System Architecture and Design Overview

Software Design

- Software design is a process to transform user requirements into some suitable form, which helps the programmer in software coding and implementation.
- Software design is the first step in SDLC, which moves the focus from problem domain to solution domain.
- It tries to specify how to fulfill the requirements mentioned in SRS.
- Done by Software Design Engineer OR UI/UX Designer



Objectives of Software Design

- 1. Correctness:** Software design should be correct as per requirement.
- 2. Completeness:** The design should have all components like data structures, modules, and external interfaces, etc.
- 3. Efficiency:** Resources should be used efficiently by the program.
- 4. Flexibility:** Able to modify on changing needs.
- 5. Consistency:** There should not be any inconsistency in the design.
- 6. Maintainability:** The design should be so simple so that it can be easily maintainable by other designers.

Software Design Process

The software design process can be divided into the following **3 levels of phases of design**:

1. **Interface Design:** It focused on interaction between the system and the users devices. Scenario & Behavioral based diagram used. Not focus on internal structure.
2. **Architectural Design:** It focused on major components of a system their responsibilities, properties, interfaces, relationships and interactions between them. Class based & flow-based diagram used.
3. **Detailed Design:** It focused on internal elements of all major system components, their properties, relationships, processing, algorithms and data structures.

Software Quality Guidelines

1. A design is generated using the recognizable architectural styles and compose a good design characteristic of components and it is implemented in evolutionary manner for testing.
- 2 . A design of the software must be modular i.e the software must be logically partitioned into sub elements.
3. In design, the representation of data, architecture, interface and components should be distinct.
4. A design must carry appropriate data structure and recognizable data patterns.
5. Design components must show the independent functional characteristic.
6. A design creates an interface that reduce the complexity of connections between the components with external environment.
7. A design must be derived using the repeatable method.
8. The notations should be use in design which can effectively communicates its meaning.

Software Quality Attributes



- The attributes of design name as 'FURPS' are as follows:

1. **Functionality:** It evaluates the feature set and capabilities of the program.

2. **Usability:** It is accessed by considering the factors such as human factor, consistency and documentation.

3. **Reliability:** It is evaluated by measuring parameters like frequency and security of failure, output result accuracy, recovery from failure and program predictability.

4. **Performance:** It is measured by considering processing speed, response time, resource consumption, throughput and efficiency.

5. **Supportability:** It combines the ability to extend the program, adaptability, serviceability. Testability, compatibility and configurability are the terms using which a system can be easily installed and found the problem easily. Supportability also consists of more attributes such as modularity, reusability, robustness, security, portability, scalability.

Software Design Concepts

- The software design concept simply means the idea or principle behind the design.
- It describes how you plan to solve the problem of designing software.
- It also shows the logic or thinking behind how you will design software.
- The software design concept for developing the right software provides a supporting and essential structure or model.



Software Design Concepts

1. Abstraction
2. Architecture
3. Design Patterns
4. Modularity
5. Information Hiding
6. Functional Independence
7. Refinement
8. Refactoring
9. Object-Oriented Design Concept

1. Abstraction

- Abstraction is used to hide background details or unnecessary implementation about the data.
- So that users see only required information.

Type 1: Procedural Abstraction:

- There is collections of subprograms.
- One is hidden group another is visible group of functionalities.

Type 2: Data Abstraction:

- Collections of data that describe data objects.
- Show representation data & hide manipulation data.
- **Examples:** Data Structure Programs directly used Push(), Pop(), Top() and Empty() methods.

- **Example:**

- Private:

Fuel_machine() Set_top_speed()
Develop engine()

- Public: Turn on(). Turn off)
Accelerate(). Break()



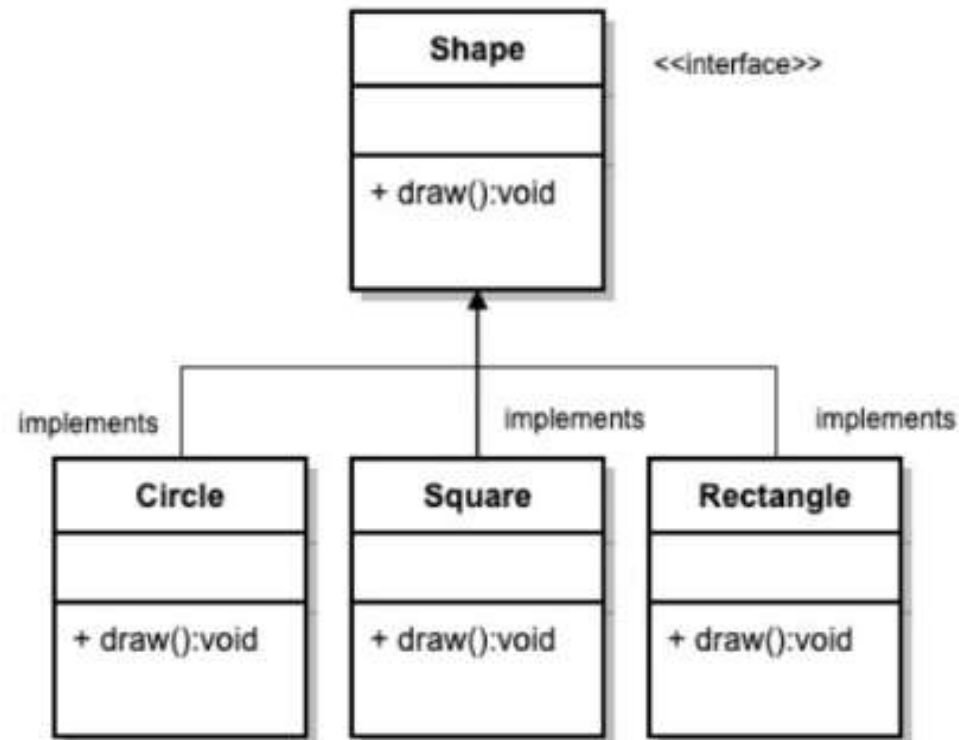
2. Architecture

The architecture is the structure of program modules where they interact with each other in a specialized way.

- **Structural Properties:** Architectural design represent different types of components. modules, objects & relationship between these.
- **Extra-Functional Properties:** How design architecture achieve requirements of Performance, Capacity, Reliability, Security. Adaptability & other System Characteristics.
- **Families of related systems:** The architectural design should draw repeatable patterns. They have ability to reuse repeatable blocks.

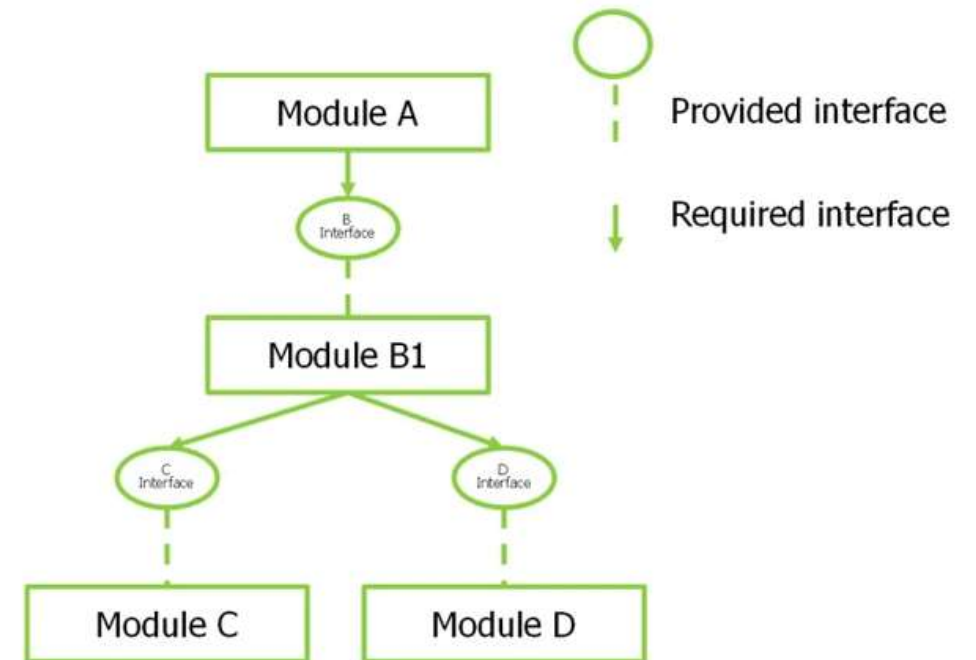
3. Design Patterns

- The pattern simply means a repeated form or design in which the same shape is repeated several times to form a pattern.
- Example:



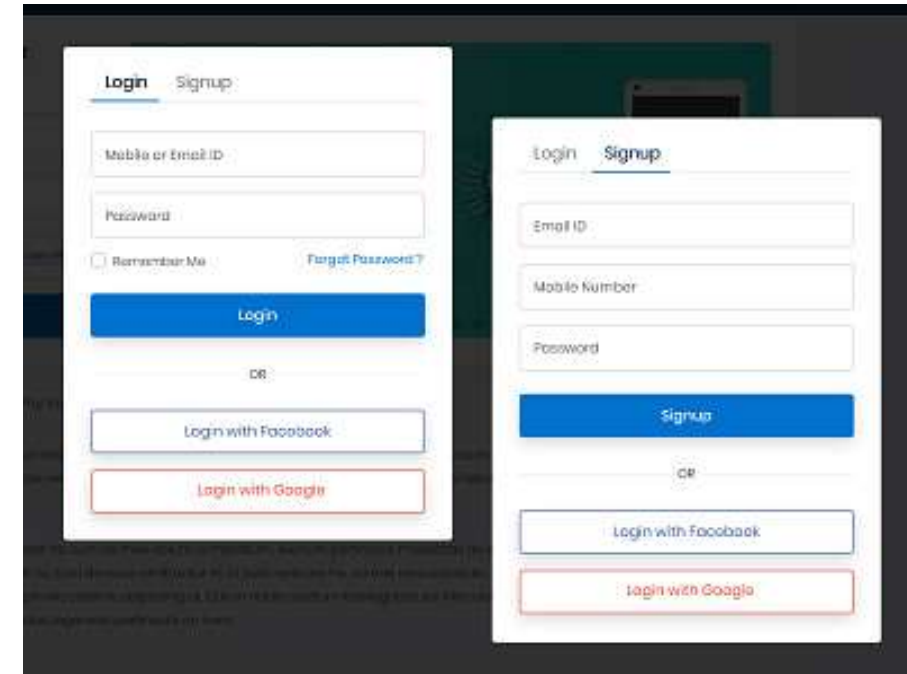
4. Modularity

- Modularity simply means dividing the system or project into smaller parts to reduce the complexity of the system or project.
- After developing the modules, they are integrated together to meet the software requirements.
- Modularizing a design helps to effective development, accommodate changes easily, conduct testing, debugging efficiently and conduct maintenance work easily.



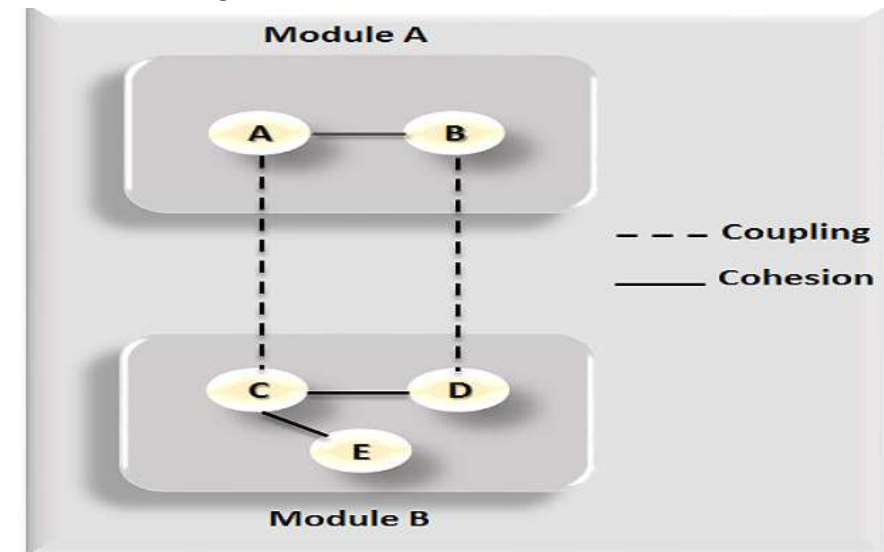
5. Information Hiding

- Modules should be specified and designed in such a way that the data structures and algorithm details of one module are not accessible to other modules.
- They pass only that much information to each other, which is required to accomplish the software functions.
- The way of hiding unnecessary details in modules is referred to as information hiding.



6. Functional Independence

- The functional independence is the concept of separation and related to the concept of modularity, abstraction and information hiding.
- **Criteria 1: Coupling**
 - The degree in which module is "connected" to other module in the system.
 - Low Coupling necessary in good software.
- **Criteria 2: Cohesion**
 - The degree in which module perform functions in inner module in the system.
 - High Cohesion necessary in good software.



7. Refinement

- Refinement is a top-down design approach.
- It is a process of elaboration.
- A program is established for refining levels of procedural details.
- A hierarchy is established by decomposing a statement of function in a stepwise manner till the programming language statement are reached.
- **Example:**

INPUT

Get number 1 (Integer)

Get number 2 (Integer)

PROCESS OUTPUT

INPUT

Get number 1 (Integer)

Get number 2 (Integer)

While (Invalid Number)

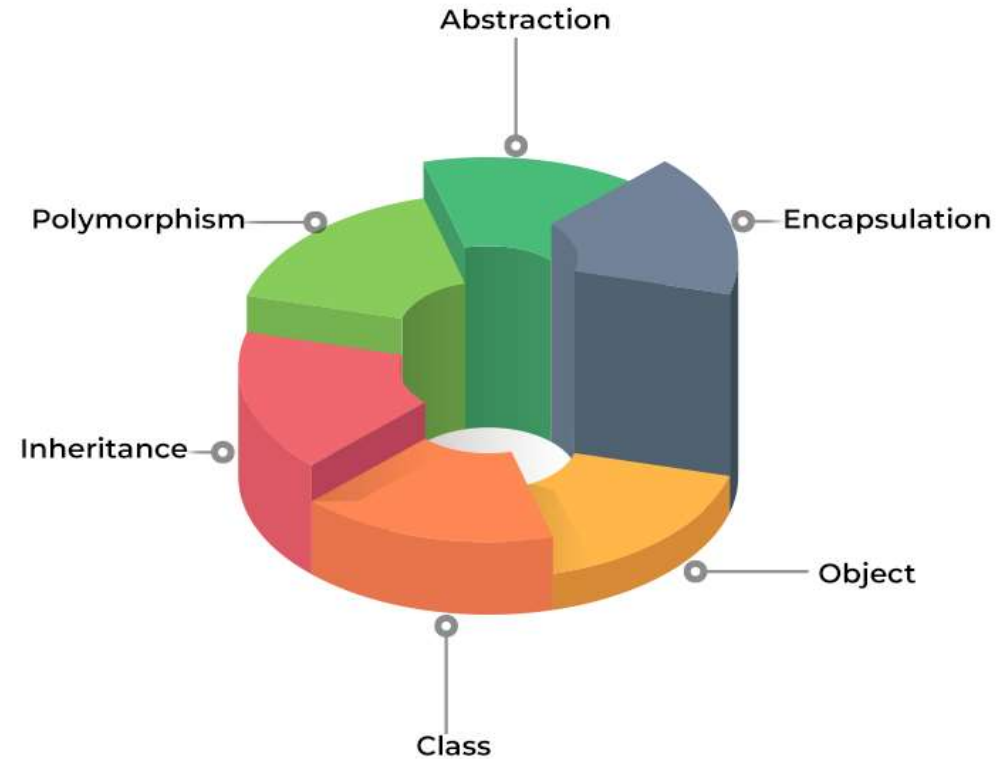
EXIT

8. Refactoring

- Refactoring is the process of changing the internal software system in a way that it does not change the external behavior of the code still improves its internal structure.
- When software is refactored, the existing design is examined for redundancy, unused design elements, unnecessary design algorithms, poorly constructed data, inappropriate data structure or any other design failure that can be corrected for better design.

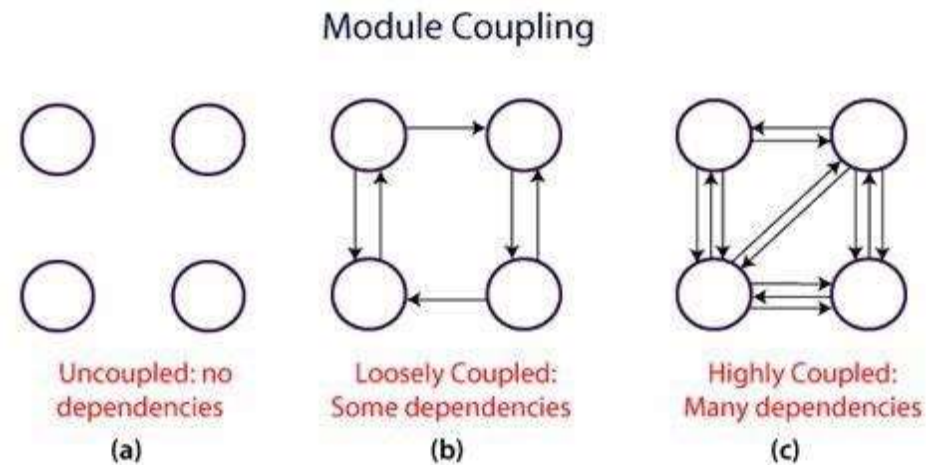
9. Object Oriented Design Concepts

- Object Oriented is a popular design approach for analyzing and designing an application.
- Advantage is that faster, low-cost development and creates a high-quality software.

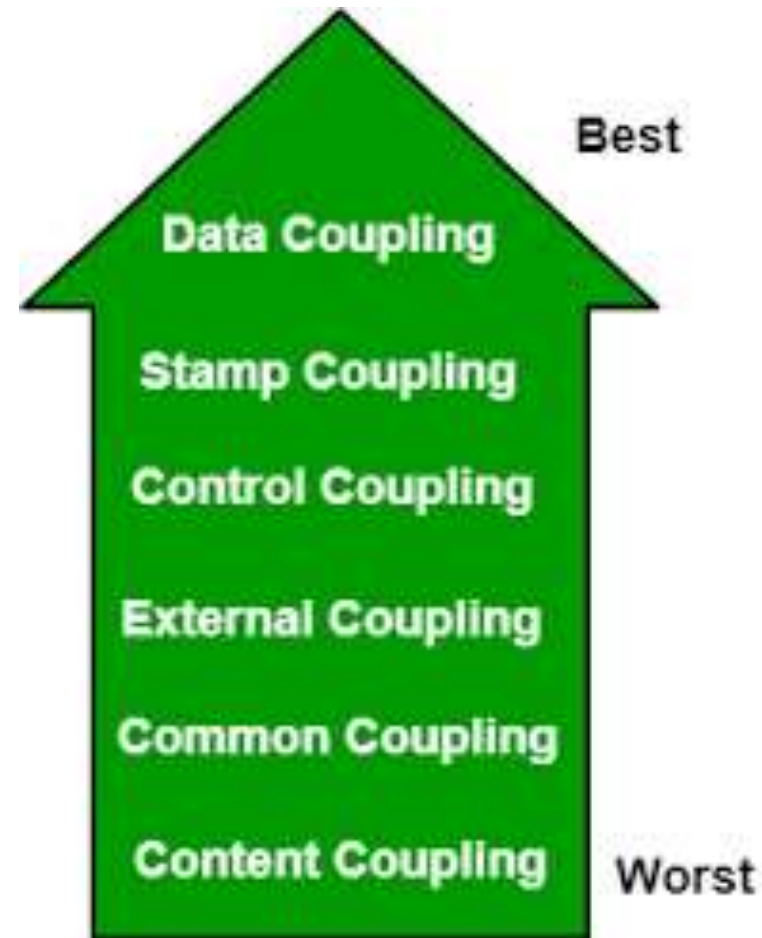


Coupling

- The coupling is the degree of interdependence or number of relations between software modules.
- Two modules that are tightly coupled are strongly dependent on each other.
- However, two modules that are loosely coupled are not much dependent on each other.
- A good design is the one that has Low coupling.
- High coupling generates more errors because they shared large number of data.



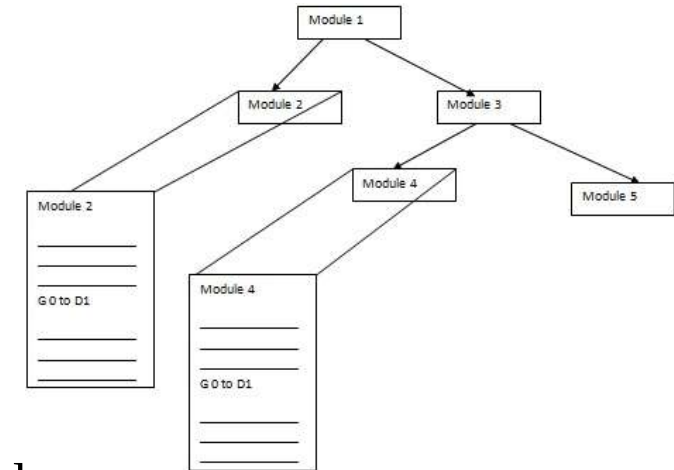
Types of Coupling



Types of Coupling

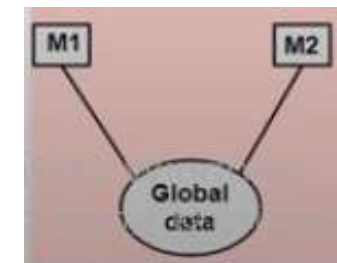
Type 1: Content Coupling

- Here, Two modules are connected as they share the same content like functions, methods.
- When a change is made in one module the other module needs to be updated as well.



Type 2: Common Coupling

- Two modules are common coupled if they share information through some global data items.



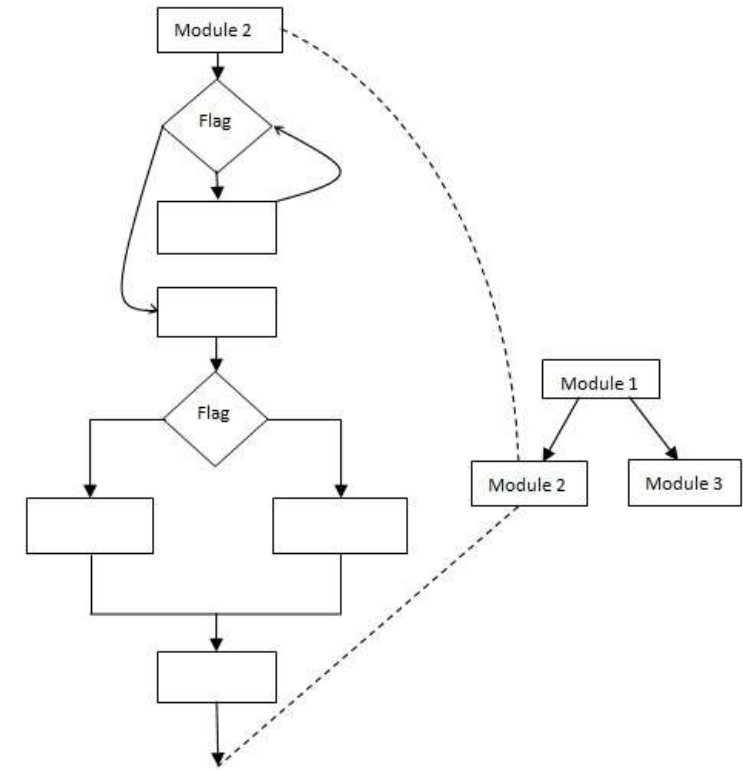
Types of Coupling

Type 3: External Coupling

- When two modules share an externally import data format, communication protocols or device interface.
- This is related to communication to external tools and devices.

Type 4: Control Coupling

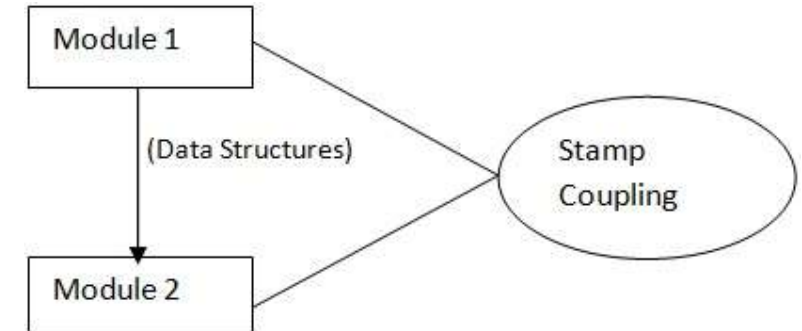
- Control coupling handle functional flow between software modules.
- Example: Module 1- Set Flag = 1 then only Module 2 perform action.



Types of Coupling

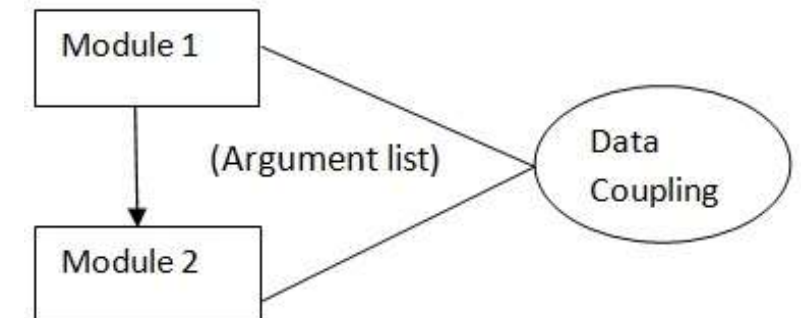
Type 5: Stamp Coupling

- Two modules are stamp coupled if they communicate using composite data items such as Complete Data Structure & objects.
- No junk or unused data shared between the two coupling modules.



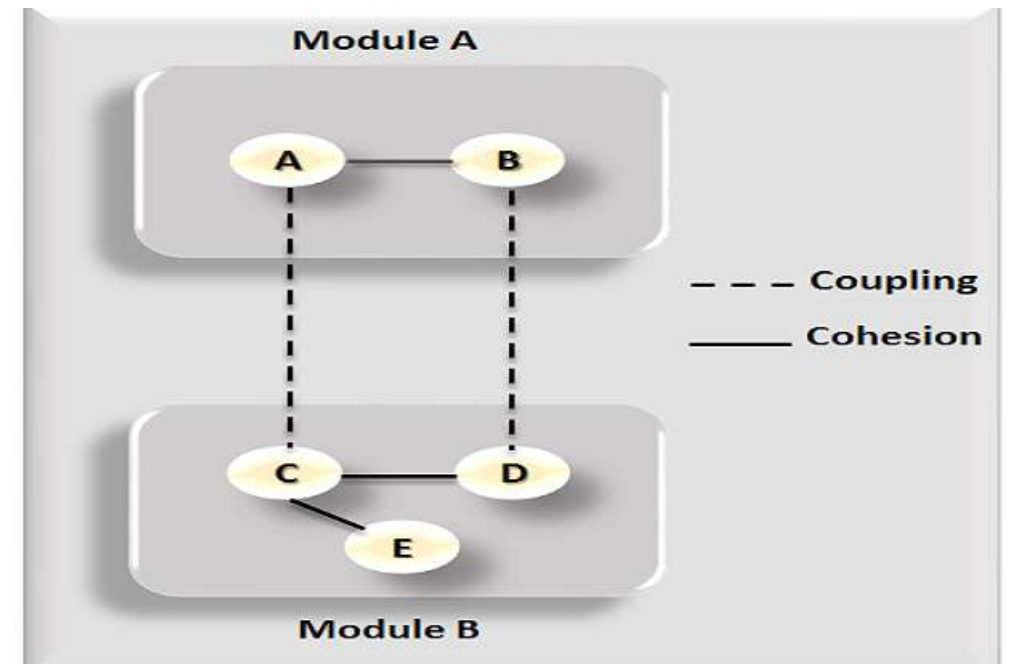
Type 6: Data Coupling

- When data are passed from one modules to another module via argument list or parameters through functional blocks.

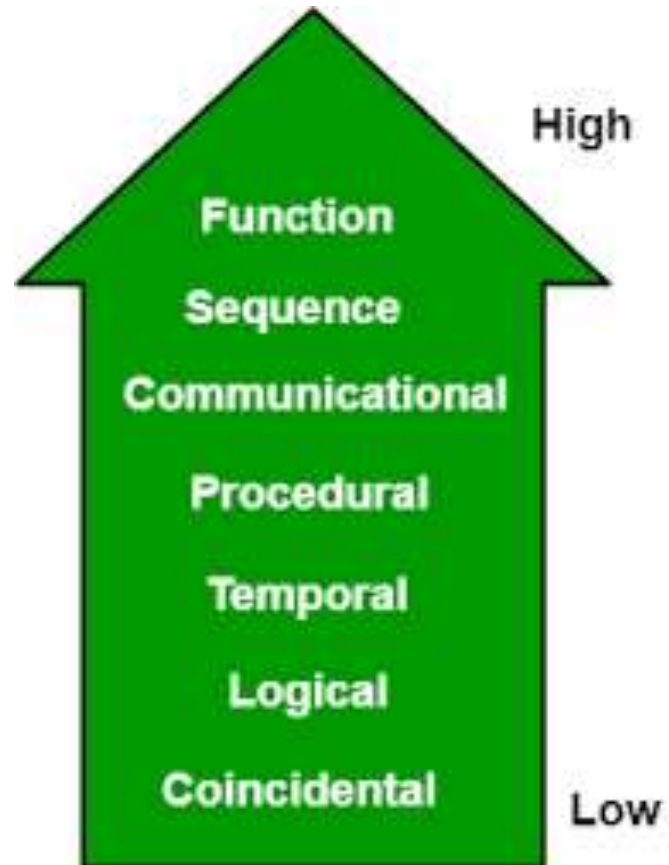


Cohesion

- Cohesion defines to the degree to which the elements of a module belong together or interrelated.
- Thus, cohesion measures the strength of relationships between pieces of functionality within a given module.
- A good software design will have high cohesion.



Types of Cohesion



Types of Cohesion

Type 1: Coincidental Cohesion

- It performs a set of tasks that are associated with each other very loosely.
- Example: Calculator: ADD, SUB, MUL, DIV

Type 2: Logical Cohesion

- If all the elements of the module perform a similar operation.
- Example: Error handling, Sorting, If Type of Record Student, then Display Student Record.

Type 3: Temporal Cohesion

- The activities related in time, Where all methods executed at same time. Temporal cohesion is found in the modules of initialization and termination. Example: Counter = 0, Open student file, Clear(). Initializing the array etc.

Types of Cohesion

Type 4: Procedural Cohesion

- All parts of a procedure execute in particular sequence of steps for achieving goal.
- Example: Calling one function to another function, Loop statements, Reading record etc.

Type 5: Communicational Cohesion

- If all the elements of a module are working on the same input & output data and are accessing that data through the same data structures.
- Example: Update record in the database and send it to the printer.

Type 6: Sequence Cohesion

- Output of one element treats as an input to the other elements inside the same module.
- Example: Enter the numbers -> Perform Addition of that numbers -> Display Addition.

Type 7: Function Cohesion

- If a single module aims to perform all the similar types of functionalities through its different elements. The purpose of functional cohesion is single minded, high, strong and focused.
- Example: Railway Reservation System

Design Models

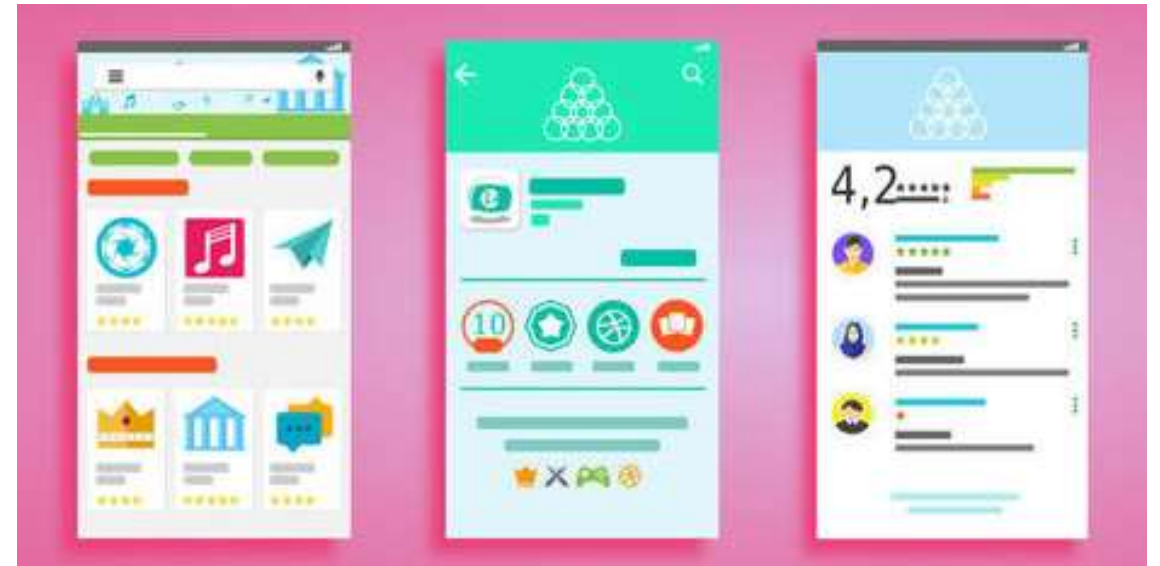
- The design phase of software development, transforming the customer requirements as described in the SRS documents into design forms.
- Designing a model is an important phase and is a multi-process that represent the data structure, program structure, interface characteristic and procedural details.
- **There are four types of design elements/models**
 1. Data Design Element / Model
 2. User Interface Design Element / Model
 3. Architectural Design Element / Model
 4. Component Level Design Element / Model

User Interface Design Model

- User interface is the front-end application view to which user interacts with the software.
- It determines how commands are given to the computer or the program and how data is displayed on the screen.

► **The software becomes more popular if its user interface is:**

1. Attractive
2. Simple to use
3. Responsive in short time
4. Clear to understand
5. Consistent on all interfacing screens



Architectural Design Model

- Architecture Design Model serves as a blueprint for a system.
- The architecture focuses on the early design decisions.

Architectural Design Styles describe:

- Set of hardware & software components that will perform a function required by the system.
Eg. Database, Modules, Frameworks etc.
- Set of connectors will help in coordination & communication between the components.
- Conditions that how components can be integrated to form the system.
- Semantic models that help the designer to understand the overall properties of the system.

Architecture Models

- Software architecture involves the high-level structure of software system abstraction, by using decomposition and composition, with architectural style and quality attributes. A software architecture design must conform to the major functionality and performance requirements of the system, as well as satisfy the non-functional requirements such as reliability, scalability, portability, and availability.
- A software architecture must describe its group of components, their connections, interactions among them and deployment configuration of all components.

A software architecture can be defined in many ways :

- **UML (Unified Modeling Language)** – UML is one of object-oriented solutions used in software modeling and design.
- **Architecture View Model (4+1 view model)** – Architecture view model represents the functional and non-functional requirements of software application.
- **ADL (Architecture Description Language)** – ADL defines the software architecture formally and semantically.

Importance of Software Architecture

- 1. Security:** The system is secured against malicious users by encryption or any other security measures due to layered software architecture.
- 2. Performance:** It handle request and response of the page in minimum time.
- 3. Maintainability:** Architectural design process uses easily modifiable and replaceable components which is easy to change them over time according to the new requirements.
- 4. Safety:** Avoid critical functionalities in small components & improve communication of the system.
- 5. Availability:** Architectural design process includes corresponding components, functionalities for handling the occurrence of any type of errors.

Decisions of Architectural Design

The architectural design process differs as the system differs depending upon the type of system being developed.

► There are some common decisions that should be taken care of in any design process.

- ☐ How can the system be distributed across the network?
- ☐ Which approach can be used to structure the system?
- ☐ Which architectural styles are suitable for the proposed system?
- ☐ How can software architecture be documented?
- ☐ How can the system be decomposed into modules?
- ☐ What control strategy must be used to control the operation of the components in the system?
- ☐ How can architectural design be analyzed?

UML

- UML stands for Unified Modeling Language.
- It is a pictorial language used to make software blueprints. UML was created by Object Management Group (OMG).
- It serves as a standard for software requirement analysis and design documents which are the basis for developing a software.
- UML can be described as a general-purpose visual modeling language to visualize, specify, construct, and document a software system.
- UML is generally used to model software system; it is not limited within this boundary. It is also used to model non software systems such as process flows in a manufacturing unit.
- The elements are like components which can be associated in different ways to make a complete UML picture, which is known as a **diagram**.
- So, it is very important to understand the different diagrams to implement the knowledge in real-life systems.
- We have two broad categories of diagrams, and they are further divided into sub-categories i.e., **Structural Diagrams** and **Behavioral Diagrams**.

Structural Diagrams

Structural diagrams represent the static aspects of a system. These static aspects represent those parts of a diagram which forms the main structure and is therefore stable.

These static parts are represented by classes, interfaces, objects, components and nodes. Structural diagrams can be sub-divided as follows :

1. **Class diagram:** Represents the object orientation of a system. Shows how classes are statically related.
2. **Object diagram:** Represents a set of objects and their relationships at runtime and also represent the static view of the system.
3. **Component diagram:** Describes all the components, their interrelationship, interactions and interface of the system.
4. **Deployment diagram:** Deployment diagrams are a set of nodes and their relationships. These nodes are physical entities where the components are deployed.
5. **Package diagram:** Describes the package structure and organization. Covers classes in the package and packages within another package.
6. **Composite structure:** Describes inner structure of component including all classes, interfaces of the component, etc.

Behavioral Diagrams

Behavioral diagrams basically capture the dynamic aspect of a system. Dynamic aspects are basically the changing/moving parts of a system.

UML has the following types of behavioral diagrams :

1. **Use case diagram:** Describes the relationships among the functionalities and their internal/external controllers. These controllers are known as actors.
2. **Sequence diagram:** Visualizes the sequence of calls in a system to perform a specific functionality.
3. **Communication diagram:** Same as sequence diagram, except that it focuses on the object's role. Each communication is associated with a sequence order, number plus the past messages.
4. **State chart diagram:** Represents the event driven state change of a system. It basically describes the state change of a class, interface, etc. Used to visualize the reaction of a system by internal/external factors.
5. **Activity diagram:** Describes the flow of control in a system. It consists of activities and links. The flow can be sequential, concurrent, or branched.
6. **Interaction overview:** Combines activity and sequence diagrams to provide a control flow overview of system and business process.
7. **Time sequence diagram:** Describes the changes by messages in state, condition and events.

Architecture View Model

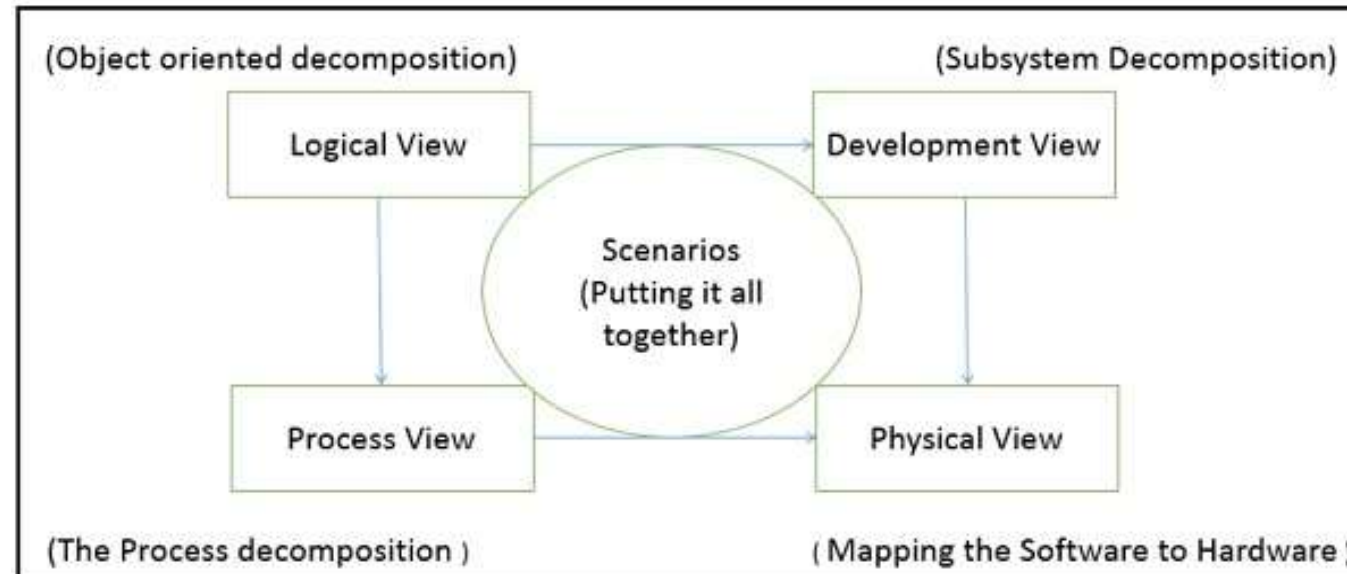
- A model is a complete, basic, and simplified description of software architecture which is composed of multiple views from a particular perspective or viewpoint.
- A view is a representation of an entire system from the perspective of a related set of concerns.
- It is used to describe the system from the viewpoint of different stakeholders such as end-users, developers, project managers, and testers.

4+1 View Model

- The 4+1 View Model was designed by Philippe Kruchten to describe the architecture of a software-intensive system based on the use of multiple and concurrent views.
- It is a **multiple view** model that addresses different features and concerns of the system. It standardizes the software design documents and makes the design easy to understand by all stakeholders.
- It is an architecture verification method for studying and documenting software architecture design and covers all the aspects of software architecture for all stakeholders.
- **It provides four essential views :**
 - **The logical view or conceptual view** – It describes the object model of the design.
 - **The process view** – It describes the activities of the system, captures the concurrency and synchronization aspects of the design.
 - **The physical view** – It describes the mapping of software onto hardware and reflects its distributed aspect.
 - **The development view** – It describes the static organization or structure of the software in its development of environment.

4+1 View Model

- This view model can be extended by adding one more view called **scenario view** or **use case view** for end-users or customers of software systems.
- It is coherent with other four views and are utilized to illustrate the architecture serving as “plus one” view, (4+1) view model.
- The following figure describes the software architecture using five concurrent views (4+1) model.



Why is it called 4+1 instead of 5?

- The **use case view** has a special significance as it details the high-level requirement of a system while other views details ,how those requirements are realized.
- When all other four views are completed, it's effectively redundant. However, all other views would not be possible without it.

Architecture Description Languages (ADLs)

- An ADL is a language that provides syntax and semantics for defining a software architecture.
- ADLs must support the architecture components, their connections, interfaces, and configurations which are the building block of architecture description.

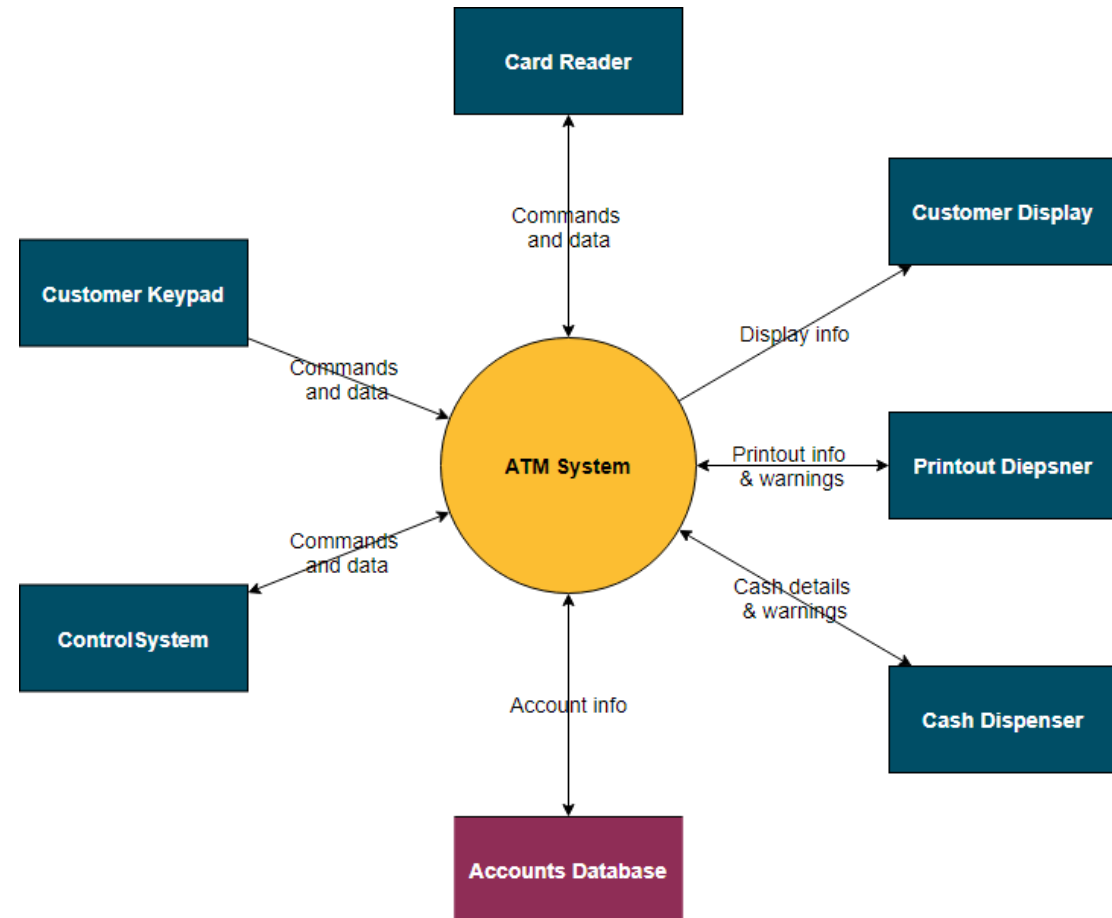
following requirements for a language to be classified as an ADL :

- It should be appropriate for communicating the architecture to all concerned parties.
- It should be suitable for tasks of architecture creation, refinement, and validation.
- It should provide a basis for further implementation, so it must be able to add information to the ADL specification to enable the final system specification to be derived from the ADL.
- It should have the ability to represent most of the common architectural styles.
- It should support analytical capabilities or provide quick generating prototype implementations.

Taxonomy of Architectural styles

Architectural styles is establishing a complete structure & components of all over the system.

1. Data Centered Architecture
2. Data Flow Architecture
3. Object Oriented Architecture
4. Layered Architecture



Data Centered Architecture

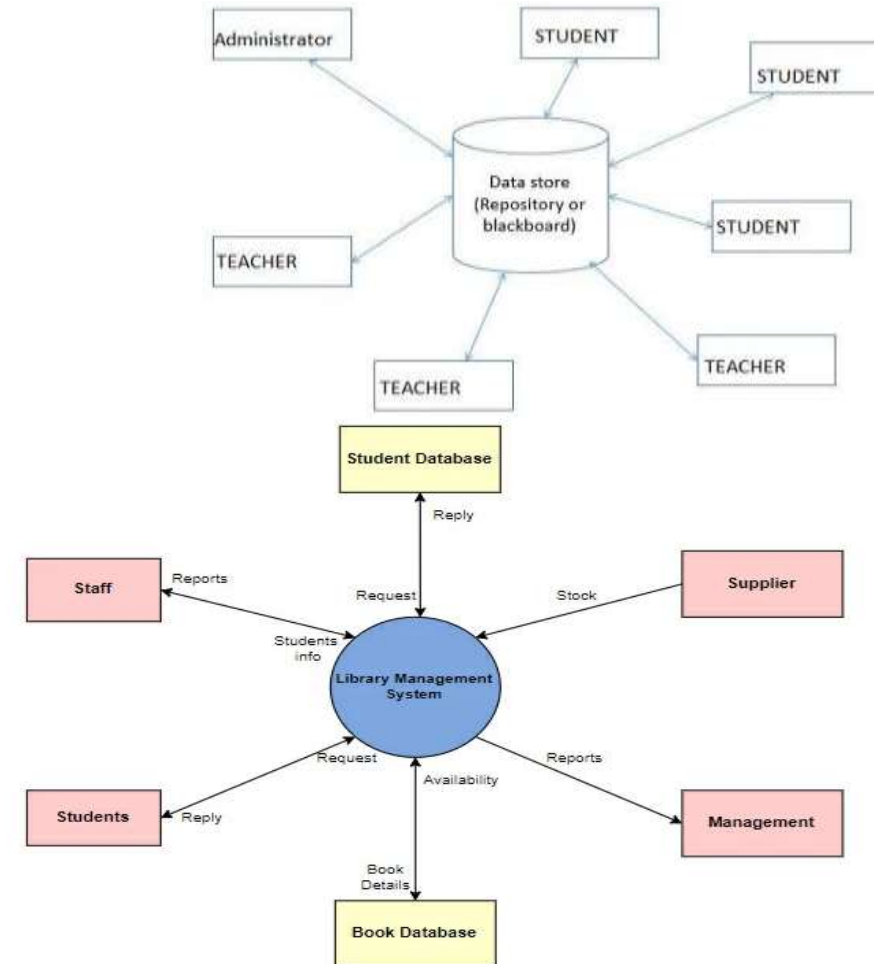
- Data store at the center of this architecture and is accessed frequently by everyone.
- Update, add, delete or modify the data present within the store.
- It is widely used in DBMS, Library Information System etc.

Advantages:

- 1. Repository of data is independent of clients
- 2. It may be simple to add additional clients.
- 3. Modification can be very easy.

Disadvantages:

- 1. Data replication or duplication is possible.
- 2. Changes in data structure highly affect the clients.



Data Flow Architecture

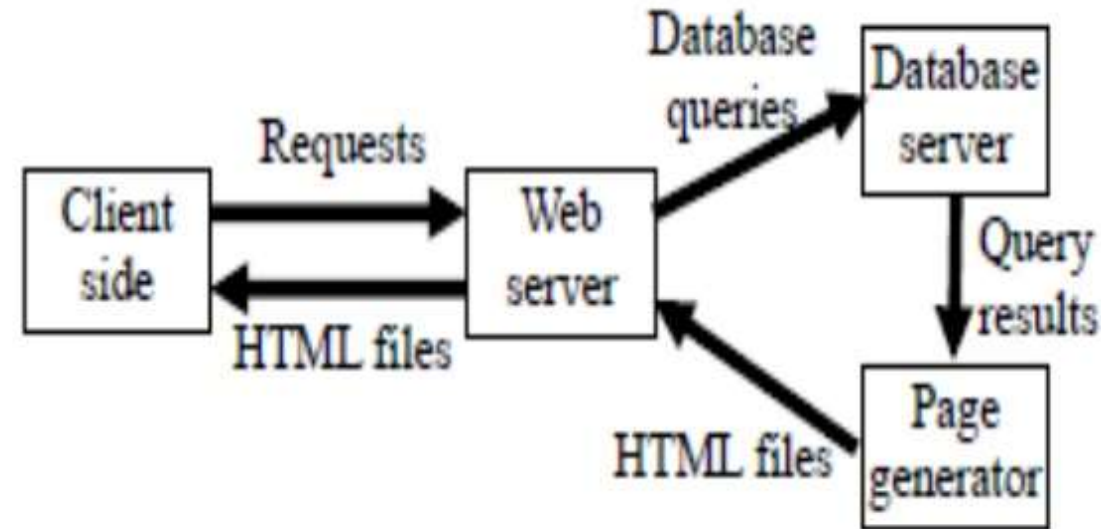
- This architecture is used when input data to be transformed into output data through a series of computational manipulative components.
- **Pipe** is a connector which passes the data directionally from one filter to the next.
- **Filter** is a component reads the data from its input pipes and performs its function.
- This data and places the result on all output pipes.

Advantages:

- 1. With this design, concurrent execution is supported.

Disadvantages:

- 1. Does not allow greater user engagement.

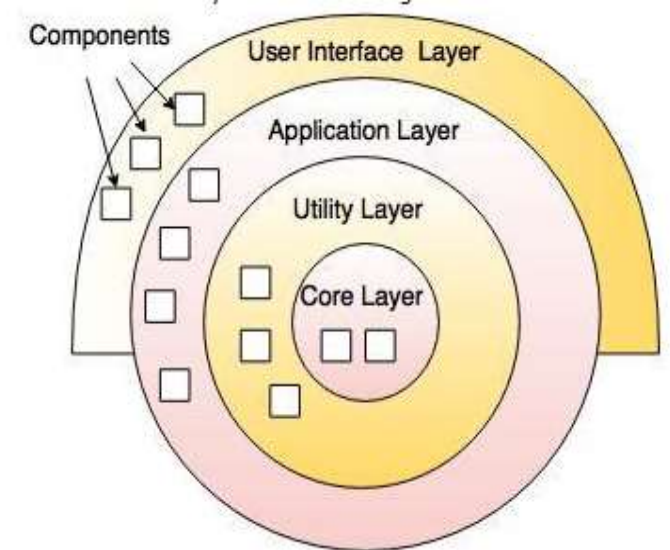
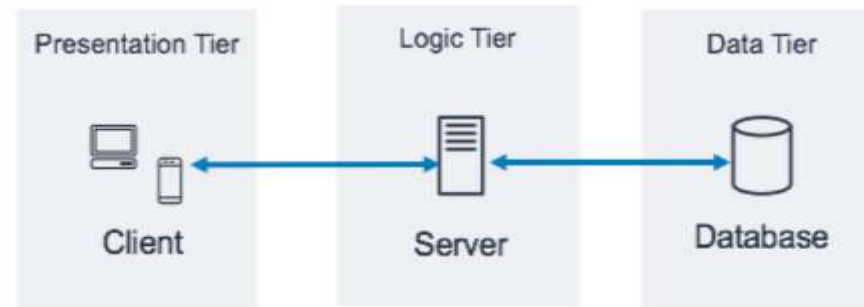


Object Oriented Architecture

- Objects are the foundational building blocks for all kinds of software applications.
1. **Object:** Object is an instance of a class. Example Student S, Person;
 2. **Class:** It defines all attributes, methods, which represents the functionality of the object.
 3. **Encapsulation:** It is the process of binding similar types of elements of an abstraction.
 4. **Abstraction:** It is the removal of irrelevant essentials from users.
 5. **Inheritance:** It deriving a new class from existing one. It increases code reusability.
 6. **Polymorphism:** It has multiple forms. Ex: draw graphic objects circle, rectangle, triangle
 7. **Message Passing:** Sending and receiving data among objects through function parameters.

Layered Architecture

- Data moves from one level to another level for processing is called layered architecture.
- Number of different layers are defined every layer performing well-defined set of operations.
- **Outer layer** components manage the user interface operations.
- **Inner layer** components will perform the operating system interfacing.
- **Intermediate layers** provide utility services and application software functions.
- Example: E-commerce web applications development like Amazon.



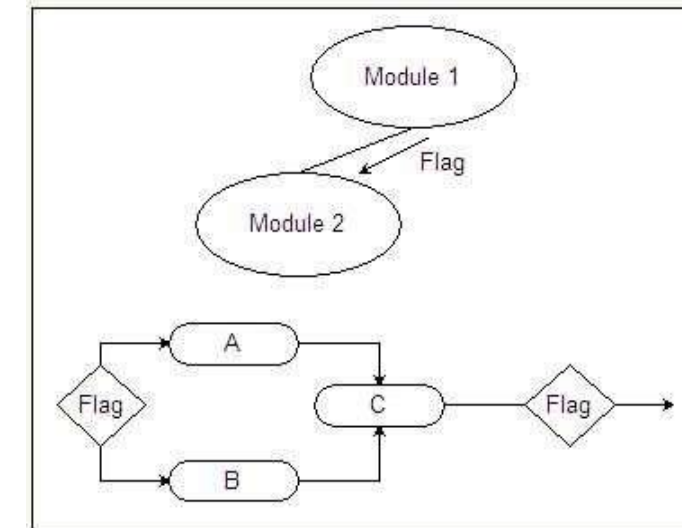
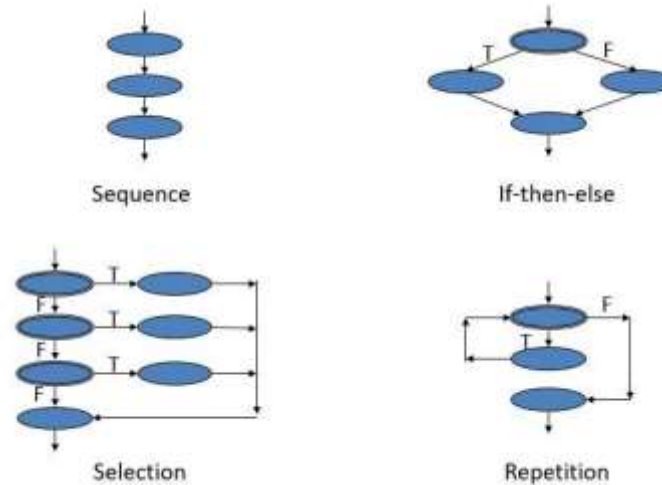
Architectural Views

- It is generally used to represent entire architecture that is useful and meaningful to one or more stakeholders in system.
1. **Use case View:** Users view to handling product as per requirement. Use case diagram used.
 2. **Design View:** Organize design information, significant features, entities and attributes.
 3. **Process View:** Describe communication, behavior & synchronization aspects of the design.
 4. **Implementation View:** It address source code integrators and developers of project.
 5. **Deployment View:** It describes & explains environment within system runner & executed.

Component Level Design Model

► What is Component?

- A component is a modular, portable, replaceable and reusable set of well-defined functionalities.



► What is Component Level Design?

The component-based architecture focuses on breaking the software design into small individual modules.

The component design describes communication, interfaces, algorithms & functionalities of each component regarding the whole software design.

Component Level Design notations are Activity Diagram, Data Flow Diagram, Conditional Notations & Tabular forms.

Benefits of Component Design

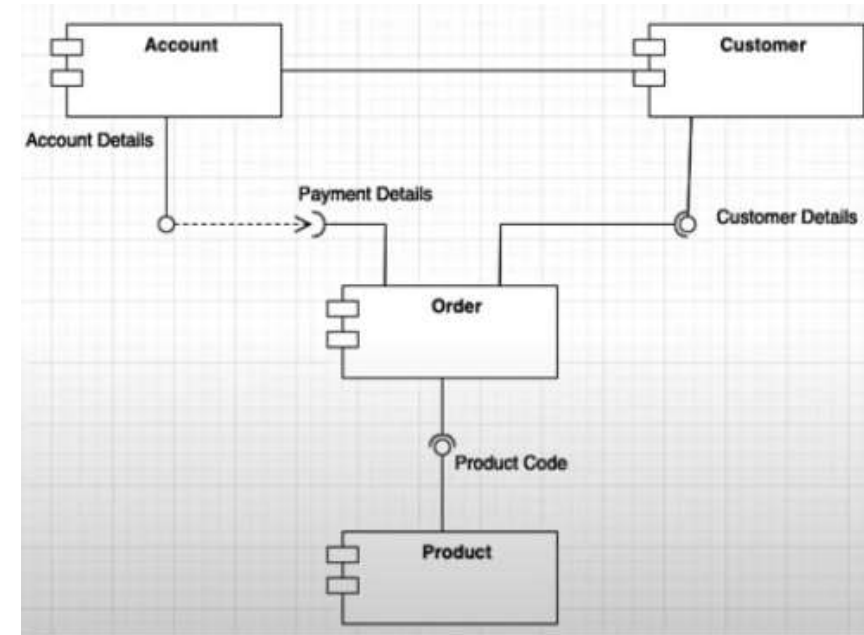
1. The major benefit of using the Component design is that it makes every module reusable.
2. It also reduces the cost as every module is reusable.
3. It is more reliable as the client gets its interaction with the system with each module, so it increases the reliability of the entire system.
4. The component design makes the system easy to maintain as we don't need to make changes in the entire system.
5. The component design describes each component's functionality more clearly.

Steps of Component Level Design

1. Identify all design classes that corresponds to problem domain.
2. Identify all design classes that corresponds to infrastructure domain. **Ex.** GUI Class, Database management class, OS Communication class etc.
3. Elaborate all design classes that are not acquired as reusable component. **Ex.** Coupling & Cohesion methods like message details between classes, data type, data structure & data flow between classes.
4. Describe persistent data source & identify classes require to manage them. **Ex.** Classes which data from databases.
5. Develop behavioral representation for a component & class. **Ex.** Classes, Operations & their Interactions through Activity & State diagram.
6. Elaborate deployment diagram. **Ex.** Hardware, OS, Server of components.
7. Factors every component level design representations. **Ex.** Diff. design solutions that provide good characteristics.

Component Design Example

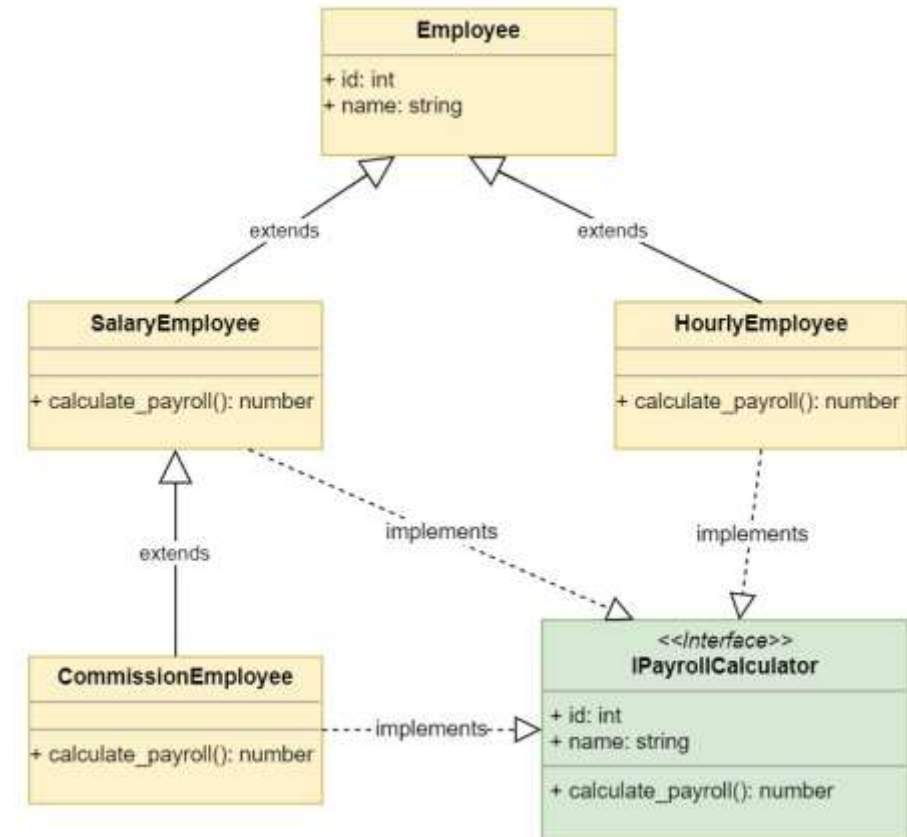
- It has Account, Customer, Order & Product this Four Components.
 - Every object is an individual component but is related to each other at the same time.
1. Customer will go online site and search product.
 2. By selecting product, the customer will place an order.
 3. System will store customer & his account information in Order Database.
 4. The order would be generated against the customer and stored in the Database.



Component Design View

1. Object Oriented View:

- An object-oriented view is a set of collaborating classes in module mentioned in architectural design
- It view all the objects, methods, properties and functionalities mentioned in classes.
- It also view how classes in each module are connected with each other.
- It detailed out data structure & algorithms required for processing.



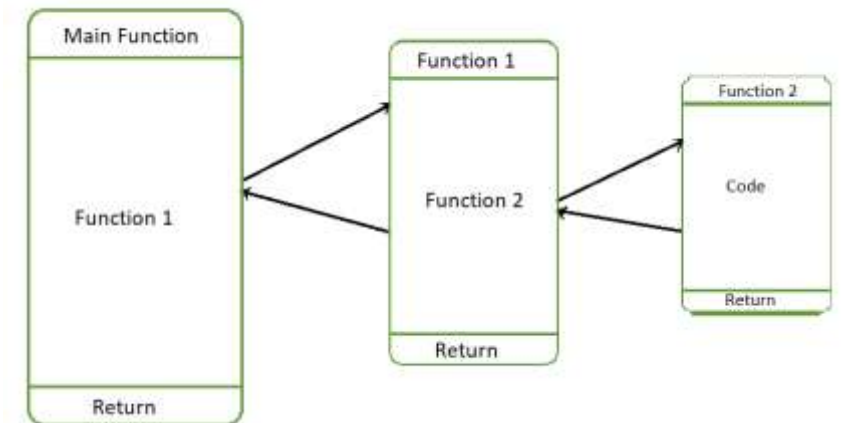
Component Design View

2. Conventional View:

- It is viewed functional element of a program that integrates the all logical & internal operations performed in class
- It also view calling and coordinating the request like calling functions, parameter passing, data passing between the of other modules.
- They analyze complete view of data flow model & state diagram.

3. Process-related View:

- The process-related view majorly focuses on the reusability feature of the component design.
- Databases and libraries are used to store pre-existed modules.
- This pre-existed data used for creating new modules.



Project Planning

Before starting a software project, it is essential to decide which tasks to be performed and how to manage all tasks involved in the software development.

- ✓Project planning is an organized process from requirement gathering to testing and support.
- ✓Focuses on all activities required for successful completion of the project.
- ✓Prevents obstacles that arise in the project such as changes in projects, organization's objectives, non-availability of resources and so on.
- ✓Helps in better utilization of resources and optimal usage of the allotted time for a project.
- ✓Defines roles and responsibilities of the project management team members.



Software Project Manager

- **Managing People :**
 - Act as Project Leader.
 - Contact with all stakeholders.
 - Managing human resources.
- **Managing Project :**
 - Defining and setting up project scope.
 - Managing project management activities.
 - Monitoring progress and performance.
 - Risk analysis at every phase.
 - Take necessary step to avoid or come out of problems.



Project Planning Process



Project Planning Process

1. Identify Stakeholders Need:

- A project must meet the expectations of stakeholders & meet all of their needs to be effective.
- To consider their needs a project planner might identify the characteristics and qualities of the project at hand.

2. Identify Project Objectives:

- Project objectives must be specific, measurable, achievable, realistic and time-bound in order to be used to validate project success.
- All team members must contribute ideas and compromise on collective goals for the project.



Project Planning Process



3. Identify Deliverables & Due Date:

- A due date is a scheduled fixed date and time that an objective is due within a project.
- Deliverables are defined as the products, services or results that project quality, size, length, or any other standard that proves to be important in the success of project.

4. Identify Project Schedules:

- This stage is created through different tasks that label a project's start and end date.
- Schedules are used to make sure projects stay on task and are completed in a timely manner.



Project Planning Process

5. Provide Roles & Responsibilities:

- This stage allows for effective communication between everyone involved in the project.
- This segment includes identifying who is involved and what their individual tasks are.
- Everyone understand expected objectives, goals & output of project.

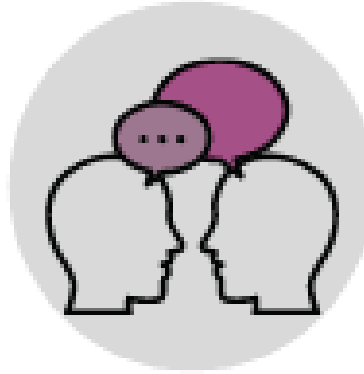


6. Identify Project Budget:

- Project budget are calculating the anticipated cost, changing budget cost and monitoring the budget.
- Therefore, it is crucial to prepare for unexpected alterations of project costs.



Project Planning Process



7. Identify Communication Plan:

- A communication plan is a tool that able to effectively communicate about a project to your client, team and other stakeholders.
- Communication plan has clear guidelines about how information will be shared & who is responsible within the project.

8. Provide Tracking & Management:

- It is an effective way to deliver projects on time and organize tasks.
- Features of this element include planning/scheduling, collaboration, documentation and evaluation.
- Management tools are track productivity and growth of project.



Recourses Used in Project Development

Project resources simply mean resources that are required for successful development and completion of project on time and on budget.

Type 1: Human Recourses:

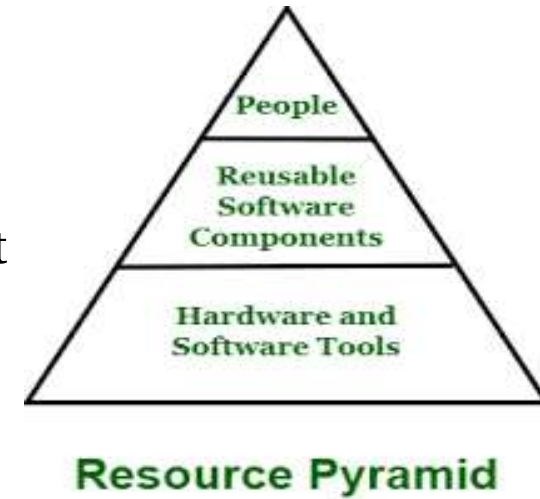
- In software industry, manager, software developer, software testing and so on.
- These positions are according to their skills and specialty.

Type 2: Reusable Components:

- Managing budget for project is one of most important tasks that all project managers. Resource Pyramid
- The reusable resources are very helpful as they help in reducing overall cost of development.

Type 3: Hardware and Software tools:

- It should be planned before starting development of project otherwise it way causes problems.
- These are actually material resources that are part of project.



Project Scope Management

- Scope refers to the detailed set of deliverables or features of a project.
- This includes all the objectives, activities, process, output, deadline need to be done in order to make a deliverable software product.
- Scope management is essential because it creates boundaries of the project by clearly defining what would be done in the project and what would not be done.



Real Life Example of Project Scope Management

- 'I need a new house', & no one has any idea about functionality of the house at this point.
- Should it be Apartment or an Independent house?
- How many rooms, swimming pool, car park, security, air conditioning the scope gets elaborated as the project progresses.
- Scope which has major impact on the architecture, budget of the project need to be fixed before we commence the detailed planning.
- The purpose of 'Project scope management' is to ensure that the project includes all the work required and exclude all the unwanted work during the beginning till the end of the project.



Steps of Project Scope Management



Project Scope Management



Step 1: Plan Scope Management

- Documentation & guidelines of Project Scope, Product Scope, Project Life Cycle, Requirement management Plan & Scope management plan etc.
- This process provides guidance and direction for managing scope across the project.
- How to define, Validate and control the project scope.

Step 2: Collect Requirements

- Collect requirements from all stakeholders who you have identified as impact on the project.
- Collect Business, Stakeholder, Product, Transition, Quality requirements.
- Collect requirements from Interviews. Brainstorming, Questionnaires', Surveys, Group Discussion, Voting & Prototype etc.



Project Scope Management

Step 3: Defining Scope

- Define the project scope by identifying Project objectives, Goals, Sub-phases. Tasks. Budget Resources & Schedule.
- Identify Human Recourses like Inter-personnel and team skills.
- This ensures that the client, stakeholders, senior management, project manager & team members are all aware of what is expected.

Step 4: Create WBS

- The Work Breakdown Structure involves subdividing project deliverables into smaller units.
- Break down project into phases, including the priority tasks required in order to complete each phase.

Project Scope Management

Step 5: Verify Scope

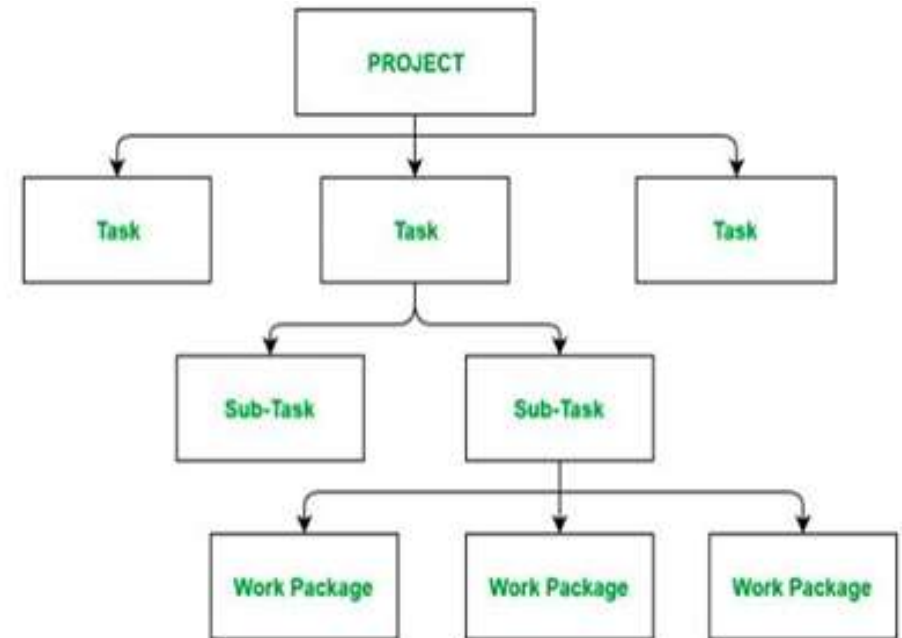
- The Validate Scope process focuses mainly on customer acceptance.
- It is when the project customer formally accepts all project deliverables & end of each phase.
- During the process, the customer gives feedback on the work that was performed.

Step 6: Control Scope

- It involves monitoring the status of the project and managing changes in scope.
- Efficient in dealing with Time & Cost management.
- Calculate how does it impact the project and its process.

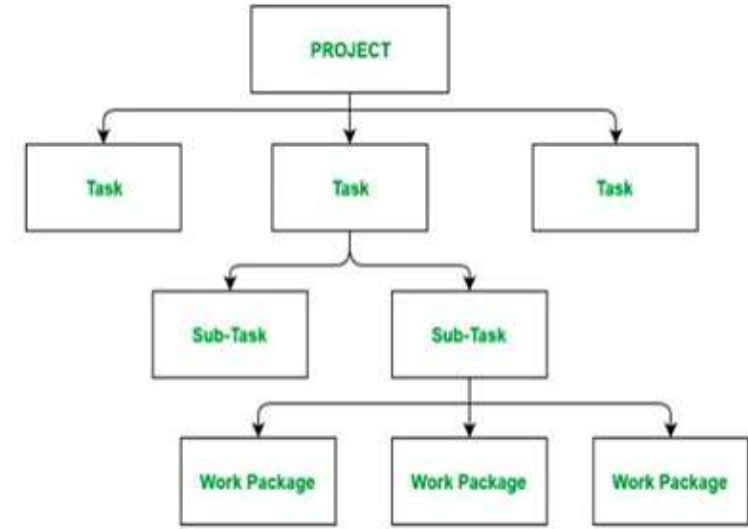
Work Breakdown Structure

- A Work Breakdown Structure includes dividing a large and complex project into simpler, manageable and independent tasks.
- It used to organize and plan projects, programs and portfolio.
- It follows a Top-Down & Hierarchical view approach.
- The root of this tree is labelled by the Project name itself.
- Perform by Project Manager & Subject Matter Expert.



Working of Work Breakdown Structure

- **Step 1:** Project managers decide project name at top/root of WBS.
 - **Step 2:** Project managers identifies the main deliverables of the project.
 - **Step 3:** These main deliverables are broken down into smaller higher-level tasks.
 - **Step 4:** This complete process is done recursively to produce much smaller independent tasks.
 - **Step 5:** Choose Task Owner. They need to get the job done.
-
- Depends on project manager that up to which level of detail they want to break down project.
 - Lowest level tasks are simplest & independent tasks it takes less than weeks of work.



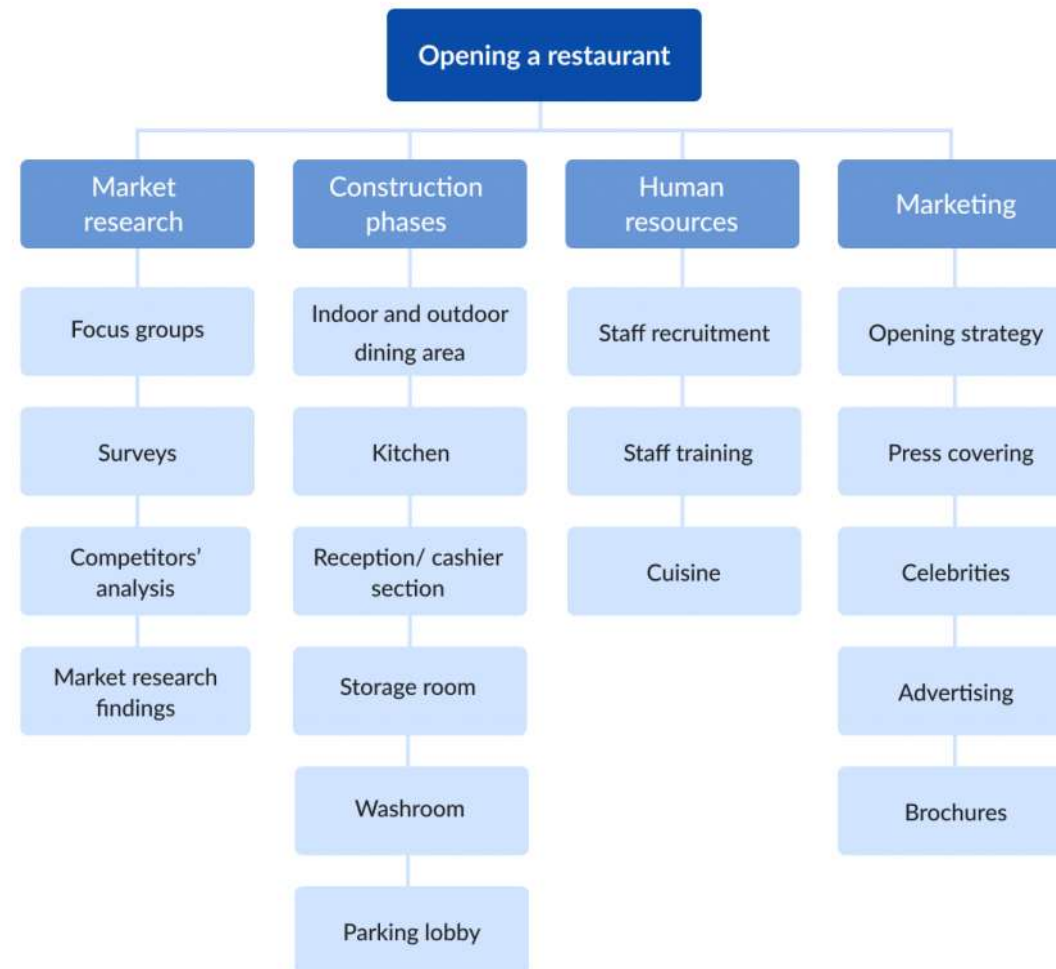
Components of Work Breakdown Structure

1. **WBS Dictionary:** Document that defines the various WBS elements.
2. **WBS Levels:** Determines the hierarchy level of a WBS element.
3. **Task:** Main deliverable tasks. Status, Description. Task owner, Dependencies and Duration.
4. **Sub Task:** Divided tasks of Main deliverable tasks.
5. **Work Packages:** Lowest level of WBS. Small group of tasks.
6. **Control Account:** Group work packages and measure their status.
7. **Project Deliverables:** Desired outcome of project tasks and work packages.

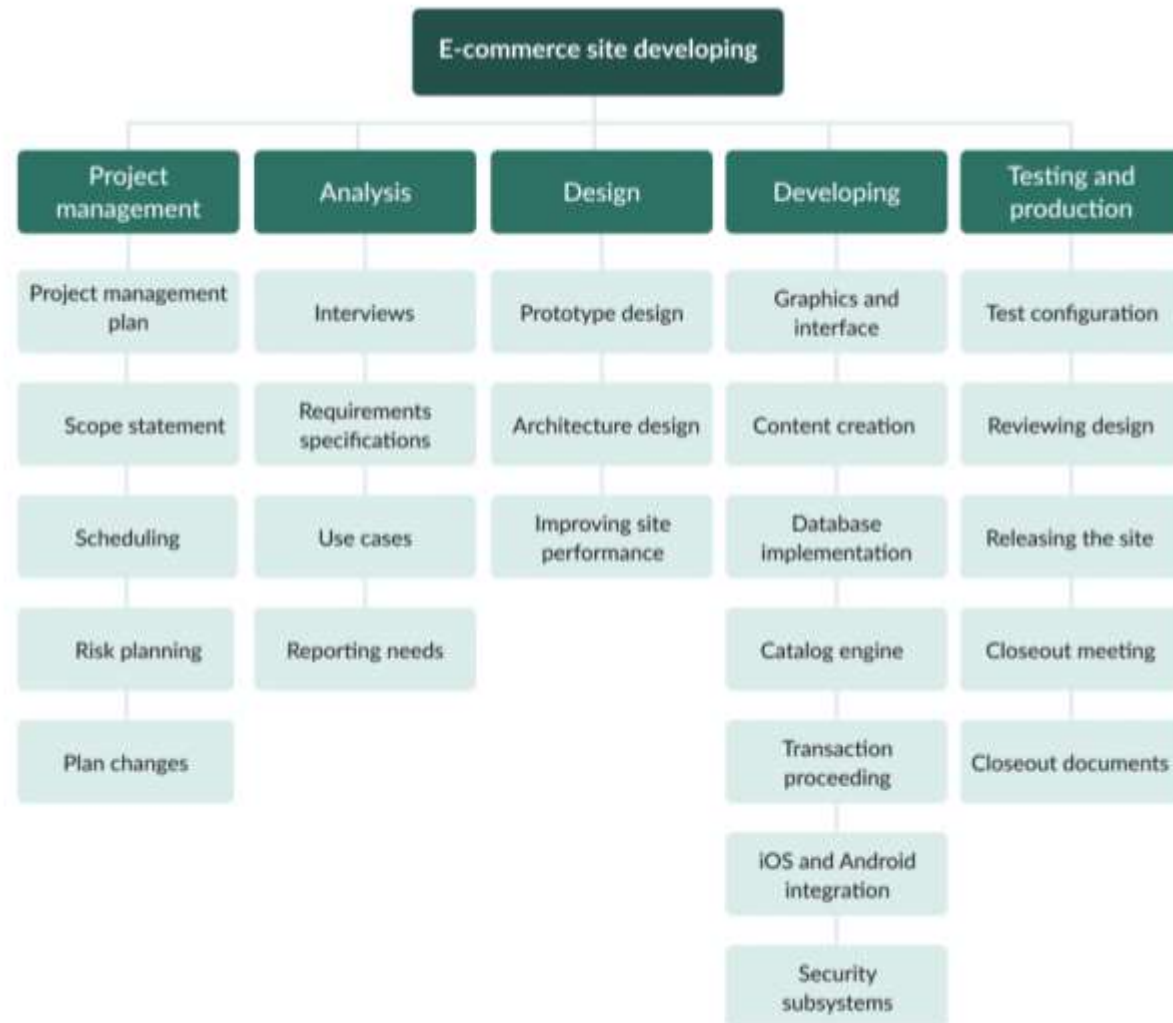
Why Work Breakdown Structure?

1. It allows easy management of the project.
2. It helps in proper organization of the project by the top management.
3. Giving visibility to important activities.
4. Giving visibility to risky activities.
5. Illustrate the correlation between the activities and deliverables.
6. It allows to do a precise cost estimation of each activity.
7. It allows to estimate the time that each activity will take more precisely.
8. Better communication with your team members regarding tasks
9. The efficiency of a work breakdown structure can determine the success of a project.

Example 1: Work Breakdown Structure of Restaurant System



Example 2: Work Breakdown Structure of E-Commerce site

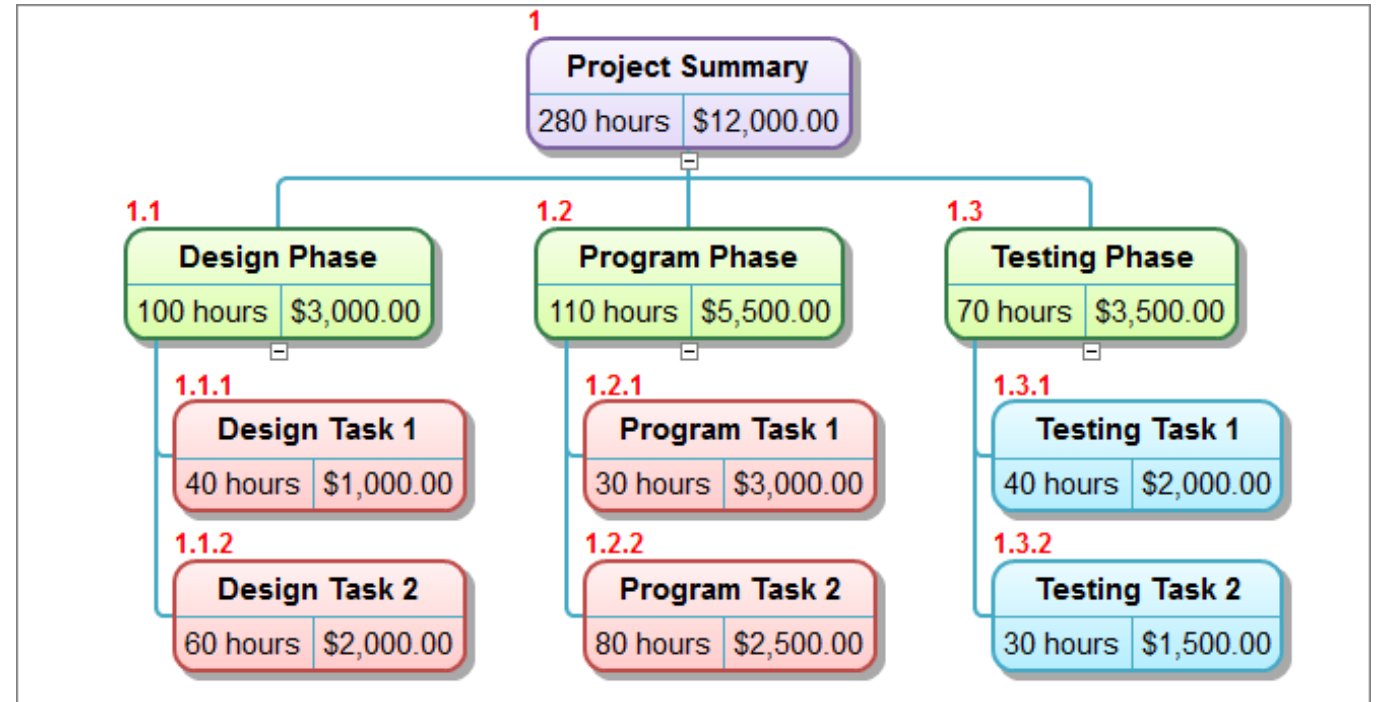


WBS Software Tools

- Many free and paid software tools can help you create WBS for your projects.

- **Common WBS Software tools are:**

1. EdrawMax
2. Lucidchart
3. SmartDraw
4. Visual Paradigm
5. MindView
6. Creately



Issues in Project Management

- Project is not completed on schedule.
- Changing customer requirements affect on schedule.
- Technical difficulties are generate.
- Miscommunication among project management. Essential software & hardware may be delivered late.
- In large project, Software engineer perform multiple tasks parallel.
- Tasks interdependencies are in project.
- Risk is not considered at beginning of project.



Project Scheduling

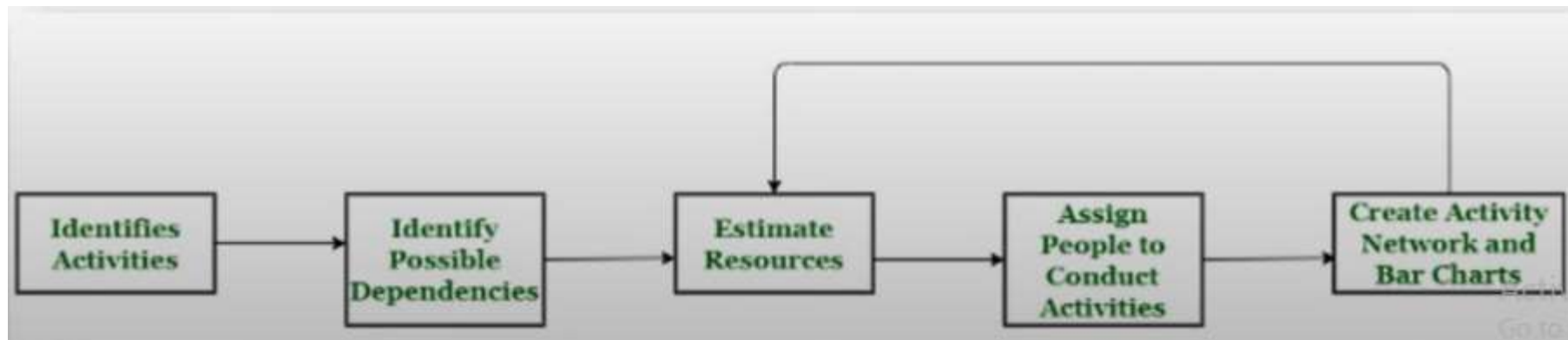
- Project scheduling is responsible activity of project manager.
- Project schedule is a mechanism that is used to communicate and know about that tasks are needed and has to be performed in project.
- Project manager separate total work task in project into different activities. i.e. WBS
- Project Manager estimate time & recourses required to complete activities & organize them into coherent sequence.
- Effective project scheduling leads to success of project, reduced cost and increased customer satisfaction. Project

Project Scheduling Management



Project Scheduling Process

1. Identify all the functions/modules required to complete the project.
2. Break down large functions into small activities i.e. Develop WBS structure.
3. Determine the dependency among various activities.
4. Allocate resources to activities.
5. Assign people to conduct different activities.
6. Plan the beginning and ending dates for different activities.
7. Create Activity Network & Bar or Gantt charts.



Basic Principles of Project Scheduling

- 1. Compartmentalization:** Project divide into number of manageable activities & tasks.
- 2. Interdependency:** Certain tasks occur in sequence whereas other tasks occur in parallel.
- 3. Time Allocation:** Each task has to be assigned specific time period i.e a start date & a completion date.
- 4. Effort Validation:** PM ensure that allocated number of people work on given task.
- 5. Defined Responsibilities:** Each task is assigned to specific member of the software team.
- 6. Defined Outcomes:** Each task has a defined outcome. i.e., Work product
- 7. Defined Milestones:** Every task should be associated with a project milestone. A milestone is accomplished when one or more work products has been reviewed for quality.

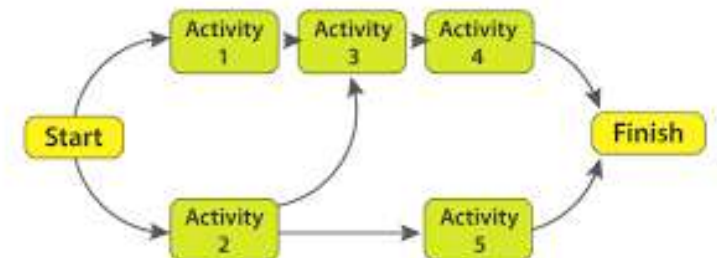
Project Scheduling Techniques

1. Critical Path Method:

- It help you determine both longest and shortest possible time take to complete a project.
- There are three essential elements: The tasks required to complete the project, Which tasks depend on the completion of others, A time estimate for each activity

Example:

- There are four tasks in the project - A, B, C. and D.
- Task B and D can only begin after task A completes, whereas task C has no such restriction.
- Task A will be time-sensitive as any delay in its completion can delay in task B & D.
- Called as Critical Task.
- This helps in identifying and separating the independent variables
- Finally, it adds milestones to the project.



Project Scheduling Techniques

2. Program Evaluation and Review Technique (PERT):

It is a way to schedule flow of tasks in a project and estimate total time taken to complete it.

PERT charts offer a visual representation of the major activities (and dependencies) in a project

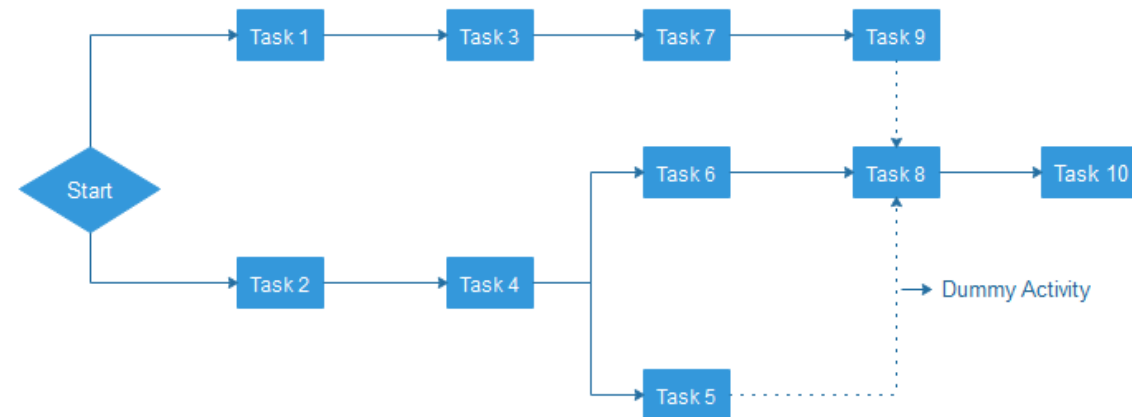
It calculate:

Optimistic time (O): Quickest time you can complete a project

Pessimistic time (P): Longest time it'll take to complete your project

Most likely time (M): How long it'll take to finish your project if there are no problems.

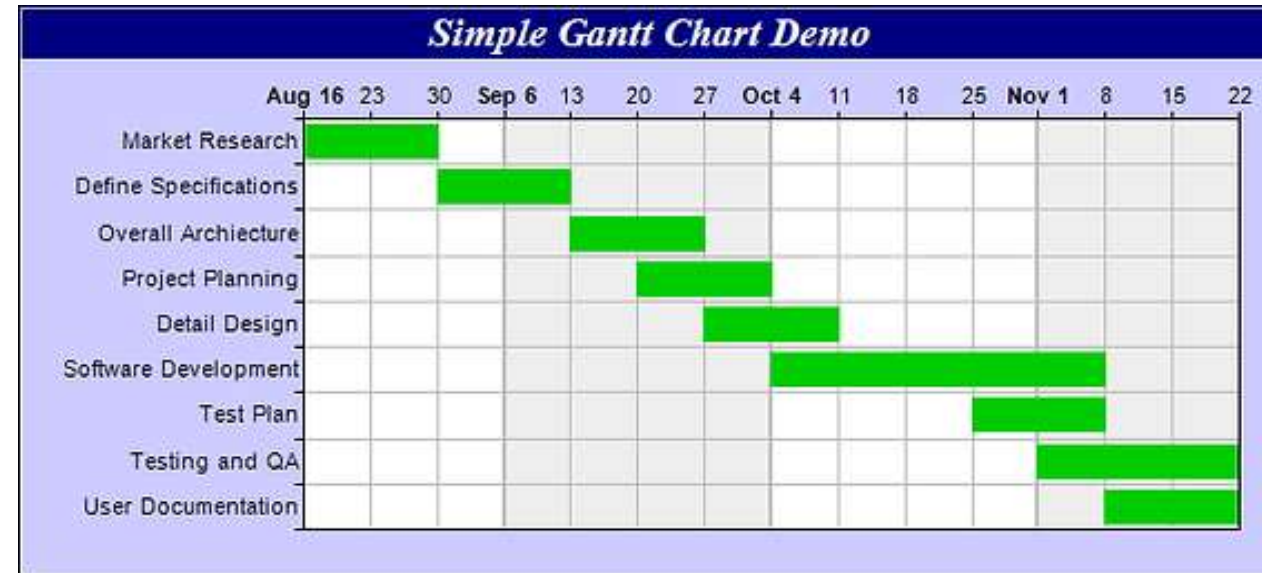
$$(O+4M+P)/6$$



Project Scheduling Techniques

3. Gantt Chart:

- A Gantt chart is a type of bar graph that project managers use for planning and scheduling in complex project.
- It represent each task horizontally on a bar chart, which shows the start and end dates & they frequently include deadlines & dependencies of tasks.
- It easier to visualize the progress of a project and see how different tasks interact with one another.



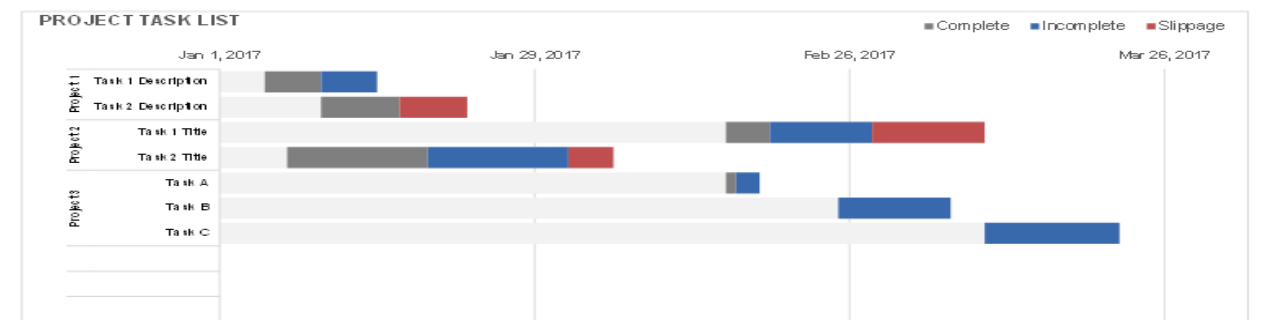
Project Scheduling Techniques

4. Task List:

- One of the simplest project scheduling techniques is the creation of a task list.
- Create task list using a word processor or spreadsheet software.
- It create a list of tasks and include important information like the task manager, start date, deadline & completion status.

PROJECT TASK LIST

Project Start 1/1/2017							Project Start	Days to Start	Complete	Incomplete	Slippage	Plan Days
PROJECT	TASK	PRIORITY	START	PLAN END	ACTUAL END	% COMPLETE						
Project 1	Task 1 Description	HIGH	1/5/2017	1/15/2017	1/15/2017	50%	1/1/2017	4	5	5	0	10
	Task 2 Description	MEDIUM	1/10/2017	1/17/2017	1/23/2017	100%	1/1/2017	9	7	0	6	7
Project 2	Task 1 Title	LOW	2/15/2017	2/28/2017	3/10/2017	30%	1/1/2017	45	3.9	9.1	10	13
	Task 2 Title	HIGH	1/7/2017	2/1/2017	2/5/2017	50%	1/1/2017	6	12.5	12.5	4	25
Project 3	Task A	LOW	2/15/2017	2/18/2017		30%	1/1/2017	45	0.9	2.1	0	3
	Task B	LOW	2/25/2017	3/7/2017			1/1/2017	55	0	10	0	10
	Task C	LOW	3/10/2017	3/22/2017			1/1/2017	68	0	12	0	12
							1/1/2017	0	0	0	0	0
							1/1/2017	0	0	0	0	0
Insert new rows above this one												



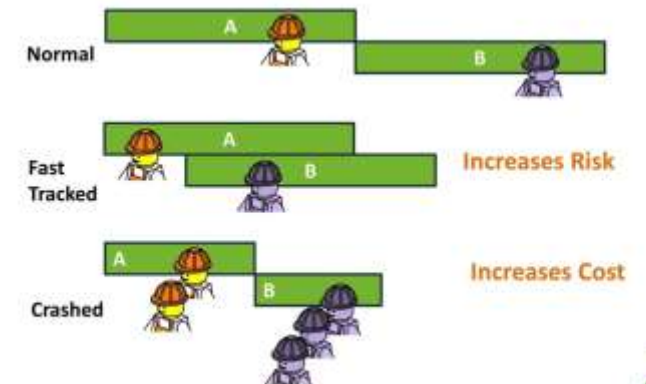
Project Scheduling Techniques

5. Fast Tracking:

- In Fast Tracking, Project is being implemented by either simultaneously executing many tasks or by overlapping many tasks to each other.
- **Example:** In software development project, designing and development can be taken up in parallel. Once design of essential features is ready and approved, development team can work on it. Meanwhile, the designing team will work on the remaining elements and functions.

6. Crashing:

- Crashing deals with involving more resources to finish the project on time.
- **Example:** Add more developer in project, Paying overtime to employee.
- Crashing can only be applied when it fits your project budget.

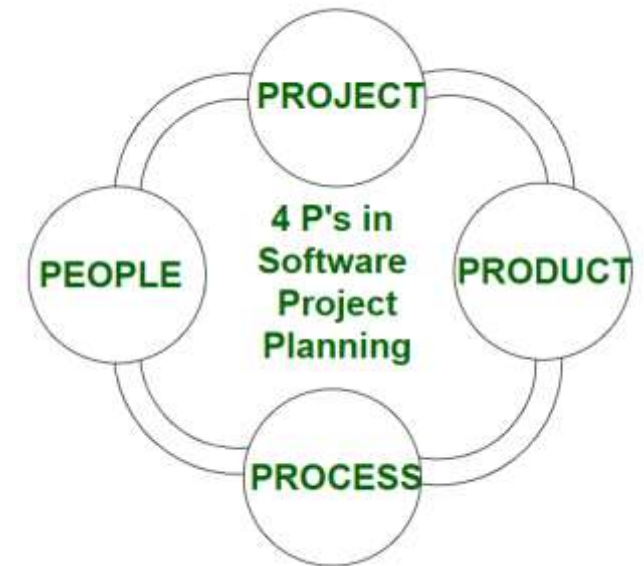


The Management Spectrum

- In software engineering, the management spectrum describes the management of a software project.
- For properly building a product, there's a very important concept handle by Project Manager.
- These components play a very important role in your project that can help your team meet its goals and objective

► Effective software project management focuses on the four P's:

1. People
2. Product
3. Process
4. Project.



1. People

- The most important contribution in software project is not made by system or tool, It is done by people. i.e. Human Resources.
- The success of project depend on selecting right kind of people with right talent.

► **Depending on there roles & responsibilities, following are the main categories:**

- 1. Senior Manager:** Define the business issues & have significant influence on the project.
- 2. Project Manager:** Plan, Motivate, Organize & Control the project work. Has problem solving skills & manageable team abilities.
- 3. Software Engineer:** Who deliver technical skills which are necessary in project.
- 4. Customer:** Who specify the requirement for the software project.
- 5. End Users:** Who interact with the software product.



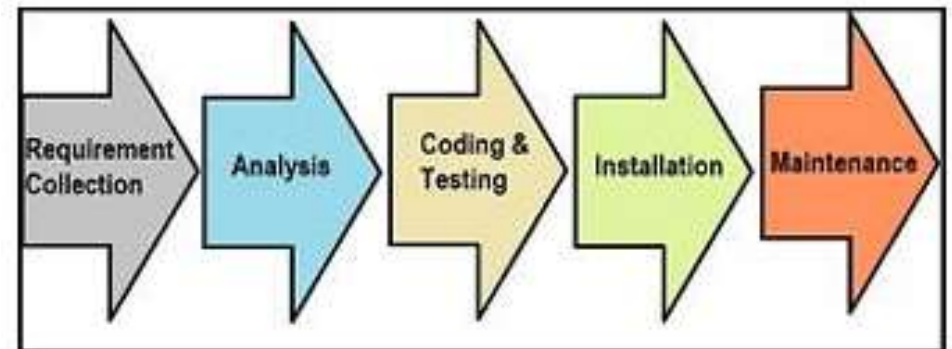
2. Product

- The product is the ultimate goal of the project.
- This is any types of software product that has to be developed or deliverable.
- Before a product can be planned, its objectives and scope should be established, alternate solutions should also be considered & technical & management constraints should be identified.
- Lack of these information, it is impossible to define reasonable and accurate estimation of the cost, identify possible risk & define manageable project schedule.



3. Process

- Project managers and team members should have a methodology and plan that complete project as per customer requirements.
- Without a clearly defined process, team members will not know what to do and when to carry out project activities.
- Using the right process will increase the project execution success rate that meets its original goals and objectives.
- The Process has several steps involved like, Documentation phase, Designing phase, Implementation phase, software configuration management, Deployment phase and Interaction phase.



4. Project

- The project is the complete software project that includes requirement analysis, development, delivery, maintenance and updates.
- The project manager plays a critical role for completing project.
- They are responsible to guide the team members to achieve the project's target and objectives.
- They helping & assisting them with issues, checking on cost and budget and making sure that the project stays on track with the given deadlines.
- They manage complete project activities to avoid project failure.



W5HH Principle

- Barry Boehm provided philosophy that prepares easy and manageable designs for software projects.
- The W5HH questions help project managers to guide objectives, timelines, responsibilities, management styles, technical approaches and resources in project.
- These method used for efficient software project management & development.

► WWWWHH Questions:

- | | |
|----------|--------------|
| 1. Why? | 5. Where? |
| 2. What? | 6. How? |
| 3. When? | 7. How Much? |
| 4. Who? | |

W5HH Questions

1. Why the system is going to be developed?

- This focus on the business reasons & problem statements for developing the software.
- Identify whether the project's purpose will justify the cost, time spent on it by people?

2. What activities are needed to be done in this?

- It help team to establish project schedule by identifying key project tasks that are required by the customer.
- It mentioned guidelines & principle for determining tasks that need to be complete.

3. When will it be completed?

- When project tasks will be started and when they enter into the final stage to reach the goal.

W5HH Questions

4. Who is responsible for each activities?

- The role and responsibility of each member of the software team are defined.

5. Where are they organizationally located?

- Not all roles and responsibilities reside within the software team itself.
- The customer, users and other stakeholders also have responsibilities.

6. How will the job be done technically and managerially?

- Once the product scope is established, management and technical strategy for the project must be defined.

7. How much of each resource is needed?

- It figure out how much resources necessary to complete the project as per customer requirement & budget of the project.

Software Metrics

Software Measurements

- Software Measurement is indicator of size, quantity amount or dimension of particular attribute of a product or process.
- It helps the project manager & entire software team to take decisions that lead to successful completion of the project by generating quantity result.

► Software measurements are of two categories:

1. Direct Measures: It include software processes like Cost & Effort applied, Lines of code produced, Execution speed & Total no. of errors that have been reported.

2. Indirect Measures: It include products like Functionality, Quality, Complexity, Reliability, Maintainability and many more.



Software Metrics

- Software Metrics provide measures, functions or formulas for various aspects of software process & product.
- It including measuring software performance, planning work items, measuring productivity, and many other uses.

Software metrics are of three categories:

1.Product Metrics: It estimate Size, Complexity, Quality & Reliability of software.

2. Process Metrics: It estimate Faults rate during development, Pattern of testing defect arrival. Time it takes for a fixed operation.

3. Project Metrics: It estimate Number of software developers. Cost, Scheduling and Productivity of software.



Software Measurement Principles

- 1. Formulation:** The derivation of software measures and metrics appropriate for the representation of the software that is being considered.
- 2. Collection:** The mechanism used to accumulate data required to derive the formulated metrics.
- 3. Analysis:** The computation of metrics and the application of mathematical tools.
- 4. Interpretation:** The evaluation of metrics resulting in insight into the quality of the representation.
- 5. Feedback:** Recommendation derived from the interpretation of product metrics transmitted to the software team.

Size Metrics: LOC

- Size-Oriented Metrics concentrates on the size of the program created.

1. LOC (Lines of Code):

- It is one of the earliest and simpler metrics for calculating the size of the computer program.
- It is generally used in calculating and comparing the productivity of programmers.

- LOC not count comment or blank line in code.

- **Example:** Total Line of Code (LOC)=09

```
//Import header file
#include <iostream>
int main() {
    int num=10;
    //Logic of even number
    if (num%2==0)
    {
        cout<<"It is even number";
    }
    return 0;
}
```

Size Metrics: LOC

Advantages:

1. Most used metric in cost estimation.
2. It is very easy in estimating the efforts.

Disadvantages:

1. It cannot measure the size of the specification.
2. Bad software design may cause an excessive line of code
3. It is language dependent
4. Users cannot easily understand it

Project	LOC	Cost \$K	Efforts (Persons/ Month)	Documenta tion	Errors
A	10000	110	18	365	39
B	12000	115	20	370	45
C	15400	130	25	400	32

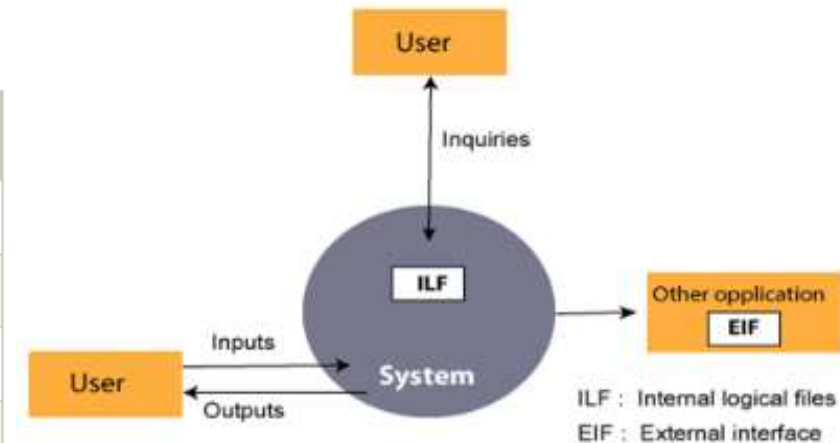
Size Metrics: FP

2. FP (Functional Point):

- Function-oriented software metrics measure functionality, what the system performs, is the measure of the system size.
- It find out by counting the number and types of functions used in the applications

Types of FP Attributes

Measurements Parameters	Examples
1.Number of External Inputs(EI)	Input screen and tables
2. Number of External Output (EO)	Output screens and reports
3. Number of external inquiries (EQ)	Prompts and interrupts.
4. Number of internal files (ILF)	Databases and directories
5. Number of external interfaces (EIF)	Shared databases and shared routines.



FPA's Functional Units System

Size Metrics: FP

Advantages:

1. Need detail specification.
2. Not restricted to code.
3. Language Interdependent.

Disadvantages:

1. Ignore quality issues.
2. Subjective counting depend on estimation.

	A	B	C	D	E	F	G
	Info Domain	Optimistic	Likely	Pessim.	Est Count	Weight	FP count
1							
2	# of inputs	22	26	30	26	4	104
3	# of outputs	16	18	20	18	5	90
4	# of inquiries	16	21	26	21	4	84
5	# of files	4	5	6	5	10	50
6	# of external inter	1	2	3	2	7	14
7	UFC: Unadjusted Function Count						342
8			Complexity adjustment factor				1.17
9						FP	400

Differentiate between FP and LOC

FP	LOC
1. FP is specification based.	1. LOC is an analogy based.
2. FP is language independent.	2. LOC is language dependent.
3. FP is user-oriented.	3. LOC is design-oriented.
4. It is extendible to LOC.	4. It is convertible to FP (backfiring)

Software Project Estimation

- Effective software project estimation is important activity in software development project.
- Estimation in software engineering is a procedure that predicts the time and budget that required for completing a project.
- Project Estimation requires the use of complex tools & good mathematical as well as knowledge about planning.
- One of the main reasons software programs fail is our inability to accurately estimate software size.

► Responsible Persons:

- ☐ Software Manager
- ☐ Cognizant Engineers
- ☐ Software Estimators.



Factors affect on Project Estimation

- 1. Cost:** The project will fail if you do not have sufficient funds to complete it. So. At early stage estimate project cost & ensure you have enough money to complete the work.
- 2. Time:** Estimate both overall project duration & timing of individual tasks. It also enables you to manage client expectations for key deliverables.
- 3. Size & Scope:** All the tasks that must be completed in order to deliver product on time. You can ensure that you have right materials & expertise on the project by estimating how much work is involved and exactly what tasks must be completed.
- 4. Risk:** Estimating predicting risk, what events will occur during the project's life cycle and how serious they will be. Create risk management plans.
- 5. Resource's:** Resource management ensures that you have all the resources you require & make best use of them. Like, tools, people, materials, hardware, software & other resources

Steps of Software Project Estimation

1. Estimate Project Size
2. Estimate Efforts (Person per month/hr)
3. Estimate Project Schedule (Month/Year)
4. Estimate Project Cost (Currency / Dollars)

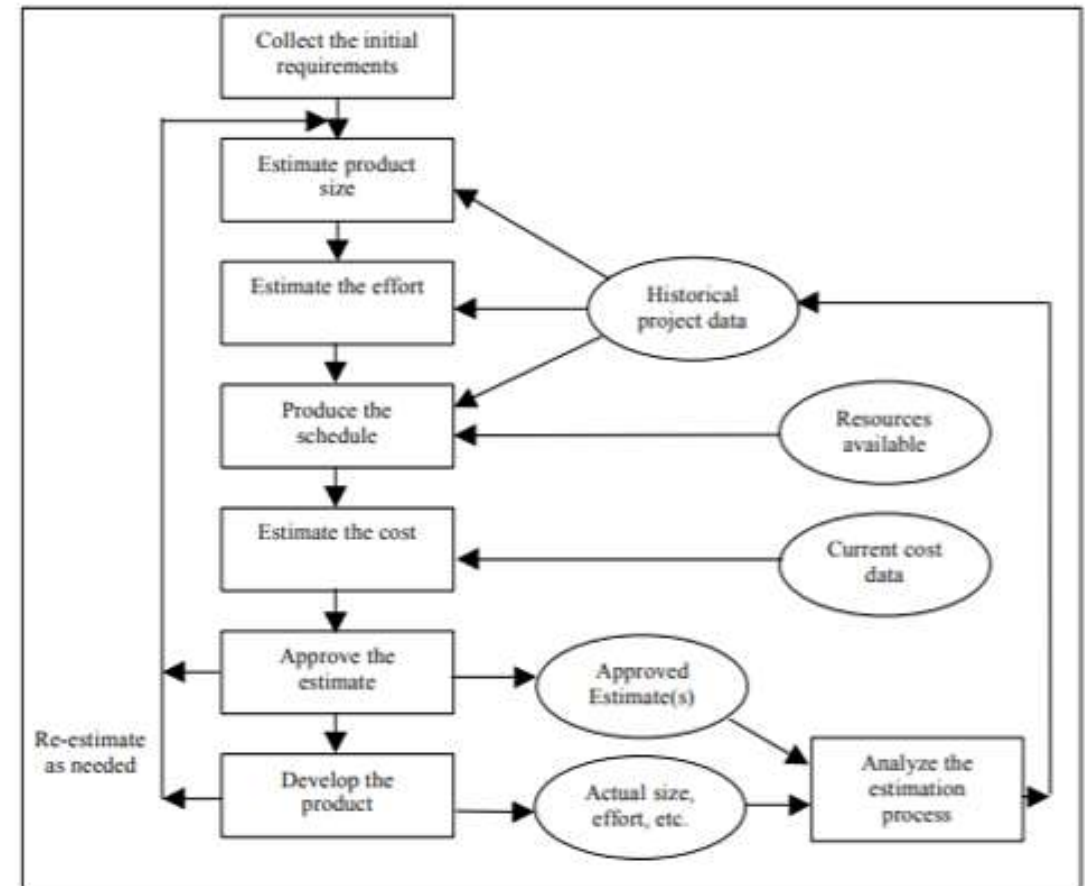
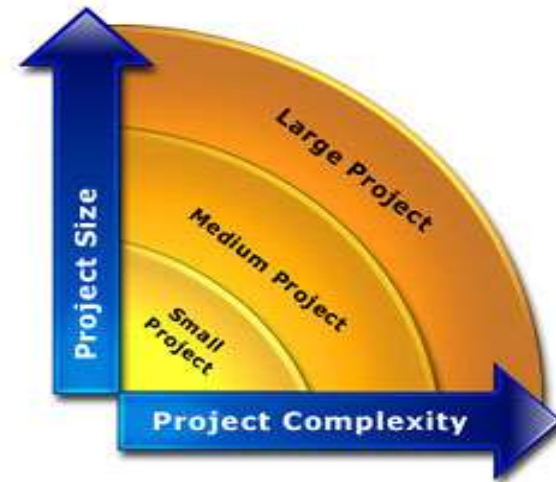


Figure 1 – The Basic Project Estimation Process

Steps of Software Project Estimation

1. Estimate Product Size (LOC & FP):

- It is the very first step to make an effective estimation of the project.
- A Customer's requirements, SRS Document & System design document used for estimating the size of a software.
- Estimate project size can be through similar project developed in past. This is called estimation by Analogy.
- The system is divided into several subsystems depending on functionality & size of each subsystem is calculated.



Steps of Software Project Estimation



2. Estimate Project Efforts:

- The estimation of effort can be made from the organizational specifics of SDLC.
- Software development project involves Design, Coding, Testing, Writing, Reviewing documents, Implementing prototypes & Deliverable it decide overall project efforts.
- The project effort estimate requires you to identify and estimate & then sum up all the activities you must perform to build a product of the estimated size.

There are two main ways:

1. The best way to estimate effort is based on the organization's own historical data of development process. Suppose you have similar SDLC, Project size, Development methodology, Tools, Team with similar skills and experience for the new project.
2. If the project is in different nature which requires the organization to adopt different strategy or different models based on algorithmic approach. Ex. COCOMO Model.

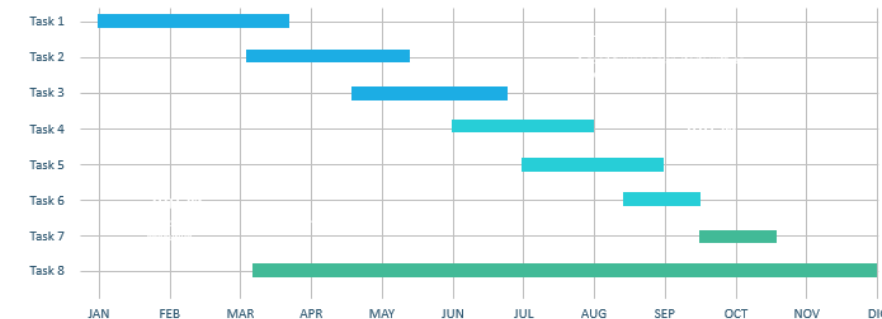
Steps of Software Project Estimation

3. Estimate Project Schedule:

- It involves estimating number of people who will work on the project & what they will work on (the Work Breakdown Structure).
- Also, when they will start working on project & when they will finish.
- Once you have this information, you need to lay it out into a calendar schedule.
- Efforts in man-month are translated to calendar months thumb rule is used.

$$\text{Schedule in calendar months} = 3.0 * (\text{man-months})^{1/3}$$

- The parameter 3.0 is variable, used depending on the situation which works best for the organization.



Steps of Software Project Estimation

4. Estimate Project Cost:

The cost of a project is derived not only from the estimates of person efforts and size.

Other parameters such as purchase hardware. software. travel for meeting. telecommunication costs (long distance phone calls, video-conferences), training, office space etc.

Exactly how you estimate total project cost will depend on how your organization allocates costs.

The simplest labor cost can be obtained by multiplying the project's effort estimate (in hours) by a general labor rate (\$ per hour).

A more accurate labor cost would result from using a specific labor rate for each staff position (e.g.. Technical, QA, Project Management, Documentation, Support, etc.)

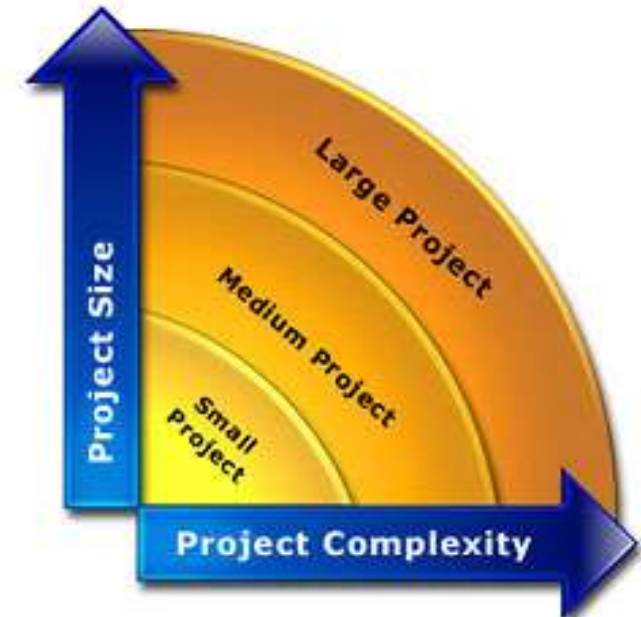


Decomposition Techniques

- **Software Project Estimation** is a form of problem solving (To estimate cost, time & efforts in software project.)
- **Decomposition Technique** is divide & conquer approach of Software Project Estimation.
- By decomposition a project into major functions like software engineering related activities, cost, schedule & efforts estimation can be performed in stepwise manner.

Decomposition Techniques are:

1. Software Sizing
2. Problem based Estimation
3. Process based Estimation



1. Software Sizing

Sizing represents the project planner's first major challenge.

► **The accuracy of a software project estimate is predicated on a number of things:**

1. The degree to which the planner has properly estimated the size of the product to be built.
2. The ability to translate the size estimate into human effort, calendar time and dollars.
3. The degree to which the project plan reflects the abilities of the software team.
4. The stability of product requirements & environment that supports software engineering effort.

► **Approaches of Software Sizing:**

1. "Fuzzy Logic" Sizing
2. Function Point Sizing
3. Standard Component Sizing
4. Change Sizing

Approaches of Software Sizing

Putnam and Myers suggest four different approaches of Software Sizing Problem:

1. "Fuzzy Logic" Sizing:

- To apply this approach, the planner must identify the type of application.
- Although personal experience can be used, the planner should also have access to historical database of projects so that estimates can be compared to actual experience.

2. Function Point Sizing:

- The planner develops estimates of the information domain characteristics.

Approaches of Software Sizing

3. Standard Component Sizing:

- Standard components for an information system are subsystems, modules, screens, reports, interactive programs, batch programs, files. LOC and object-level instructions.
- Uses historical project data to determine the delivered size per standard component.

4. Change Sizing:

- This approach is used when a project encompasses the use of existing software that must be modified in some way as part of a project.
- The planner estimates reuse, adding code, changing code, deleting code of modifications that must be accomplished.
- Using an "effort ratio" for each type of change, the size of the change may be estimated.

2. Problem based Estimation

- Lines of code and function points were described as measures from which productivity metrics can be computed. (LOC/pm & FP/pm)
- **LOC and FP data are used in two ways during software project estimation:**
 1. As an estimation variable to "size" each element of the software.
 2. As baseline metrics collected from past projects and used in conjunction with estimation variables to develop cost and effort projections.

➤ A three-point or expected value can then be computed

$$S = (s_{opt} + 4s_m + s_{pess})/6$$

– S = expected-value for the estimation variable (size)

– s_{opt} = optimistic value

– s_m = most likely value

– s_{pess} = pessimistic value

Examples of LOC & FP

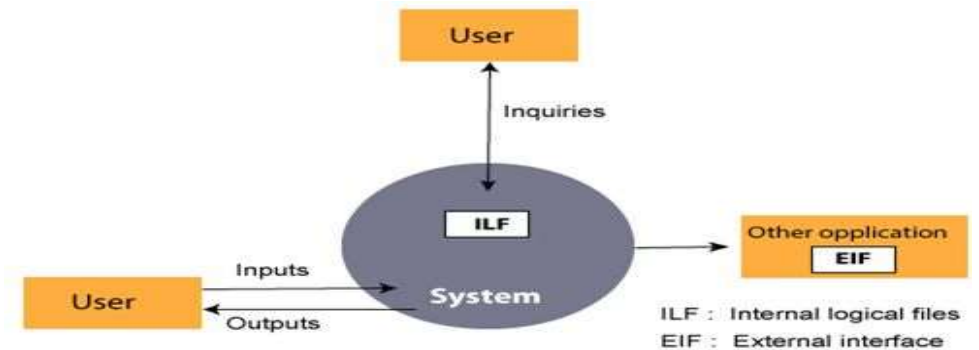
1. LOC (Lines of Code)

Project	LOC	Cost SK	Efforts (Persons/ Month)	Documentation	Errors
A	10000	110	18	365	39
B	12000	115	20	370	45
C	15400	130	25	400	32

Function	Estimated LOC
User interface and control facilities (UICF)	2,300
Two-dimensional geometric analysis (2DGA)	5,300
Three-dimensional geometric analysis (3DGA)	6,800
Database management (DBM)	3,350
Computer graphics display facilities (CGDF)	4,950
Peripheral control function (PCF)	2,100
Design analysis modules (DAM)	8,400
<i>Estimated lines of code</i>	<i>33,200</i>

2. FP(Functional Points)

Information domain value	Opt.	Likely	Pess.	Est. count	Weight	FP count
Number of external inputs	20	24	30	24	4	97
Number of external outputs	12	15	22	16	5	78
Number of external inquiries	16	22	28	22	5	88
Number of internal logical files	4	4	5	4	10	42
Number of external interface files	2	2	3	2	7	15
Count total						320



3. Process based Estimation

- Most common technique for estimating project is to estimate on the process that will be used.
- The process is decomposed into a relatively small set of tasks and the effort required to accomplish each task is estimated.
- Once problem functions & process activities are decided, the planner estimates the effort (e.g., person-months) that will be required to accomplish each software process activity for each software function.
- Average labor rates (cost/unit effort) are then applied to the effort estimated for each process activity.
- It is very likely the labor rate will vary for each task.
- Senior staff heavily involved in early activities are generally more expensive than junior

Software Cost Estimation

- The Software Cost Estimation is a process of predict/estimate the approximate cost of the software project before the development starts.
- Cost estimation models are some mathematical algorithms or parametric equations that are used to estimate the cost of a product or a project.

► Factors considering for Project Budgeting:

1. Specification & Scope
2. Location
3. Duration
4. Project Team Efforts
5. Resources



Tools and Techniques of Software Cost Estimation

1. Expert Judgment:

- The experts are the people who have prior knowledge on similar kind of projects.
- So, they can suggest valuable insight based on their experience.
- You can also take their advice on various tools and techniques that can be used to estimate similar kind of project.

2. Analogous Estimation:

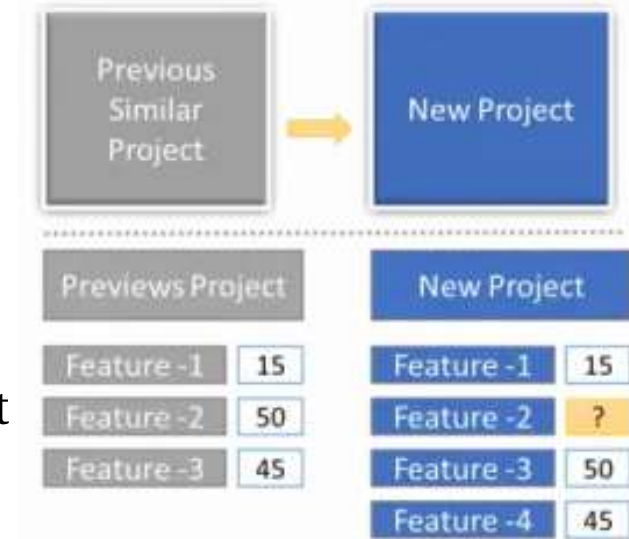
- It uses historical data from similar projects like scope, budget, duration, size, weight and complexity etc.
- It measures the current project on that basis and does the estimation.
- It used in the early phases of a project and is less costly, time & accurate than other methods.



Tools and Techniques of Software Cost Estimation

3. Parametric Estimation:

- It analyze a past project and get the man-hours for each unit.
- Then it estimate the total man-hour for the new project's activity.
- It use statistical modelling technique to identify both cost and time for current project using data from past projects.



4. Bottom-up Estimation:

- In WBS, All the project is divided into smaller work packages & cost of each work package is estimated one by one
- It estimate every single activity and rolling them up into an overall project estimate.
- It is a time consuming and costly technique but it provides the most accurate estimation.

Tools and Techniques of Software Cost Estimation

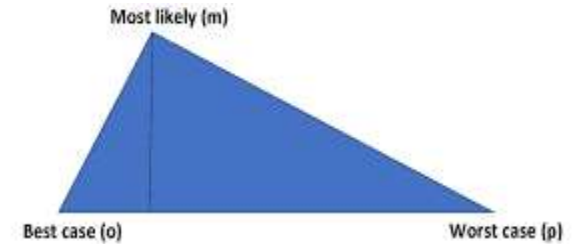
5. Three-Point Estimation:

- To deal with uncertainties and risk PERT method is used.
- Program Evaluation and Review Technique which provide three different estimates.

1. Most Likely Estimate: Assuming a normal case in which everything goes as usual.

2. Pessimistic Estimate: Assuming everything goes wrong i.e. worst case scenario

3. Optimistic Estimate: Assuming everything goes as planned i.e. best case scenario



6. Reserve Analysis:

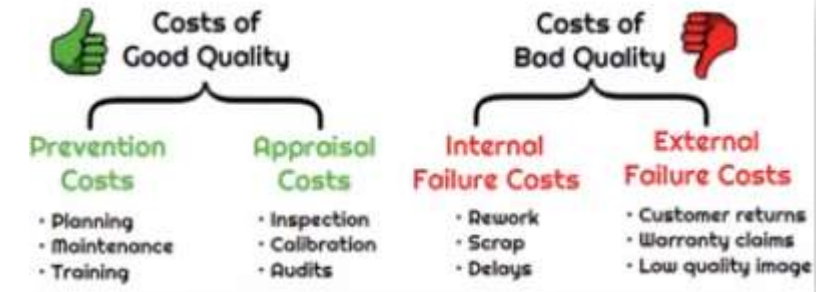
- It is important to keep the reserve budget to deal with uncertain events.
- Project manager need to take approval from the sponsors for management of reserve budget.



Tools and Techniques of Software Cost Estimation

7. Cost of Quality:

- It includes money spent during the project to avoid failures.
- It includes money spent during & after the project due to failures.
- These money included in the project cost estimate.



8. Vendor Bid Analysis:

- It is used to estimate what the project should cost by comparing the bids submitted by multiple vendors.
- There may be differences in their bids but you can get an idea considering the average bid values.



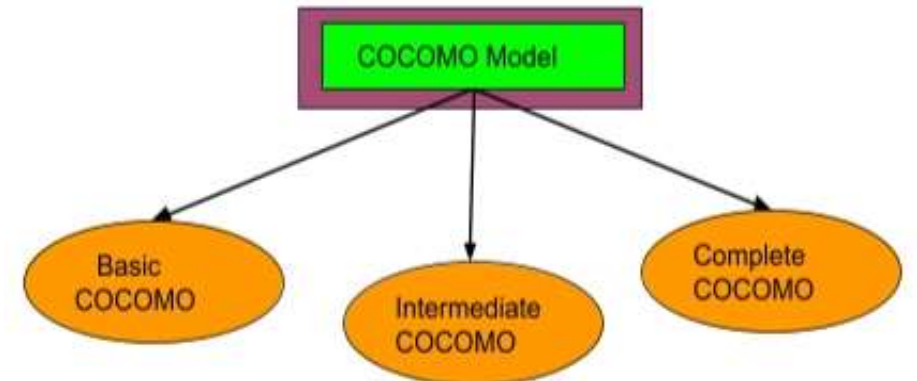
Typical Problems with IT Cost Estimation

1. To estimate large software project is complex task. There are various modifications are there But estimation process done at early stage of process not at various stages.
2. Many people doing estimation but they have little experience about it. Try to provide training & mentoring them.
3. Human beings are biased towards underestimation. Senior IT professionals might make estimates based on their own abilities and forget that many younger people will be working on a project.
4. Estimators might also forget to allow for extra costs needed for integration and testing on large IT projects.
5. Management might ask for an estimate but really want a more accurate number to help them create a bid to win a major contract or get internal funding.



About COCOMO Model

- It was developed by a scientist Barry Boehm in 1981.
- The COCOMO (Constructive Cost Model) is one of the most popularly used software cost estimation models.
- This model depends on the size means number of lines of code for software product development.
- It estimates Effort required for project, Total project cost & Scheduled time of project.



Software Project Types

- In COCOMO, projects are categorized into three types:

1. Organic Type:

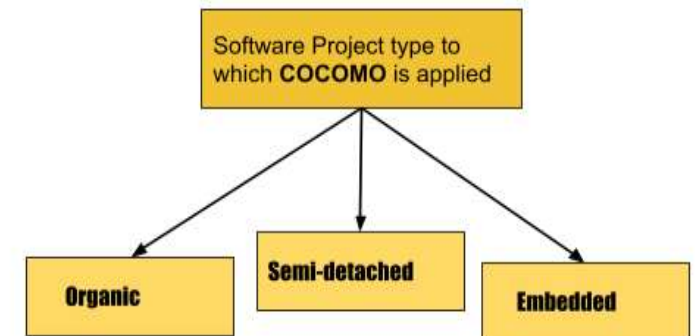
Project is small and simple. (2-50 KLOC)

Few Requirements of projects.

Project team is small with prior experience.

The problem is well understood and has been solved in the past.

Examples: Simple Inventory Management Systems & Data Processing Systems.



Software Project Types

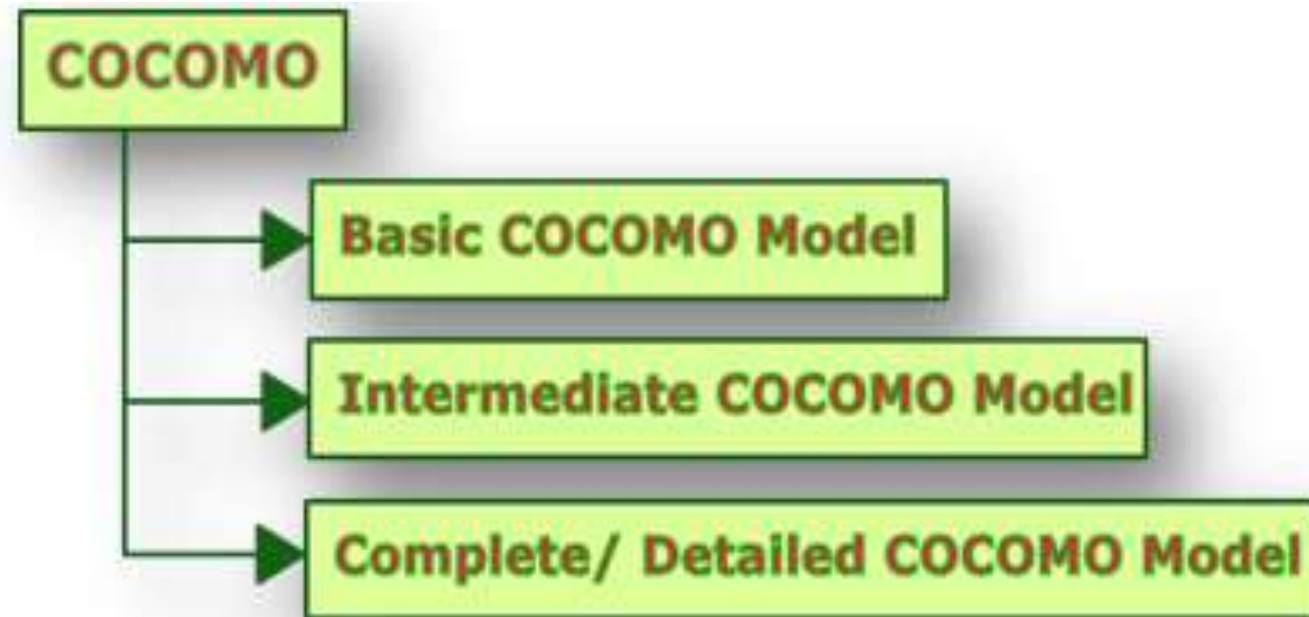
2. Semidetached Type:

- Medium size and has mixed rigid requirements. (50-300 KLOC) Project has Complexity not too high & low.
- Both experienced and inexperienced members in Project Team.
- Project are few known and few unknown modules.
- Examples: Database Management System, Difficult inventory management system.

3. Embedded Type:

- Large project with fixed requirements of resources. (More than 300 KLOC)
- Larger team size with little previous experience.
- Highest level of complexity, creativity and experience requirement.
- Examples: ATM, Air Traffic control, Banking software.

Types of COCOMO Model



Type 1: Basic COCOMO Model

- It is type of static model to estimates software development effort quickly and roughly.
- It mainly deals with the number of lines of code in project.

Formula:

- **Effort (E)** = $a \cdot (\text{KLOC})^b$ MM
- **Scheduled Time (D)** = $c \cdot (E)^d$ Months(M)

Where,

- **E** = Total effort required for the project in Man-Months (MM).
- **D**=Total time required for project development in Months (M).
- **KLOC** = The size of the code for the project in Kilo lines of code.
- **a, b, c, d** = The constant parameters for a software project.

Software Project	A	B	C	D
Organic	2.4	1.05	2.5	0.38
Semi-Detached	3.0	1.12	2.5	0.35
Embedded	3.6	1.20	2.5	0.32

Type 1: Basic COCOMO Model Example

Problem Statement:

► Consider a software project using semi-detached mode with 300 Kloc. Find out Effort estimation, Development time and Person estimation.

Solution:

$$\text{Effort (E)} = a * (\text{KLOC})^b = 3.0 * (300)^{1.12} = 1784.42$$

$$\text{PM Development Time (D)} = c(E)^d = 2.5(1784.42)^{0.35} = 34.35 \text{ Months(M)}$$

$$\text{Person Required (P)} = E/D = 1784.42/34.35 = 51.9481 \text{ Persons} \sim 52 \text{ Persons}$$

Type 2: Intermediate COCOMO Model

- Extension of Basic COCOMO model which enhance more accuracy to cost estimation model result.
- It include cost drivers (Product, Hardware, Resource & project Parameter) of project.

Formula:

- **Effort (E)** = $a * (KLOC) * EAF$ MM
- **Scheduled Time (D)** = $c(E)^d$ Months(M)

Where,

- E = Total effort required for the project in Man-Months (MM).
- D=Total time required for project development in Months (M).
- KLOC=The size of the code for the project in Kilo lines of code.
- a, b, c, d =The constant parameters for the software project.

Software Project	A	B	C	D
Organic	2.4	1.05	2.5	0.38
Semi-Detached	3.0	1.12	2.5	0.35
Embedded	3.6	1.20	2.5	0.32

Type 2: Intermediate COCOMO Model

- **EAF**: Effort Adjustment Factor, which is calculated by multiplying the parameter values of different cost driver parameters. For ideal, the value is 1.

Cost Drivers	RATINGS					
	Very low	Low	Nominal	High	Very High	Extra High
Product Attributes						
RELY	0.75	0.88	1.00	1.15	1.40	..
DATA	..	0.94	1.00	1.08	1.16	..
CPLX	0.70	0.85	1.00	1.15	1.30	1.65
Computer Attributes						
TIME	..	..	1.00	1.11	1.30	1.66
STOR	..	..	1.00	1.06	1.21	1.56
VIRT	..	0.87	1.00	1.15	1.30	..
TURN	..	0.87	1.00	1.07	1.15	..

Cost Drivers	RATINGS					
	Very low	Low	Nominal	High	Very high	Extra high
Personnel Attributes						
ACAP	1.46	1.19	1.00	0.86	0.71	..
AEXP	1.29	1.13	1.00	0.91	0.82	..
PCAP	1.42	1.17	1.00	0.86	0.70	..
VEXP	1.21	1.10	1.00	0.90	..	..
LEXP	1.14	1.07	1.00	0.95	..	..
Project Attributes						
MODP	1.24	1.10	1.00	0.91	0.82	..
TOOL	1.24	1.10	1.00	0.91	0.83	..
SCED	1.23	1.08	1.00	1.04	1.10	..

Type 2: Intermediate COCOMO Model Example

Problem Statement:

- For a given Semidetached project was estimated with a size of 300 KLOC. Calculate the Effort, Scheduled time for development by considering developer having high application experience and very low experience in programming.

Solution:

- $EAF = 0.82 * 1.14 = 0.9348$
- $Effort (E) = a * (KLOC)^b * EAF = 3.0 * (300)^{1.12} * 0.9348 = 1668.07 \text{ MM}$
- $Scheduled Time (D) = c * (E)^d = 2.5 * (1668.07)^{0.35} = 33.55 \text{ Months (M)}$

Type 3: Detailed / Complete COCOMO Model

- The model incorporates all qualities of both Basic COCOMO and Intermediate COCOMO strategies on each software engineering process.
- The whole software is divided into different modules and then apply COCOMO in different modules to estimate effort and then sum the effort.

The Six phases of detailed COCOMO are:

1. Planning and requirements
2. System design
3. Detailed design
4. Module code and test
5. Integration and test
6. Cost Constructive model

Type 3: Detailed / Complete COCOMO Model

Problem Statement:

A distributed Management Information System (MIS) product for an organization having offices at several places across the country can have the following sub-components:

- Database part
- Graphical User Interface (GUI) part
- Communication part

Solution:

- ✓ The communication part can be considered as Embedded software.
- ✓ The database part could be Semi-detached software.
- ✓ The GUI part Organic software.

The costs for these three components can be estimated separately and summed up to give the overall cost of the system.

Advantages of COCOMO Model

1. Provides a systematic way to estimate the cost and effort of a software project.
2. Estimate cost and effort of software project at different stages of the development process.
3. Helps in identifying the factors that have the greatest impact on the cost and effort of a software project.
4. Provide ideas about historical projects.
5. Easy to implement with various factors.

Disadvantages of COCOMO Model

1. It ignores requirements, customer skills and hardware issues.
2. It limits the accuracy of the software costs.
3. It is based on assumptions and averages.
4. It mostly depends on time factors.
5. Assumes that the size of the software is the main factor that determines the cost and effort of a software project, which may not always be the case.

Source of Risk

► What is Risk?

✓ "Risk is an uncertain future event with a probability of occurrence and potential for loss"

► Source of Risk:

1. Misunderstanding of customer requirements.
2. Uncontrolled & continuous changing of customer requirements.
3. Unrealistic promises given to the customers.
4. Misunderstanding of real impact of new methodologies.
5. Miscalculation of robustness & extensibility of software design.
6. Miscalculation of Team work & group effectiveness.
7. Wrong budget estimation



Risk Management

- Risk Management is an important part of project planning activities.
- It involves identifying and estimating the probability of risks with their order of impact on the project.



Risk Identification

- The project organizer needs to find out risk in the project as early as possible.
- So, the impact of risk can be reduced by making effective risk management planning.
- By doing Brainstorming, SWOT Analysis, Casual Mapping & Flowchart methods are used.

There are different types of risks which can affect a software project:

1. Technology risks: Software or Hardware technologies that are used to develop the system.
2. People risks: Risks that are connected with the person in the development team.
3. Organizational risks: Organizational environment where the software is being developed.
4. Tools risks: Software tools and other support software used to create the system.
5. Requirement risks: Changes to customer requirement & process of managing the requirements change.
6. Estimation risks: Management of estimation resources required to build the system

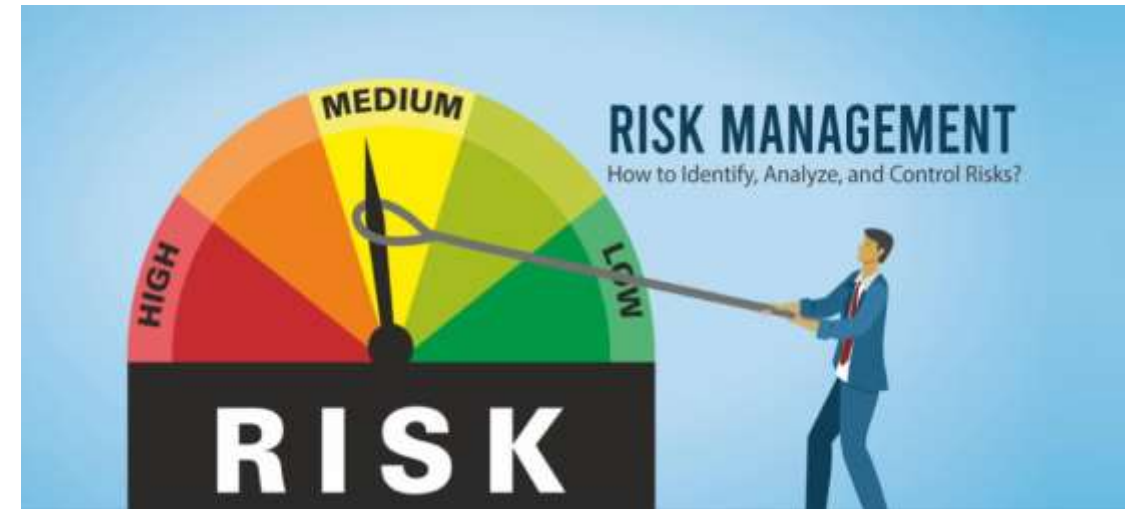
Risk Analysis & Projection or Prioritization

In Risk Analysis Process:

1. Identifying the problems causing risk in projects.
2. Identifying the probability of occurrence of problem.
3. Identifying the impact of problem.

► **The probability of a risk can be categorized as:**

1. **Very Low (0-10%):** Tolerable Risk (No harm)
2. **Low (10-25%):** Low Risk (Minor effect)
3. **Moderate (25-50%):** Medium Risk (Impact on Time)
4. **High (50-75%):** High Risk (Impact on Time & Budget)
5. **Very high (+75%):** Intolerable Risk (Impact on Output, Time, Budget & Performance)



Risk Control

- It is the process of managing risks to achieve desired outcomes.

1. Risk Planning:

- The risk planning technique considers all of the significant risks that have been identified and develop strategies to mitigate them.



► There are three main methods to plan for risk management:

- 1. Avoid the risk:** Discussing with client to change requirements, Decrease scope of work, Giving incentives to engineers to avoid the risk of human resources turnover etc.
- 2. Transfer the risk:** The risky element developed by third party. Buying insurance cover etc.
- 3. Risk reduction:** This means planning method to include the loss due to risk. If there is a risk that some key personnel might leave new recruitment can be planned.

Risk Control

2. Risk Monitoring:

- This is an ongoing process throughout the project and requires continuous evaluation and assessment of potential risks.
- Any changes in the assumptions made about risks should be identified and appropriate actions are taken to manage those risks.

3. Risk Resolution:

- This process ensures that the project stays on track and risks are controlled within acceptable levels.
- The effectiveness of risk resolution depends on the accuracy of risk identification, analysis, and planning of risk solving.
- It has ability to respond promptly and effectively to any issues that arise during the project.

About RMMM

- A risk management technique is usually seen in the software Project plan.
- This can be divided into Risk Mitigation, Monitoring and Management Plan (RMMM).
- The project manager typically uses this RMMM plan as part of the overall project plan.
- Risk is documented with the help of a Risk Information Sheet (RIS).
- RIS maintain Risk ID. Date. Probability, Impact. Description, Avoidance, Monitoring, Management plan & Current status.
- This RIS is controlled by using a database system for easier management of information i.e creation, priority ordering, searching and other analysis.



Risk Mitigation

- Risk Mitigation is a technique for avoiding risks (Risk Avoidance).
- It is proactive approach. Apply before risk have generate.

►Steps for mitigating the risks as follows:

1. Communicate with concerned staff to find off probable risk.
2. Removing all causes that are the reason for risk creation.
3. Develop a policy in an organization which will help to continue the project.
4. Controlling the corresponding documents from time to time.
5. Conducting timely reviews to speed up the work.



Risk Monitoring

- Risk monitoring is an activity used to track a project's progress.
- Performed by Project Manager.

► Objectives of Risk Monitoring Process:

1. To check if predicted risks occur or not.
2. To ensure proper application of risk avoidance are apply or not.
3. To gather information for future risk assessments.
4. To determine which risks generate which problems throughout the project.

```
If (RISK is avoidable)
{
  ➤ Risk Mitigation.      //(1)
  ➤ Risk Monitoring.      //(2)
}
else                        //(i.e. RISK is unavoidable)
{
  ➤ Risk Management.      //(3)
}
```

Risk Management

- It is reactive approach, Apply after risk have generate.
- It assumes that the mitigation activity failed, and the risk generate in reality.
- This task is done by Project manager. They will solve generated risk.
- If the project manager effectively uses project mitigation to remove risks successfully then it is easier to manage the risks

Example:

Consider a scenario that many people leaving the organization but

- If sufficient additional staff is available
- If current development activity knows to everybody in the team
- If systematic documentation available

Then any new employee easily understand & start current development activity.



RMMM Plan Example

- Risk generated: Late Delivery of Project

1. Mitigation (Avoid Risk):

- Before development apply some precautionary measures.
- Developer already knew Project will be complete in 20 days. But he told customer Project will complete in 30 days.

2. Monitoring:

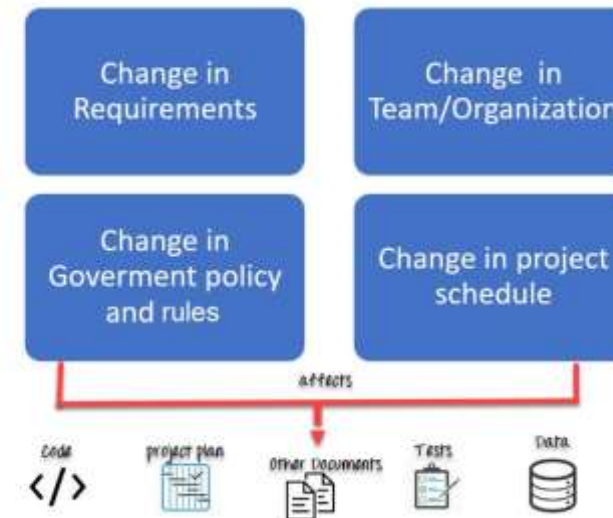
- To develop project schedule. Mentioned start & end date of Project. Within 20 to 30 days.

3. Management:

- Project not completed within deadline then negotiate with customers.
- Ask some extra time or give some additional features etc.

What is SCM?

- Software Configuration Management (SCM) is process to systematically manage, organize, & control changes in documents, codes & other entities during the Software Development Life Cycle.
- The primary goal is to increase productivity with minimal mistakes.
- SCM Tools: Puppet, ConfigHub, Saltstack, Ansible, Git, BitBucket, Docker & CHEF etc.

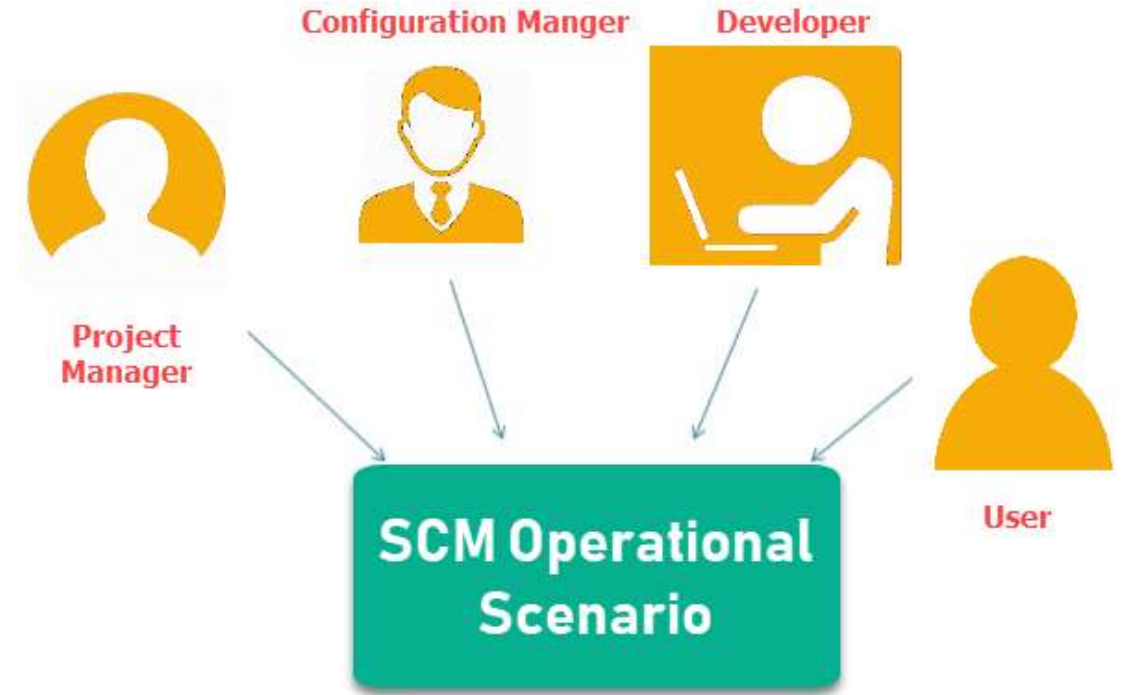


Importance of SCM

1. It ensure changes to software system are properly planned, tested & integrated into the final product.
2. Helps teams to collaborate & coordinate their work, everyone working from the same version of the software system.
3. It manage & track different versions of the system and to revert to earlier versions if necessary.
- 4 . It ensure that software systems can be easily replicated & distributed to other environments such as test, production & customer sites.
5. It improve quality & reliability of software systems, as well as increase efficiency and reduce the risk of errors.



SCM Process



SCM Process

1. Planning & Identification:

- This method determining the scope of the software system.
- This is accomplished by having meetings and brainstorming sessions with your team.
- Identifying items like test cases, specification requirements, modules & schedule time.
- Identifying each computer software configuration items in the process.
- Group basic details of why, when and what changes will be made and who will be in charge of making them

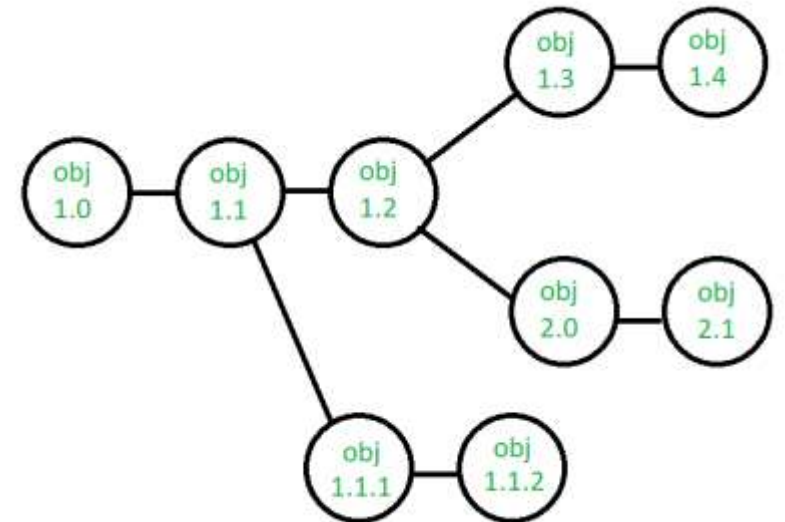
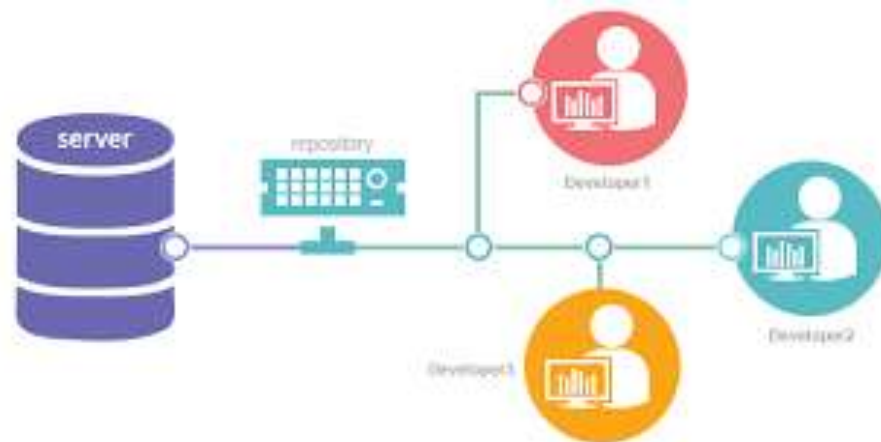
Examples:

- 1. Instead of naming a File login.php it should be named login v1.2.php where v1.2 stands for the version number of the file
- 2. Instead of naming folder "Code" it should be named "Code_D" where D represents code should be backed up daily.

SCM Process

2. Version Control Process or Baselines:

- The aim of this step is to control the alteration and modification done to the product.
- It handle different version of configuration objects that are generated during the software process.
- Also focuses on developing way to track the hierarchy of different versions of the software.
- Developing standardized label scheme for all products, revisions and files so that everyone is on the same page.
- **Examples:**



SCM Process

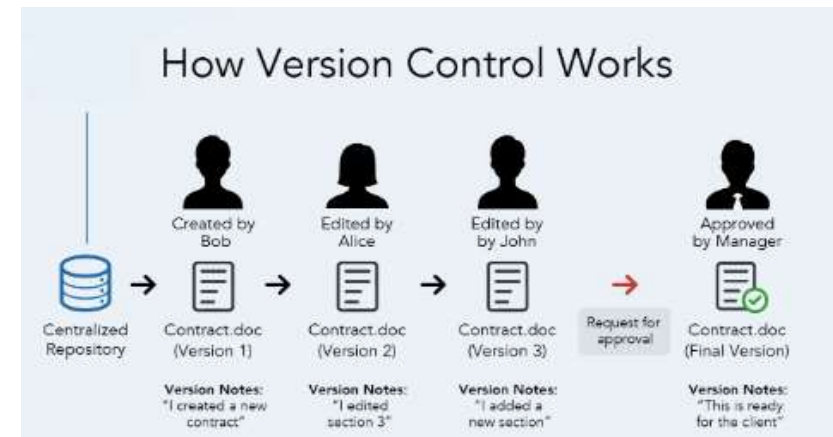
3. Change Control Process:

This method used to ensure that any changes that are made are consistent with the rest of the project.

Examples: To add or edit various configuration items, Change user permissions or Changing requirements of clients.

Process:

1. Software Team send changes to the Software Configuration Manager (SCM).
2. SCM checking examining the overall impact they will have on the project.
3. Making approved changes or explaining why change requests were denied to the team.
4. If it is approved them implement all necessary changes.
5. After that Review or Reporting it.



SCM Process

4. Configuration Auditing Process:

- This process is used to ensure that application will develop as per the project plan and test/verify the application as per scope.
- The audit confirms the completeness, correctness and consistency of modified items in the SCM system and track action items from the audit to closure.
- It mentioned what is new in each version and why the changes were necessary.
- It ensures that what is built is what is delivered.



SCM Process

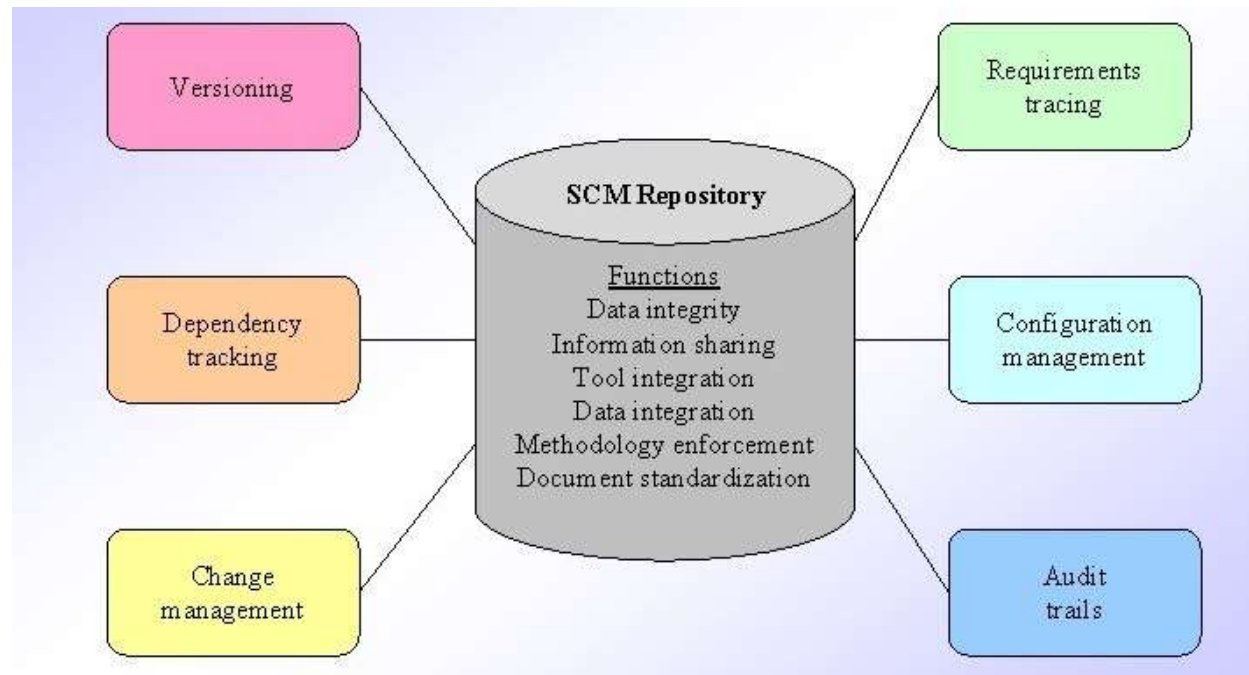
5. Review and Status Reporting Process:

- It is a technical review on the Application workflow, Process, Configuration items and Change requests etc.
- It generate the accurate status report in every phase of SDLC process.
- Configuration Status report provide to the Developers, Testers, End users, Customers and Stakeholders.
- It develop some application-related documents like User manual, Installation process guide, Configuration guide, Do's and Don't Do's etc.



SCM Repository

- SCM Repository is a set of control process & data structure that allow software team to manage change in effective manner.
- It manage version control, change control & release control process.



SCM Repository Functions

1. **Data Sharing:** Ensure consistent & accurate information. Validates entries.
2. **Information Sharing:** Manage & Control multi-user access, Share with among developers.
3. **Tool Integration:** Establish data models by using SE tools.
4. **Data Integration:** SCM task can be perform using one or more resources.
5. **Methodology Enforcement:** Define Entity Relationship model of repository.
6. **Document Standardization:** There is standard approach for creating Software Engineering documents.

SCM Repository Tools

- 1. Versioning:** Save & retrieve all repository object based on there version number.
- 2. Dependency Tracking & Change Management:** Track & manage to the changes in relationship of all objects in repository. Ex. UML diagram.
- 3. Requirement Tracing:** Trace design & construction of components & there deliverable result (Forward Tracing). As per the work product, which requirement is responsible for which feature. (Backward Tracing).
- 4. Configuration Management:** Track a series of configuration representing specific project milestone or production release.
- 5. Audit Trails:** Establish information when, why & by whom changes are made in the repository

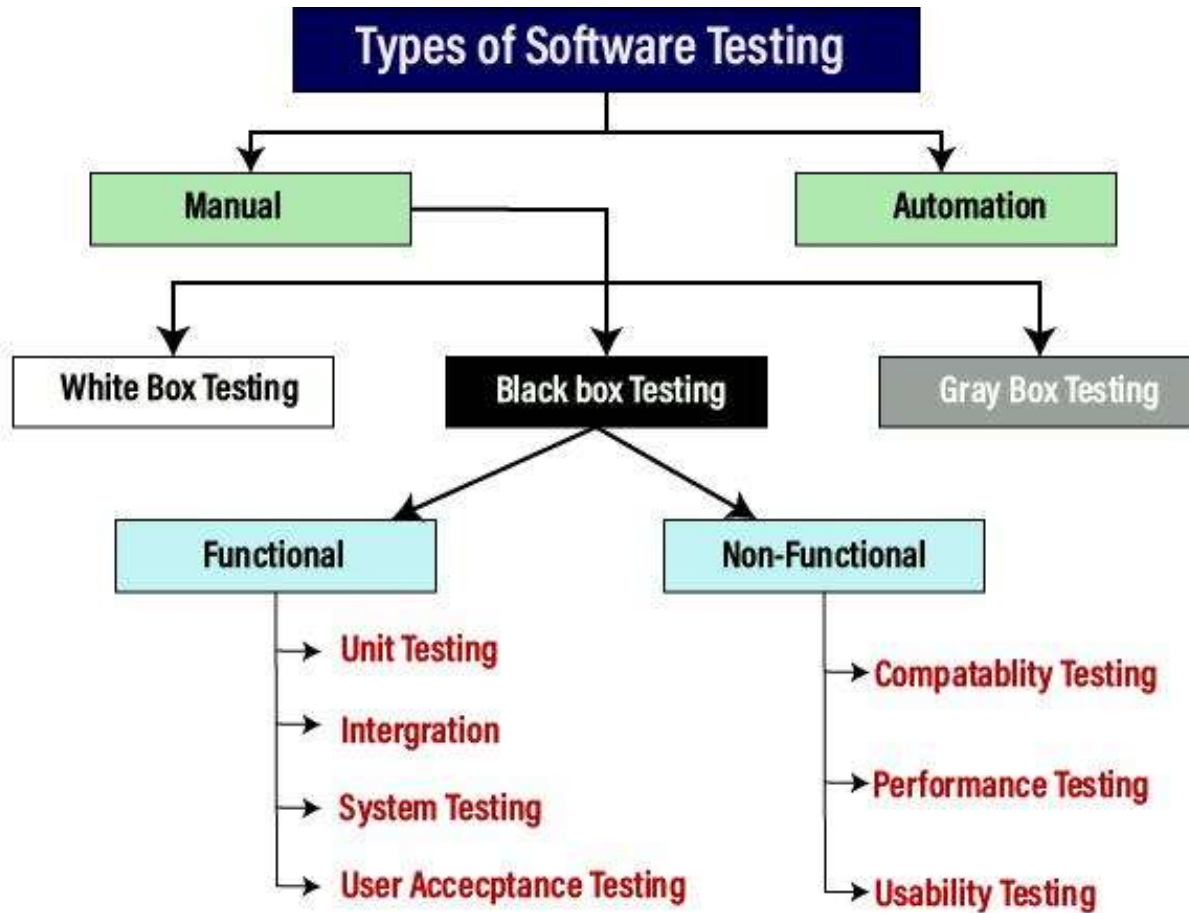
Testing and Maintenance

About Software Testing

- Software Testing is a method to check whether the actual software product matches expected requirements and to ensure that software product is Defect free.
- The purpose of software testing is to identify errors, gaps or missing requirements in contrast to actual requirements.
- It is a process of identifying the correctness of software by considering its all-attributes Reliability, Scalability, Portability. Re-usability & Usability of the product.
- Testing is mandatory because it will be a dangerous situation if the software fails any of time due to lack of testing. So, without testing software cannot be deployed to the end user.
- ✓ Software Testing is done at every phase level in software development life cycle(SDLC).
- ✓ Performed by Software Tester, Software Developer, Project Manager, End-User.



Types of Software Testing

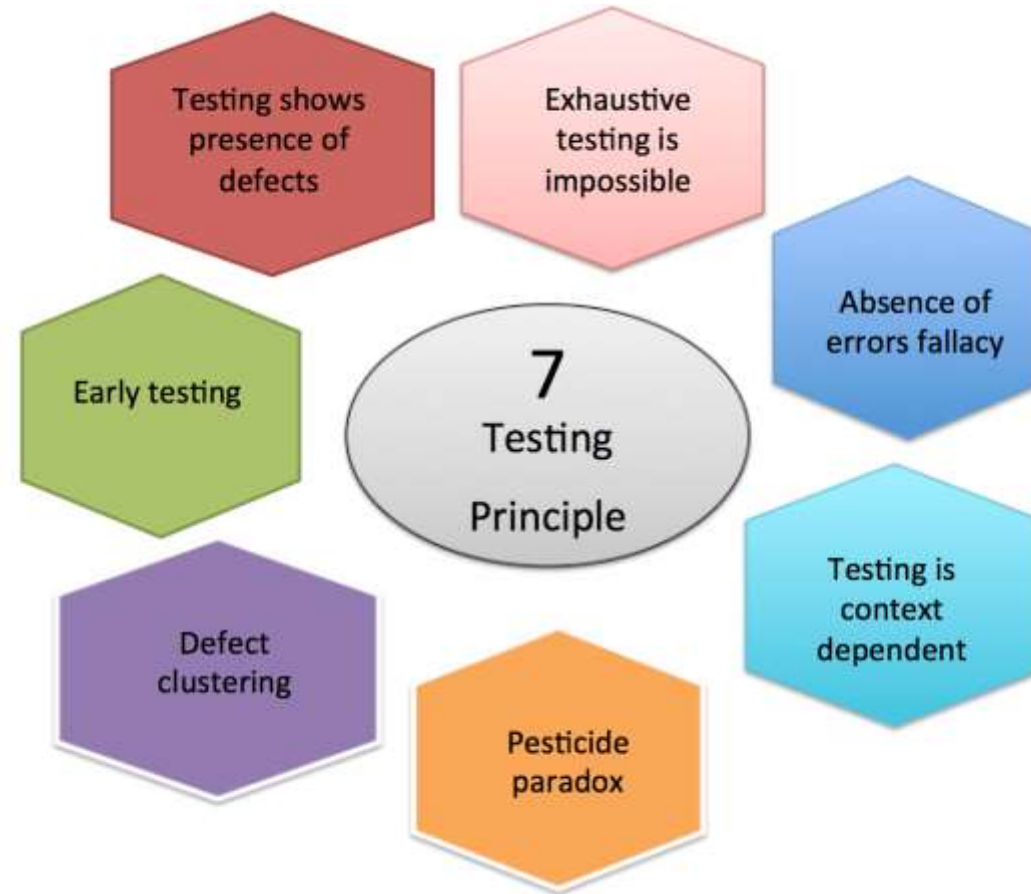


What is Test Cases?

- The test case includes set of actions or conditions, using which a testing engineer can compare expected result and actual results.
- They determine whether a software product is functioning as per the requirements of the customer.

Test Scenario ID	Login-1		Test Case ID	Login-1A			
Test Case Description	Login – Positive test case		Test Priority	High			
Pre-Requisite	A valid user account		Post-Requisite	NA			
Test Execution Steps:							
S.No	Action	Inputs	Expected Output	Actual Output	Test Browser	Test Result	Test Comments
1	Launch application	https://www.facebook.com/	Facebook home	Facebook home	IE-11	Pass	[Priya 10/17/2017 11:44 AM]: Launch successful
2	Enter correct Email & Password and hit login button	Email id : test@xyz.com Password: *****	Login success	Login success	IE-11	Pass	[Priya 10/17/2017 11:45 AM]: Login successful

Principles of Software Testing



Principles of Software Testing

1. Testing shows the Presence of Defects

- The primary purpose is to find any defects that might cause the product failure.
- The testing team cannot say that the product is 100% error-free even though testing is done
- It reduces the number of undiscovered defects in the application.
- Therefore, design the needed test cases to find the defects as much as possible.

2. Exhaustive Testing is Impossible

- Sometimes it seems to be very hard to test all the modules and their features.
- So, instead of performing exhaustive testing they use some important testing criteria effects such as risk analysis and priorities modules.
- Then Product can be delivered on schedule.

Principles of Software Testing

3. Early Testing

- The testing team will have a deep understanding of the product as they were involved from the beginning of the requirement gathering and analysis phase.
- If errors are found in an initial stage of the development life cycle, then it will be easier to fix, save time and cost is also less.
- Otherwise, if they found late, then you need to change the whole system process.

4. Defect Clustering

- A small number of modules contain most of the defects detected.
- As per Pareto's Principal or 80-20 Rule, 80% of Defects are Caused by 20% of code.
- Testers focus on a small area of team; they may fail to miss the bugs from other areas.

Principles of Software Testing

5. Pesticide Paradox

- If same set of test cases used again and again over a particular time, then these kinds of the test will not be able to find the new bugs in the software.
- So, new and different tests are necessary to be written for the implementation of multiple parts of the software, which helps us to find more bugs.

6. Testing is context-dependent

- There are multiple fields such as e-commerce, commercial websites, banking applications etc. These projects and products include different elements, features & requirements.
- So, different types of products can be tested differently based on nature.
- Same test cases cannot be applied for different projects.

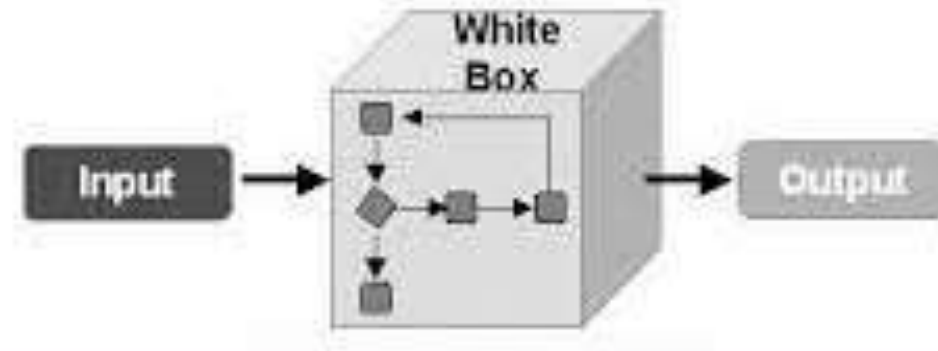
Principles of Software Testing

7. Absence of Errors Fallacy

- Once the application is completely tested and there are no bugs identified before the release, so we can say that the application is 99 percent bug-free.
- Finding and fixing defects doesn't help if the system is unusable or does not meet users' needs and expectations.
- **Absence of error fallacy** means identifying and fixing the bugs would not help if the application is impractical and not able to accomplish the client's requirements and needs.

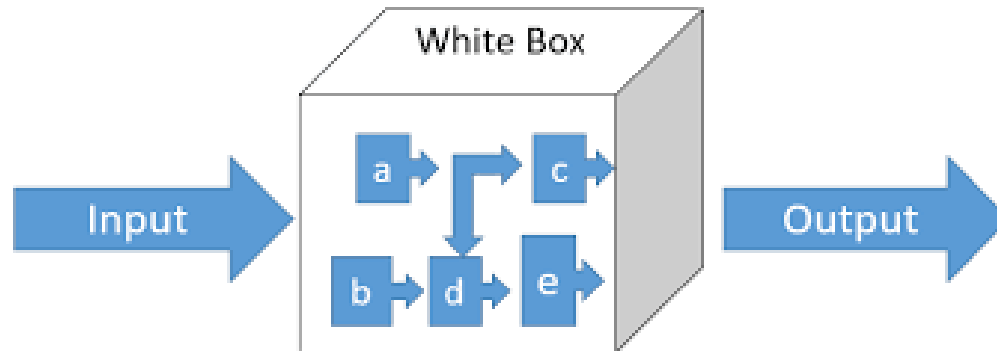
White Box Testing

- White box testing techniques analyze Internal design, Code structure & working_of the complete software product.
- It is also called Clear box testing. Open box testing. Transparent box testing, Code-based testing and Glass box testing.
- Tester has knowledge about the internal structure or the programming of the software.
- It is mostly done by software developers.
- Perform with Unit Testing & Integration Testing.
- **White Box Testing Tools:** EclEmma, Nunit, PyUnit, HTMLUnit, CppUnit



What do you verify in White Box Testing?

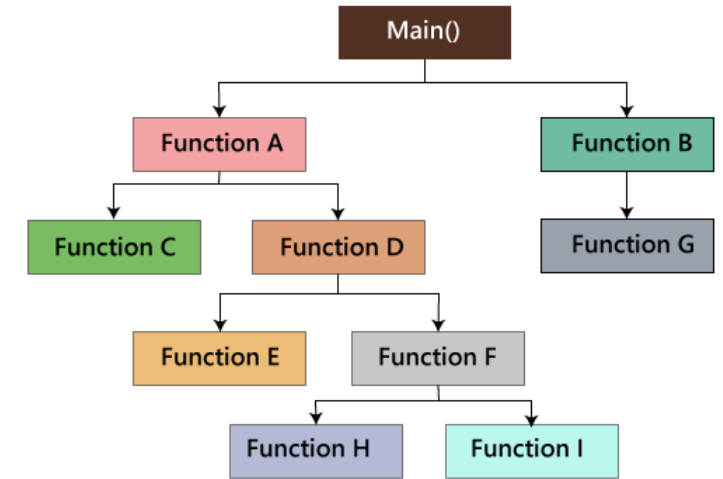
- ✓ Internal security of code
- ✓ Poorly structured paths in the coding processes
- ✓ The flow of specific inputs through the code
- ✓ Loops, Decision Conditions
- ✓ Each statement of your program
- ✓ Functions & Modules on an individual basis
- ✓ Expected output



White Box Testing Techniques

1. Path Coverage / Testing:

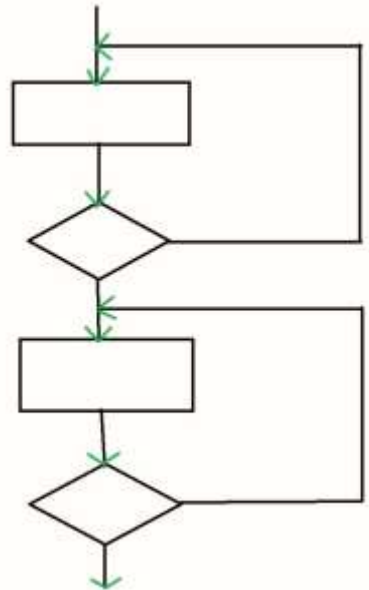
- Testing is dependent on the program's control or flow structure.
- All possible paths are defined with entry to exit points in the system.



2. Loop Testing:

- There are loops such as While, For and Do-while loops etc.
- It check all Simple, Nested & Concatenated loops.
- It check ending condition if working correctly and if the size of the conditions is enough.

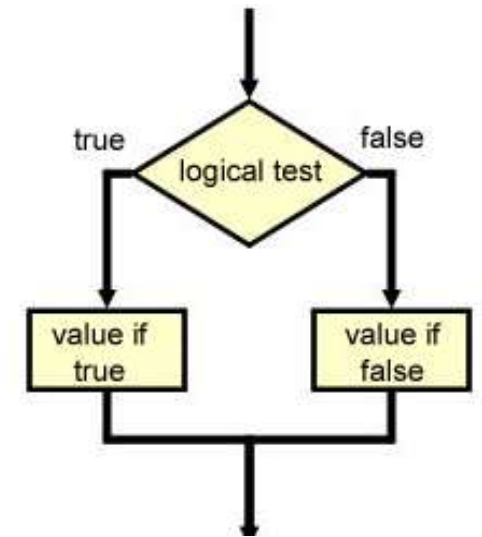
```
while(condition 1) {  
    statement(s);  
}  
while(condition 2) {  
    statement(s);  
}
```



White Box Testing Techniques

3. Branch Coverage / Condition Testing:

- The logical conditions for every value are checked, whether it is true or false.
- This means that both the if and else conditions are verified.



4. Statement Coverage:

- Statement coverage is a testing technique in which tester traverse all statements in code.
- It ensure that each statement of the code is executed at least once.
- Hence each line of the code is verified

```
1 Prints (int a, int b) {  
2   int result = a+ b;  
3   If (result > 0)  
4     Print ("Positive", result)  
5   Else  
6     Print ("Negative", result)  
7 }  
8
```

Advantages & Disadvantages of White Box Testing

► Advantages of White Box Testing:

1. White box testing optimizes code so hidden errors can be identified.
2. Test cases of white box testing can be easily automated.
3. Early detection of errors that improve code quality.
4. It cover most path & code of the software.

► Disadvantages of White Box Testing:

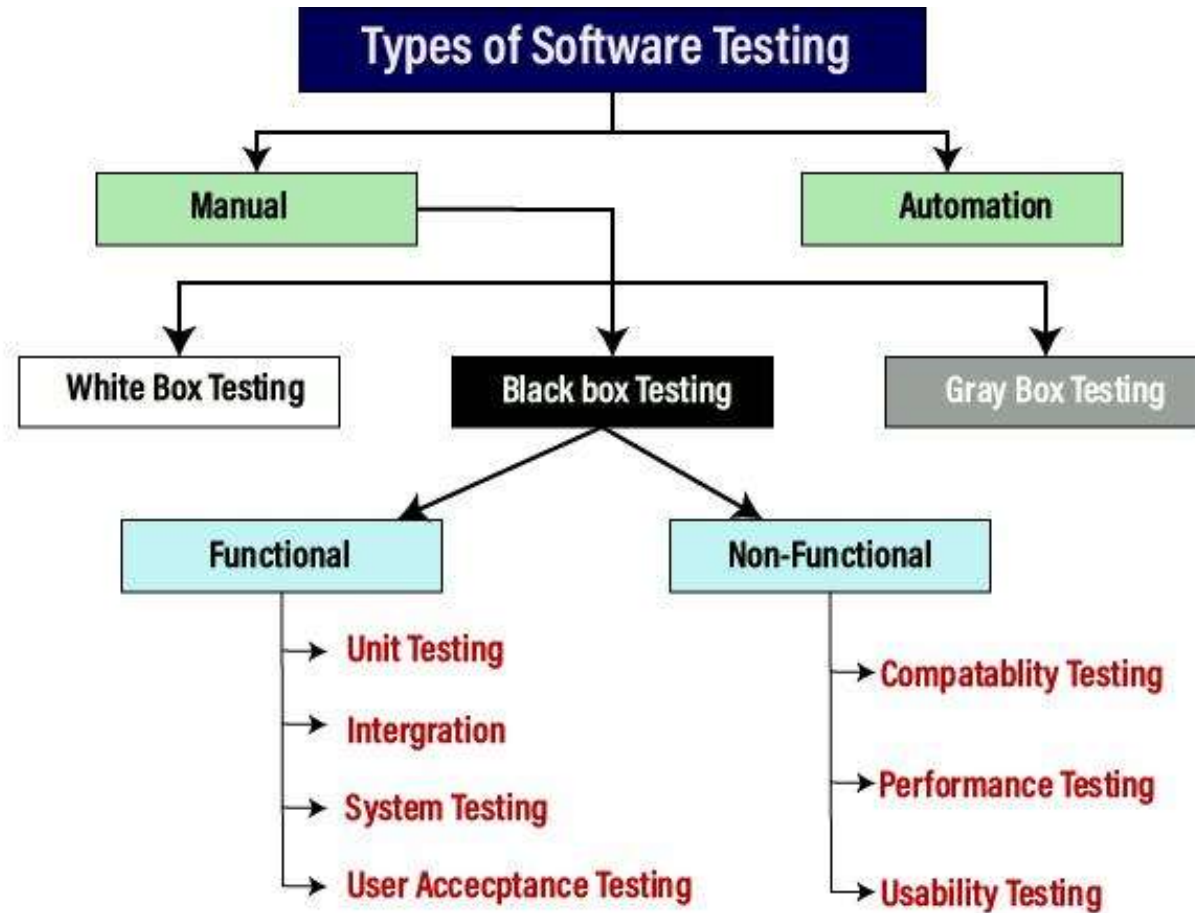
1. White box testing can be quite Complex, Expensive & Time-consuming.
2. Requires professional resources with detailed understanding of programming.
3. Missing functionalities cannot be detected.
4. Redesign of code needs test cases to be written again.

Black Box Testing

- In Black Box Testing, the functionalities of software applications are tested without having knowledge of Internal code structure. Implementation details and Internal paths.
- Tester gives input value to examine its functionality & checks whether function is giving expected output or not.
- If the function produces correct output, then it is passed in testing, otherwise failed.
- It is also known as Behavioral Testing. Functional & Closed box Testing.
- Perform by the Software Testers.
- **Black Box Testing Tools:** QTP, Selenium, Loadrunner & Jmeter



Types of Black Box Testing



Black Box Testing Techniques

1. Equivalence Partitioning:

- Here, input values that provide to system are divided into different classes or groups based on its similarity in the outcome.
- Instead of using each & every input value, use any one value from the group to test outcome.

AGE

*Accepts value 18 to 56

EQUIVALENCE PARTITIONING		
Invalid	Valid	Invalid
≤ 17	18-56	≥ 57

2. Boundary Value Analysis:

- It test, boundary values are those that contain the upper and lower limit of a variable.
- It tests, while entering boundary value whether the software is producing correct output or not.

AGE

*Accepts value 18 to 56

BOUNDARY VALUE ANALYSIS		
Invalid (min -1)	Valid (min, +min, -max, max)	Invalid (max +1)
17	18, 19, 55, 56	57

Black Box Testing Techniques

3. Decision Table Testing:

- Various input combinations & their respective system behavior are captured in tabular form.
- Check logical relationship between two and more than two inputs. Ex. Gmail Account

Email (condition1)	T	T	F	F
Password (condition2)	T	F	T	F
Expected Result (Action)	Account Page	Incorrect password	Incorrect email	Incorrect email

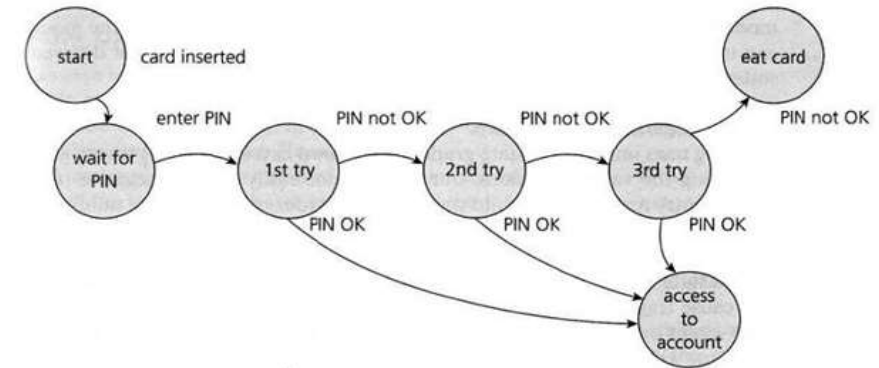
4. Error Guessing:

- It is based on the experience of the tester, where tester uses experience to guess the problematic areas of the software.
- **Examples:** Divide by zero, Handling null values in text fields, Accepting the Submit button without any value, File upload without attachment, File upload with less than or more than the limit size.

Black Box Testing Techniques

5. State Transition Testing:

- It is used to capture behavior of the software application when different input values are given to the same function.
- Applies to those types of applications that provide specific number of attempts to access application.



6. All Pairs Testing Techniques:

- It is used to test all the possible discrete combinations of values.
- This combinational method is used for testing the application that uses checkbox input, radio button input, list box, text box etc.

Advantages & Disadvantages of Black Box Testing

► Advantages of Black Box Testing:

1. The tester does not need to have more functional knowledge or programming skills.
2. It is efficient for implementing the tests in the larger system.
3. Tests are executed from the user's or client's point of view.
4. It is used in finding the ambiguity and contradictions in the functional specifications.

► Disadvantages of Black Box Testing:

1. Without clear programming knowledge, test cases are difficult to implement.
2. It does not reveal the errors in the control structure.
3. Working with a large sample space of inputs can be exhaustive and consumes a lot of time.

Black Box VS White Box Testing

No.	Black Box Testing	White Box Testing
1	The <u>internal structure & working of the software did not known</u> to the tester. Focus on <u>Input & Output</u> of Product.	The <u>internal structure & working of the software is known</u> to the tester.
2	Is also known as <u>Functional Testing, Data-driven testing, Closed-box testing & Behavioral Testing.</u>	It is also known as <u>Structural testing, Glass box testing, Code-based testing, and Transparent testing.</u>
3	<u>Less programming knowledge</u> is required.	Requirement of <u>complete programming knowledge.</u>

Black Box VS White Box Testing

No.	Black Box Testing	White Box Testing
4	It is done at <u>higher levels of testing</u> that are system and acceptance testing.	It is done at <u>lower levels of testing</u> that are unit testing and integration testing.
5	Mainly performed by <u>Software Testers</u> .	Mainly performed by <u>Developers</u> .
6	<u>Less time</u> -consuming	<u>More time</u> -consuming
7	The base of this testing is <u>external expectations</u> .	The base of this testing is coding which is responsible for <u>internal working</u> .

Black Box VS White Box Testing

No.	Black Box Testing	White Box Testing
8	<u>Its main objective is to fulfill customer need.</u>	Its main objective is to <u>check the code quality.</u>
9	Defects are identified <u>once the code is ready.</u>	Defects are identified at <u>early stage</u>
10	<u>Types</u> of black-box testing: <u>Functional & Non-Functional Testing</u> <u>and Regression testing.</u>	<u>Types</u> of white box testing: <u>Path testing, Loop testing and Condition testing.</u>

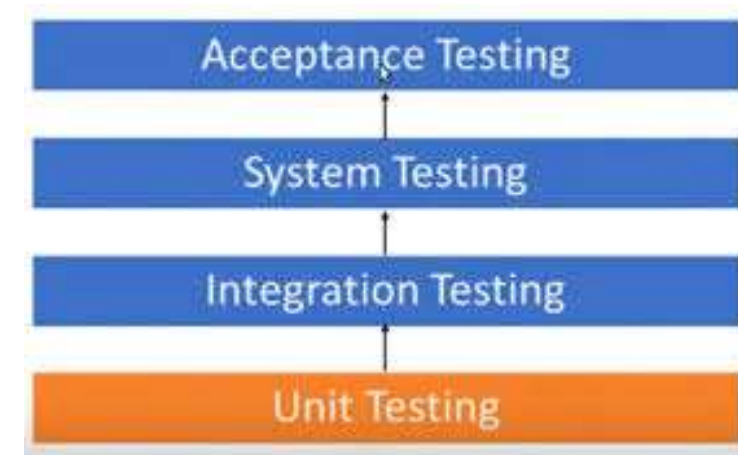
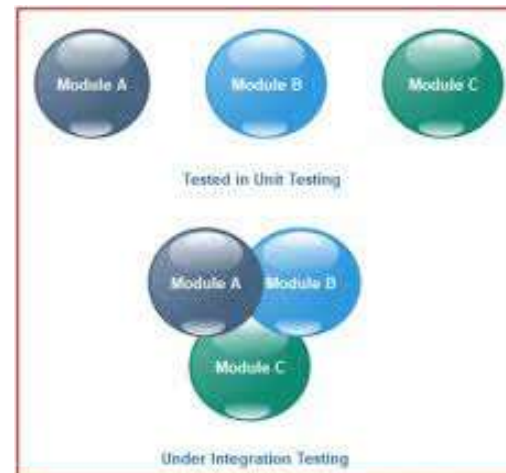
Black Box VS White Box Testing

No.	Black Box Testing	White Box Testing
11	Can be done by <u>trial and error ways and methods.</u>	Data domains along with <u>inner or internal boundaries</u> can be better tested.
12	Example: Search something on Google by using keywords	Example: By input to check and verify loops



Unit Testing

- Unit testing is the first level of testing done before other remaining levels of the testing.
- In Unit Testing individual Units or Components of a software are tested.
- A Unit may be an individual function, method, procedure, module or object in program.
- Every module of project check one by one which is also called as components testing. It used white box testing techniques.
- It is done during coding phase of SDLC by the Software Developers.
- **Unit Testing Tools:** JUnit, NUnit, PHPUnit, EMMA & Jmockit



Why Unit Testing?

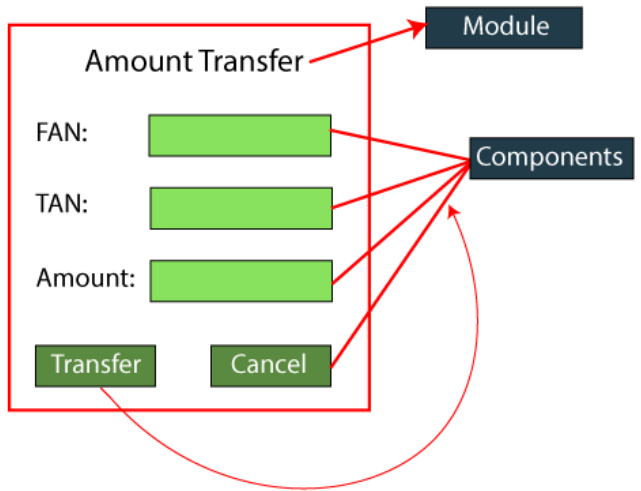
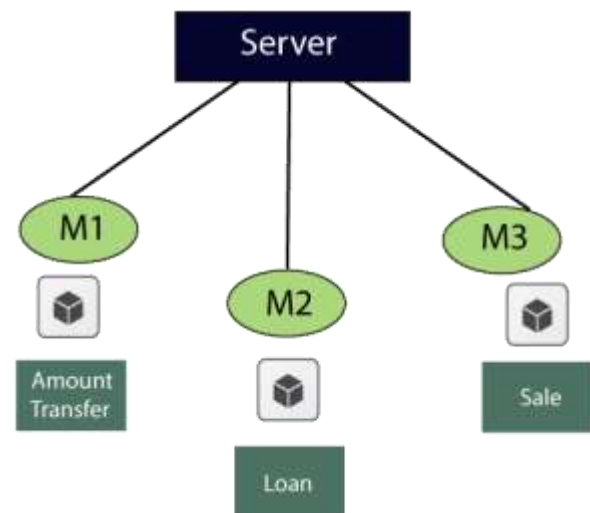
1. To understand correctness code & enables them to make changes quickly.
2. To test every function and procedures in project.
3. Help to fix bugs early in the development cycle and save costs.
4. It helps in the documentation.
5. It help with code re-use. Migrate both your code & tests to your new project.
6. Makes Faster & Efficient Software.



► Unit Testing in the OO Context:

- In program, There is different Packages & Classes which contains number of different operations like functions, methods, subclasses, derived classes, private, public & protected attributes etc.

Example of Unit Testing



For From Account Number (FAN) & To Account Number (TAN)	
Values	Description
1234	Account Number Accepted
4300	Error: Account Number Invalid
Blank	Error: Enter Account Number
Alphanumeric	Error: Enter digits only
Block Acc. No	Error Message

For Transfer Button
Description
Enter valid FAN value
Enter valid TAN value
Enter correct amount
Click on Transfer Button

For Cancel Button
Description
Enter value of FAN, TAN & Amount
Click on Cancel Button: All data should be clear

Advantages of Unit Testing

1. Due to the modular nature of the unit testing, Tester test parts of the project without waiting for others to be completed.
2. Developing team focuses on the functionality of the unit and how functionality should work.
3. It allows developers to detect and fix issues early in the development process, before they become larger and more difficult to fix.
4. It ensure that each unit of code meets the requirements, improving the overall quality of the software.
5. Unit testing enables developers to work faster and more efficiently



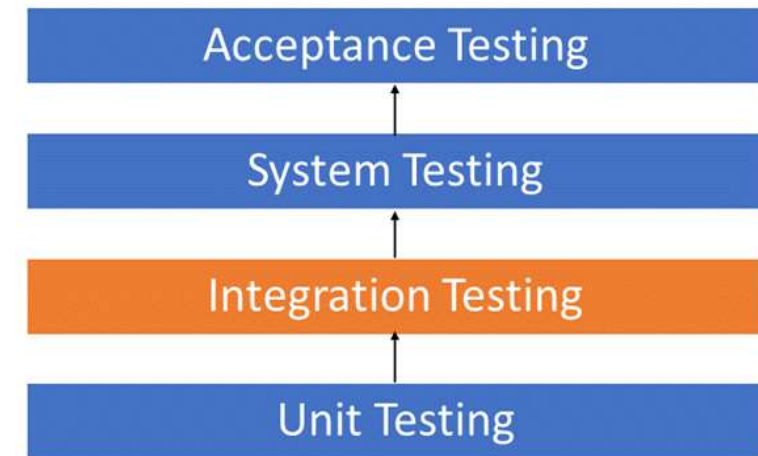
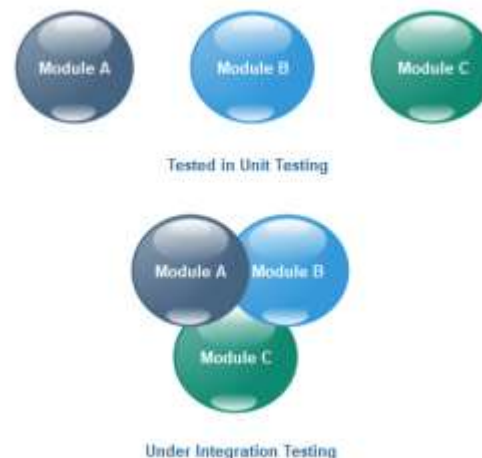
Disadvantages of Unit Testing

1. Unit testing requires more time and effort to create and maintain the test cases, especially for complex systems.
2. Unit testing may not be sufficient for testing interactions between units, as it only focuses on individual units.
3. Unit testing requires ongoing maintenance and updates, as the code and test cases must be kept up-to-date with changes to the software.



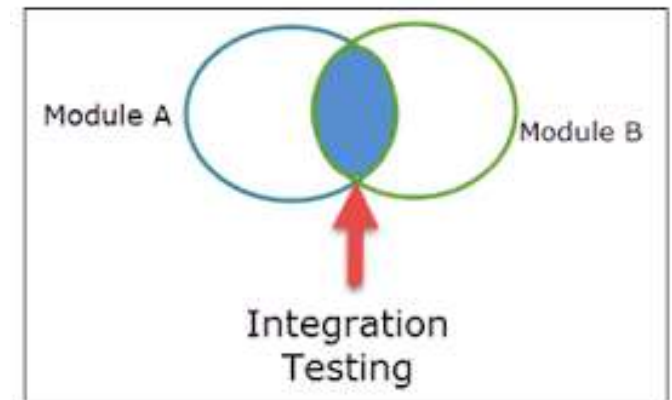
Integration Testing

- Integration testing is the second level of the software testing process comes after unit testing
- All modules may be coded by the different programmers or developers.
- Integrating various components or modules of an application & testing their behavior as a single unit or group called as Integration Testing.
- Also known as Thread testing or String testing.
- The goal of Integration Testing is to check correctness & communication among all modules.
- **Integration Testing Tools:** Selenium, PyTest, Junit, Jasmine, Steam, Mockito etc.



Why Integration Testing?

1. Each module is designed by different software developer whose programming logic may differ from developers of other modules, so integration testing becomes essential to determine the working of software modules.
2. To check the interaction of software modules with the database whether it is correct or not.
3. Customer requirements can be changed at the time of module development. These new requirements may not be tested at the unit testing hence integration testing becomes mandatory.
4. Incompatibility between modules of software could create errors.
5. To test hardware's compatibility with software.



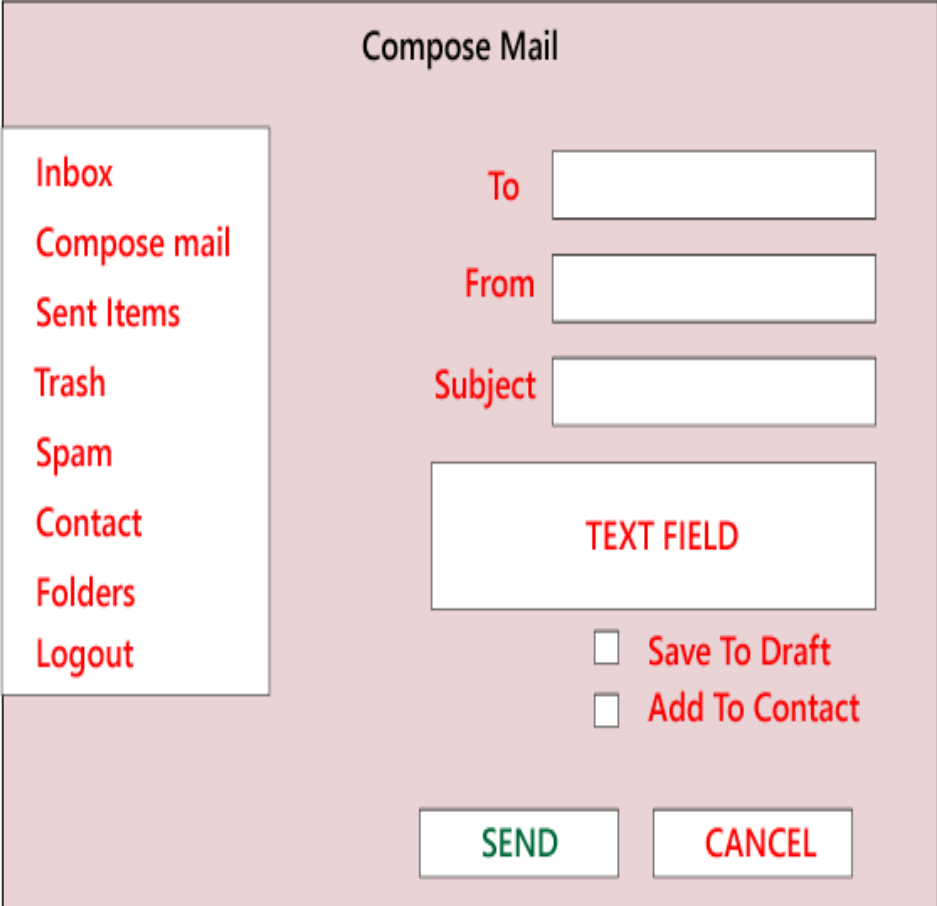
Example of Integration Testing

After User 1 Login to Gmail Account

User 1: Click on Compose mail, Fill details & Send Mail to User 2.

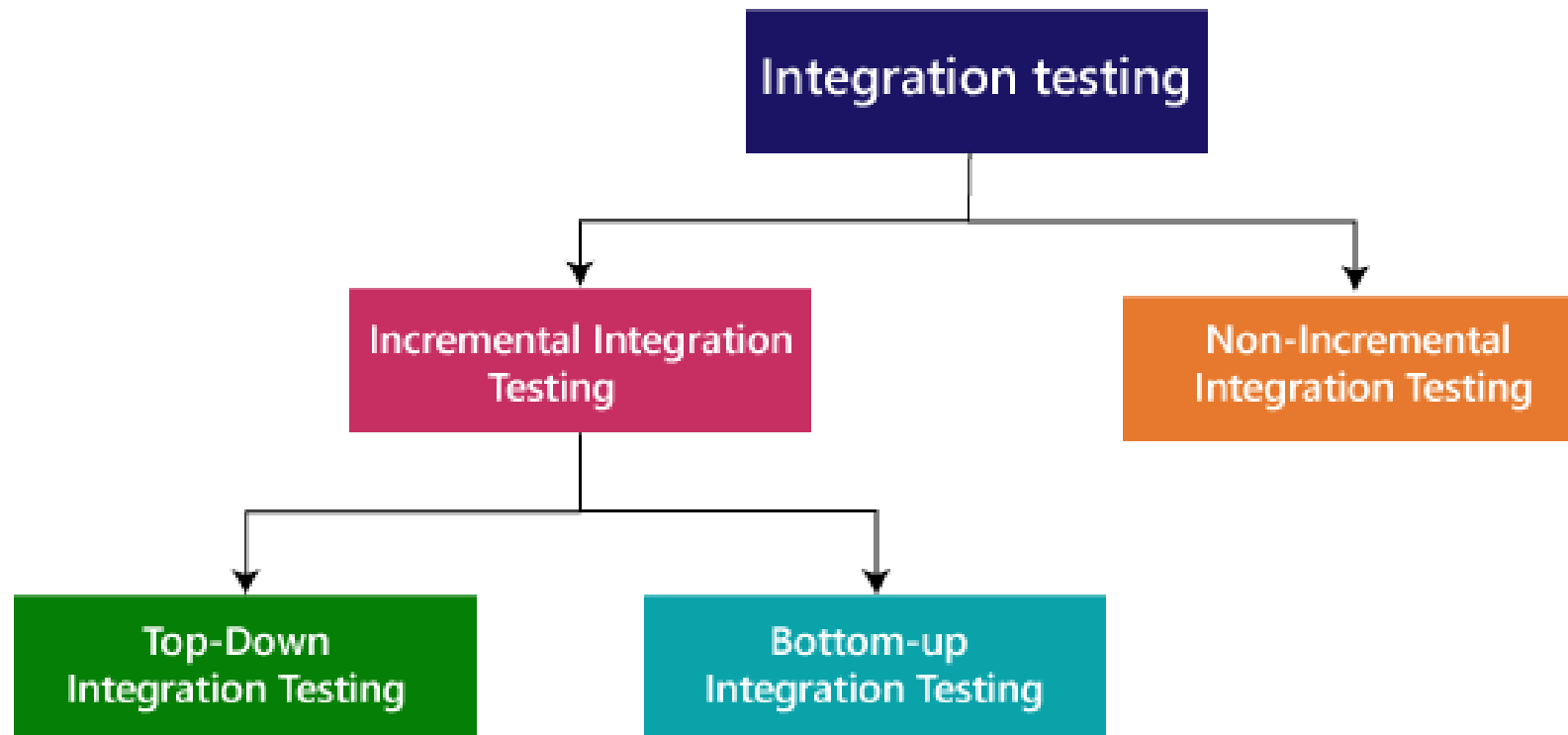
If not send mail save in 'Draft' otherwise save in 'Sent Items'.
Log out the Account.

After User 2 Login to Gmail Account User 2: Move to the 'Inbox' and verify that if the mail has reached. Open mail & check. If needed, Reply it. Log out the Account.



The diagram illustrates the Gmail 'Compose Mail' interface. It features a left-hand sidebar with navigation links: 'Inbox', 'Compose mail', 'Sent Items', 'Trash', 'Spam', 'Contact', 'Folders', and 'Logout'. The main area is titled 'Compose Mail' and contains the following elements: 'To' and 'From' labels with corresponding text input fields; a 'Subject' label with a text input field; a large 'TEXT FIELD' for the email body; two checkboxes labeled 'Save To Draft' and 'Add To Contact'; and two buttons at the bottom, 'SEND' and 'CANCEL'.

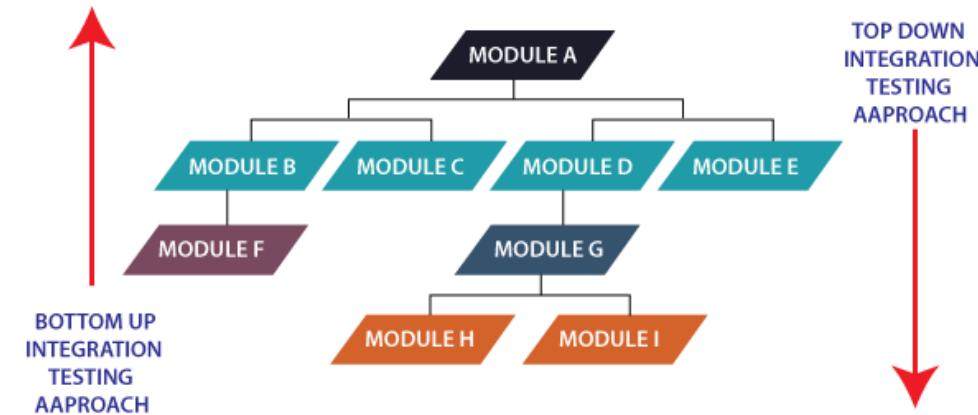
Types of Integration Testing



Types of Integration Testing

1. Incremental Integration Testing:

- Here, modules are added in ascending order one by one or according to need.
- The selected modules must be logically related.
- They test data flow between modules & correctness of each function.
- The process continues until the successful testing of all the modules.
- **It has two types: Top-Down & Bottom-Up**



Example:

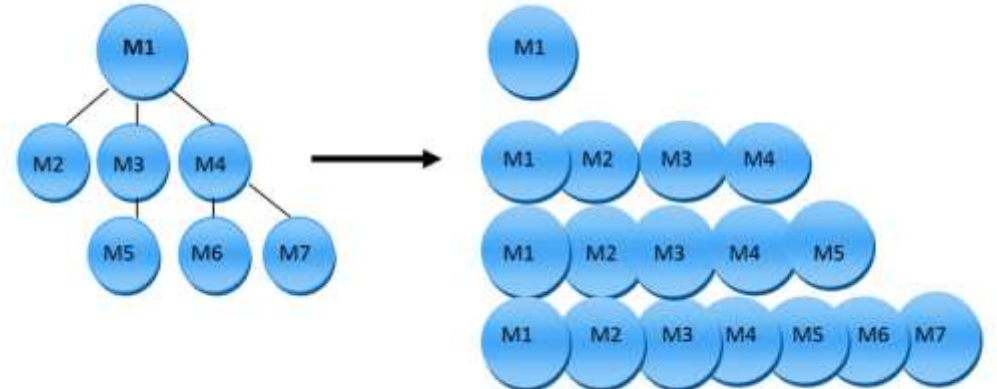
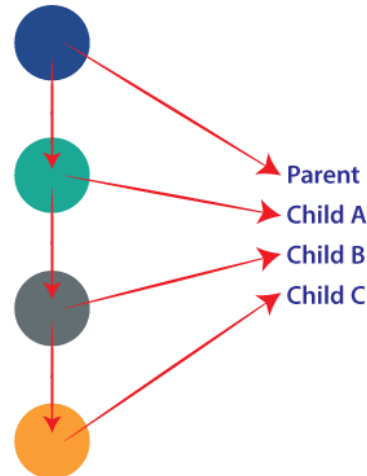
Flipkart Application, Flow of testing application would like this:

Flipkart → Login → Home → Search → Add cart → Payment → Logout

Types of Integration Testing

1.1 Top-Down Integration Testing:

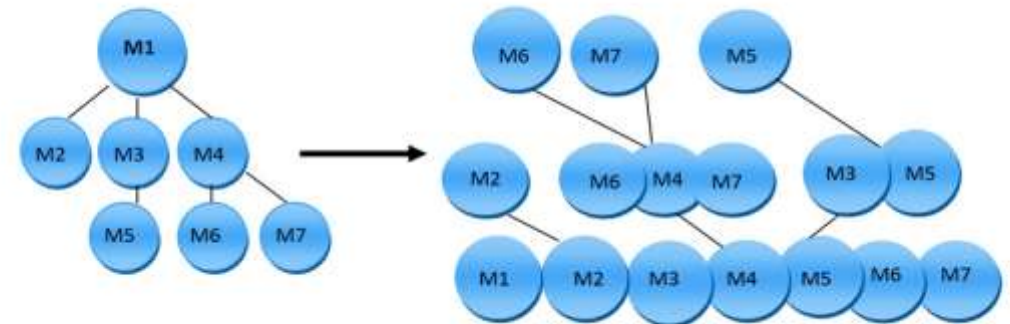
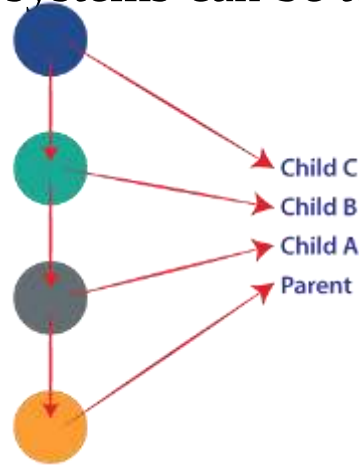
- It starts by testing top-most modules & moving down to the lowest set of modules one-by-one & check control flow of system.
- As there is a possibility that the lower-level modules might not have been developed while top modules are tested.
- It will use Stubs as a dummy programs which act as a substitutes for the missing models in the testing.
- Here, Critical modules are tested on priority & Major design flaws are detected as early as possible.



Types of Integration Testing

1.2 Bottom-Top Integration Testing:

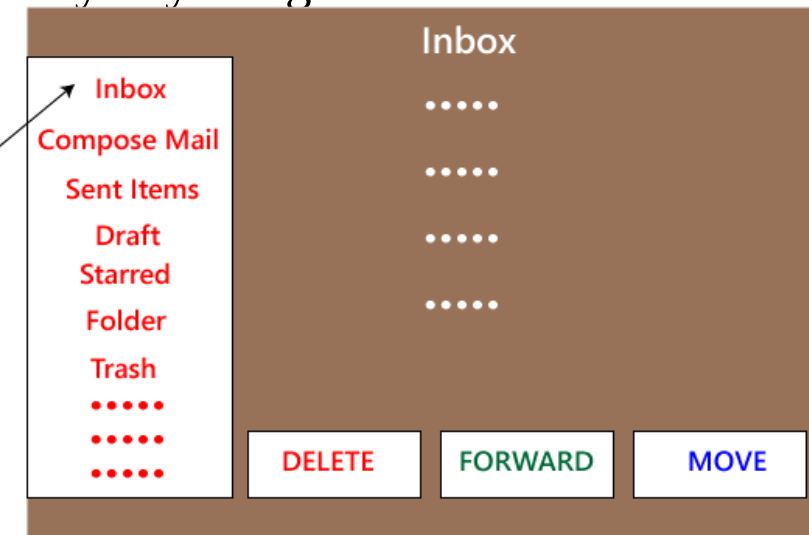
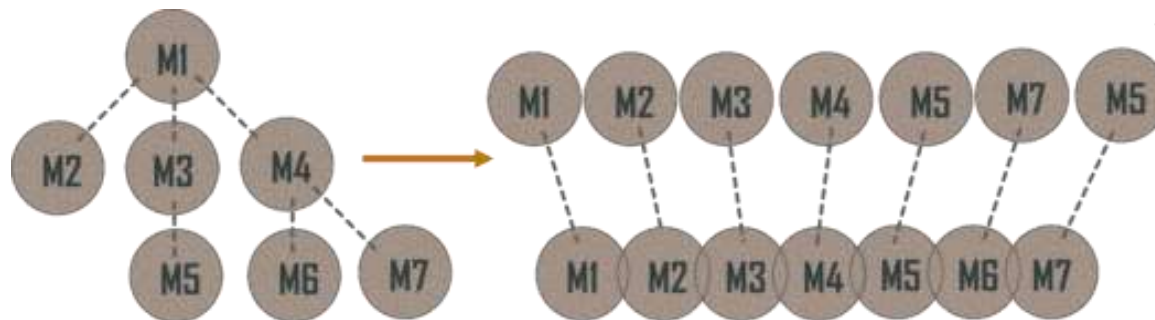
- It starts with testing the lowest units of the application and moving up one-by-one.
- Here testing takes place from the bottom of the control flow to upwards.
- Again, it's possible that the higher-level modules might not have been developed by the time lower modules are tested.
- It will use Driver as a dummy programs which act as a substitutes for the missing models in the testing.
- Here, several disjoint subsystems can be tested simultaneously & easy to observe test result



Types of Integration Testing

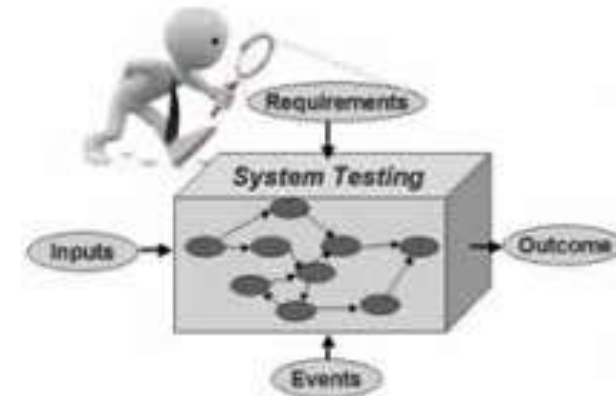
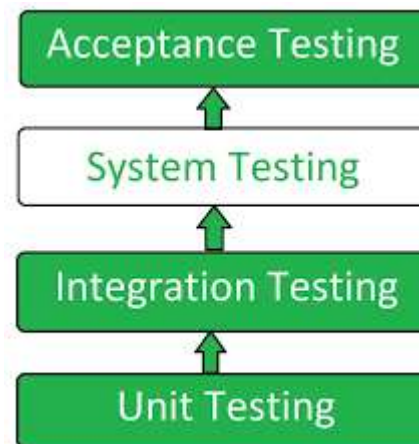
2. Non-Incremental Integration Testing/Big Bang Testing

- In this, once all the modules are developed and tested individually, they are integrated once and tested together at once. No parent & child concept.
- The only advantage of this type of testing is that it is very much suitable for smaller systems.
- If an error is found, then it is very difficult to solve because it belongs to any of the modules being integrated.
- The goal is to verify the overall functionality of the system and to identify any integration problems that arise when all components are combined.



System Testing

- System Testing is validates the complete and fully integrated software product.
- The purpose of a system test is to evaluate the end-to-end system specifications.
- It's a type of black box testing perform by Quality Assurance Team in Testing phase.
- It focus on the Functionality, Accuracy. Quality. Expected Output & Overall behavior of an application rather than the inner working of system.
- **System Testing Tools:** Selenium, LoadRunner, Jmeter, Microsoft Test Manager, SoapUI
- The choice of tool depends on various factors like Technology used. Size of the project, Budget & Testing requirements etc.



Importance of System Testing

- 1. Improved Product Quality:** It test product can successfully work across different platforms and environments.
- 2. Error Reduction:** It verifies a system's code & functionality as per requirements, so errors that aren't detected during integration and unit testing can be exposed during system testing.
- 3. Cost Savings:** Conducting timely & continuous system testing reduces unexpected costs & project delays.
- 4. Security:** They ensure that the tested system doesn't contain potential vulnerabilities or bugs that can put end users' system data at risk.
- 5. Customer Satisfaction:** It builds customer confidence & improves overall user experience.
- 6. Software Performance:** It track complete system performance. Also help to understand changes in a system's performance & behavior, such as memory consumption, central processing unit utilization etc. Inform developers to take proactive action.

Types of System Testing

1. Performance Testing
2. Load Testing
3. Usability Testing
4. Regression Testing
5. Migration Testing
6. Functional Testing
7. Recovery Testing
8. Stress Testing
9. Software & Hardware Testing



Types of System Testing

1. Performance Testing:

It measures Speed, Load time, Stability, Reliability & Response times of the system under various conditions.

2. Load Testing:

It determine how system or software performs under a real-life extreme load. Such as Throughput, Number of users etc.

3. Usability Testing:

It evaluate system is easy to use and functional for the end user. Such as User error rates, Task success rates, Time takes a user to complete a task & User satisfaction etc.



Types of System Testing



4. Regression Testing:

- It ensure that any changes done during the development process have not introduced a new defects or bugs.
- Also ensure that old defects or bugs will not exist on the addition of new software over the time.

5. Migration Testing:

- It ensure that if system needs to be modified in new infrastructure so it should be modified without any issue.

6. Functional Testing:

- Tester find out any missing function in the system & make list of it.
- It can be added during functional testing and should improve quality of the system.

Types of System Testing



7. Recovery Testing:

- It ensure that system is capable of recovering from certain system errors, crashes and failures.

8. Stress Testing:

- It ensure that robustness of the system under the various loads & capacities.

9. Software & Hardware Testing:

- This testing of the system check hardware and software compatibility & interaction.
- The hardware configuration must be compatible with the software to run it without any issue.

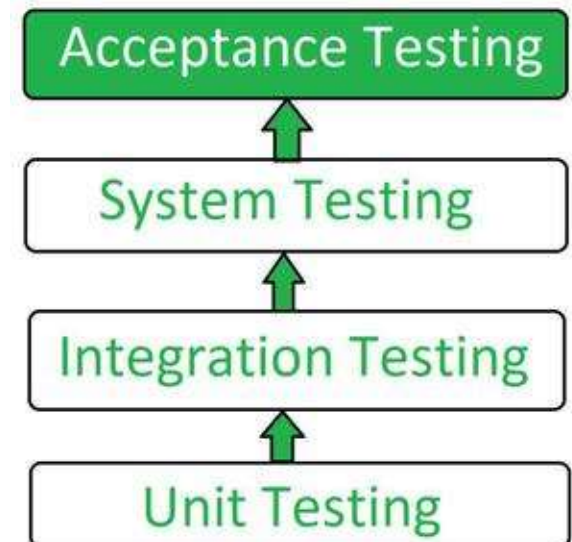


System Testing Example

Test Case Type	Description	Test Step	Expected Result	Status
Functionality	Area should accommodate up to 20 characters	Input up to 20 characters	All 20 characters in the request should be appropriate	Pass or Fail
Security	Verify password rules are working	Create a new password in accordance with rules	The user's password will be accepted if it adheres to the rules	Pass or Fail
Usability	Ensure all links are working properly	Have users click on various links on the page	Links will take users to another web page according to the on-page URL	Pass or Fail

Acceptance Testing

- It is a final level of Software testing before making the system available for actual use.
- It is type of Black box testing & performed by the End user or Client.
- Here, end-user or customer uses the application to find out if the software meets the user's expectations and works as expected.
- It performed on user/ live environment or in real time scenarios.
- Also called as User Acceptance Testing (UAT), Functional Acceptance Testing (FAT), End User Testing or Red Box Testing (RBT).
- **Acceptance Testing Tools:** Fitness Tools, Watir etc.



Importance of Acceptance Testing

1. Helps identify bugs that may have been left during the development process.
2. It helps to identify that the end product works according to client's or user's expectations or requirements.
3. It brings confidence and satisfaction to the clients as they are directly involved in the testing process.
4. Helps to deliver the end product without any bugs.
5. To satisfy all the functionalities that mentioned in SRS document.



Types of Acceptance Testing

1. User Acceptance Testing (UAT):

- It performed from the end-user point of view.
- Client check whether the software product is working as per the requirements of the user correctly before moving it into the production environment.
- This testing is performed in a different production-like set-up environment.
- It does not focus on errors & bugs mainly focus on functionality required by the clients.
- This testing is also known as End-User- Testing.





Types of Acceptance Testing

2. Business Acceptance Testing (BAT):

- It check whether software product is able to meet the business requirements & operational need or not as per real world.
- It mainly focuses on the business risk & financial factors, which is one of the challenging things in the changing market conditions and the advancement of technologies.

3. Regulations Acceptance Testing (RAT):

- It check whether software product developed with the rules & regulations of the country or region where it is getting released.
- If any product is released even in the case of violation of rules and regulations, the software product owner will be considered responsible.



Alpha Testing VS Beta Testing

No	Alpha Testing	Beta Testing
1	It performed by a team of <u>highly skilled testers</u> who are usually the internal employee of the organization.	It performed by <u>clients or end-users</u> in <u>client location</u> or in <u>real-time environment</u> .
2	It involves both <u>white box & black-box</u> techniques.	It uses only <u>black-box testing</u> .
3	Reliability or Security not performed in-depth, <u>Focus on bugs, errors</u> etc.	<u>Reliability, Security & Robustness</u> checked during beta testing.
4	<u>Long execution cycles</u> maybe require.	Only a <u>few weeks are required</u> for the execution.

Alpha Testing VS Beta Testing

No	Alpha Testing	Beta Testing
5	<u>Critical issues can be identified by developers immediately.</u>	Most of the <u>issues or feedback is collecting from user</u> will be <u>implemented for the future versions</u> of the product.
6	It focus on the <u>product's quality before going to beta testing.</u>	It focus on <u>product's quality & ensures that the product is ready for real-time users.</u>
7	It performed nearly the <u>end of the software development.</u>	It is a final test <u>before shipping a product to the customers.</u>

Example of Verification & Validation

► What's App Application:

Verification Process	Validation Process
It means “Are we <u>implementing</u> the software right?”	It means “Are we <u>implemented</u> the right software?”
Developer developed: ✓ Chatting Functionality then Verify it. ✓ Status Functionality then Verify it. ✓ Audio Call Functionality then Verify it. ✓ Video Call Functionality then Verify it. ✓ Share Photos Functionality then Verify it.	Tester <u>Validate complete developed product</u> at same time. ✓ Validate Chatting, Status, Audio Call, Video Call, Share Photos functionality at a same time.

Verification VS Validation

No	Verification	Validation
1	It means “Are we implementing the software right?”	It means “Are we implemented the right software?”
2	It comes <u>before validation</u>	It comes <u>after verification</u>
3	It is known as <u>Static Testing</u> .	It is known as <u>Dynamic Testing</u> .
4	It is executed by <u>Quality assurance team or Developers</u> .	It is executed by the <u>Testing team</u> .
5	It includes checking <u>documents, design, code and program</u> .	It includes <u>testing and validating the actual product</u>
6	Whether the software conforms to <u>specification</u> is checked	It checks whether the software meets the <u>requirements and expectations of a customer</u>
7	It <u>does not involve executing the code</u>	It always <u>involves executing the code</u>

Verification VS Validation

No	Verification	Validation
8	It finds bugs <u>early</u> in the <u>development cycle</u>	It can find bugs that the <u>verification process</u> <u>can not catch</u>
9	It includes different methods like <u>Inspections, Reviews & Walkthroughs.</u>	It includes testing like <u>Functional, System, Integration & User acceptance testing.</u>
10	Developer can verify that the <u>inputs follow the outputs or not.</u>	Tester can validate that the <u>user accepts the product or not.</u>



What is Defect or Bug?

- A **Defect or Bug** is an errors in an application that is created during building or designing software and due to which software starts to show abnormal behaviors during its use.
- The variation between the actual results and expected results is known as Defect.

➤ **Defect/Bug Life Cycle:**

- In software testing is the specific set of states that defect or bug goes through in its entire life.
 - The purpose of Defect life cycle is to easily coordinate and communicate current status of defect make the defect fixing process systematic and efficient.
- Performed by: Developer & Tester
- Tools: JIRA, Trac, Redmine etc.

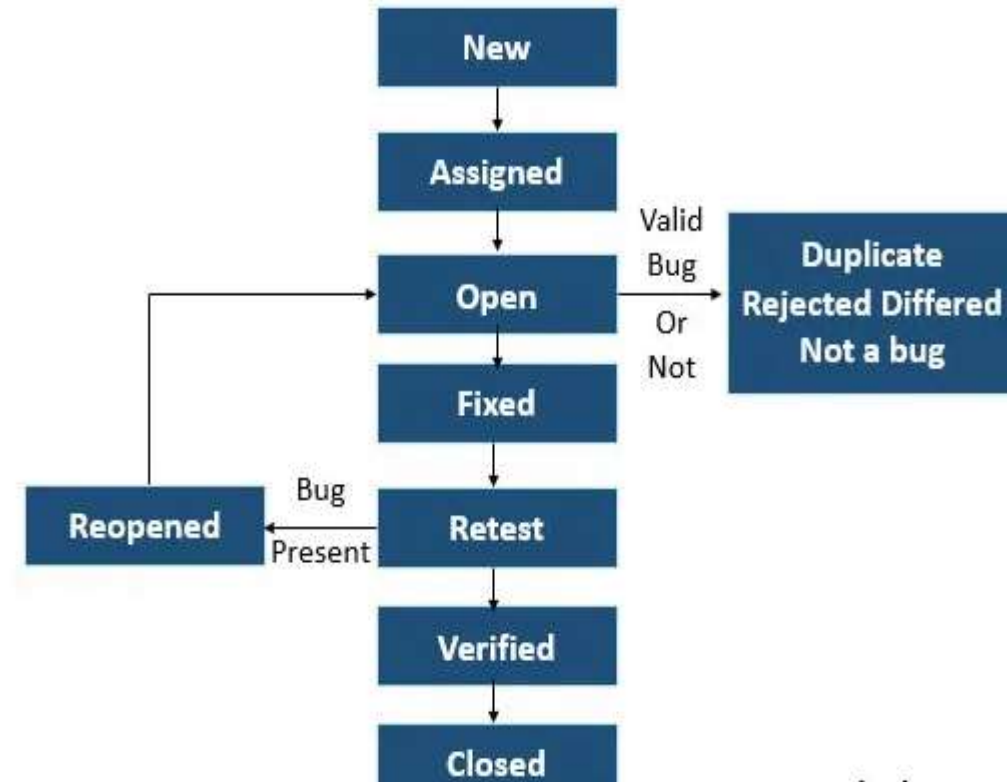
Defect Life Cycle

1. NEW:

- Tester identified Defect.
- Tester send proper Defect document to Development team.

2. ASSIGNED:

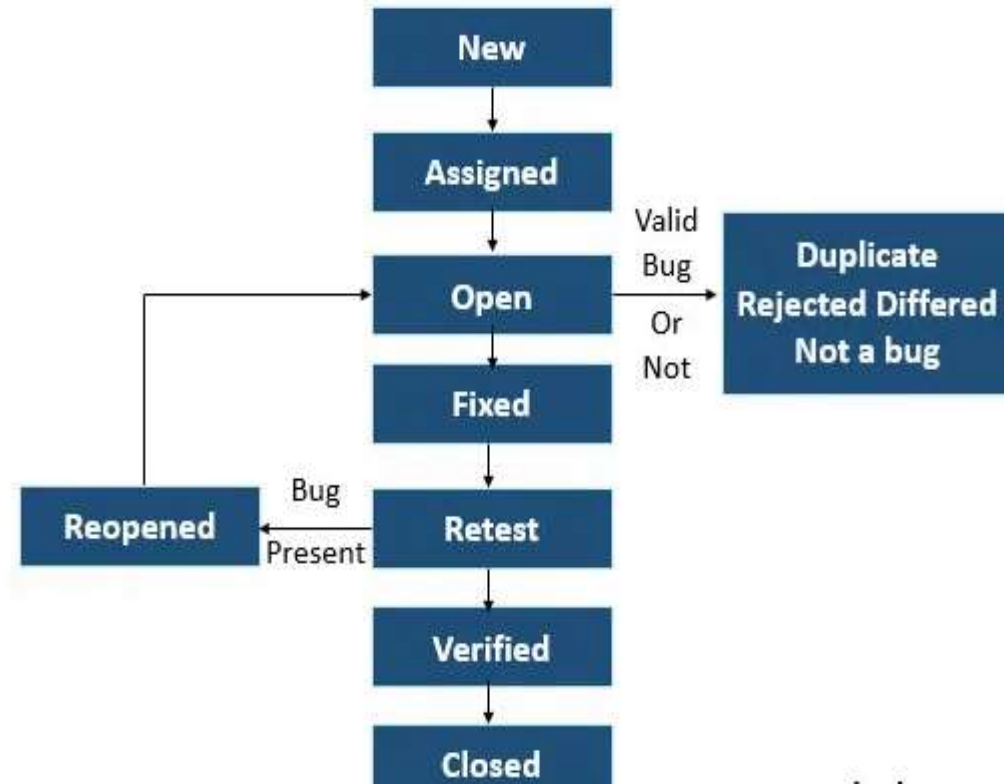
- Defect is assigned to Developer team.



Defect Life Cycle

3. OPEN:

- Developer team works on Defect for fixing the issues.
- If developer team feels that defect is not appropriate then it is transferred to either 'Duplicate', 'Rejected', or 'Deferred' state.
- **Duplicate:** Defect is repeated or send twice.
- **Rejected:** Defect is invalid.
- **Deferred:** Defect is not of a high priority and is expected to get fixed in the next release.

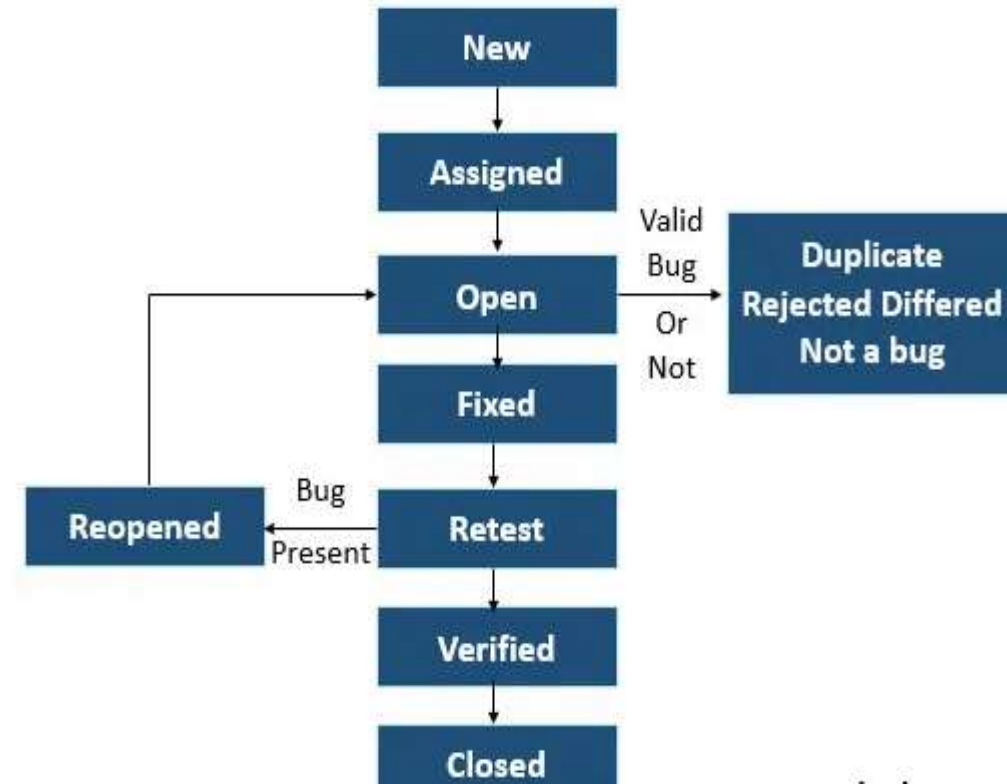


Defect Life Cycle

- **4. FIXED:**
- Developer takes necessary actions on coding to fix the Defect.
- The defect will remove from the application.

5. RETEST:

- Testing team to check whether the bug has fixed by the developer or not.



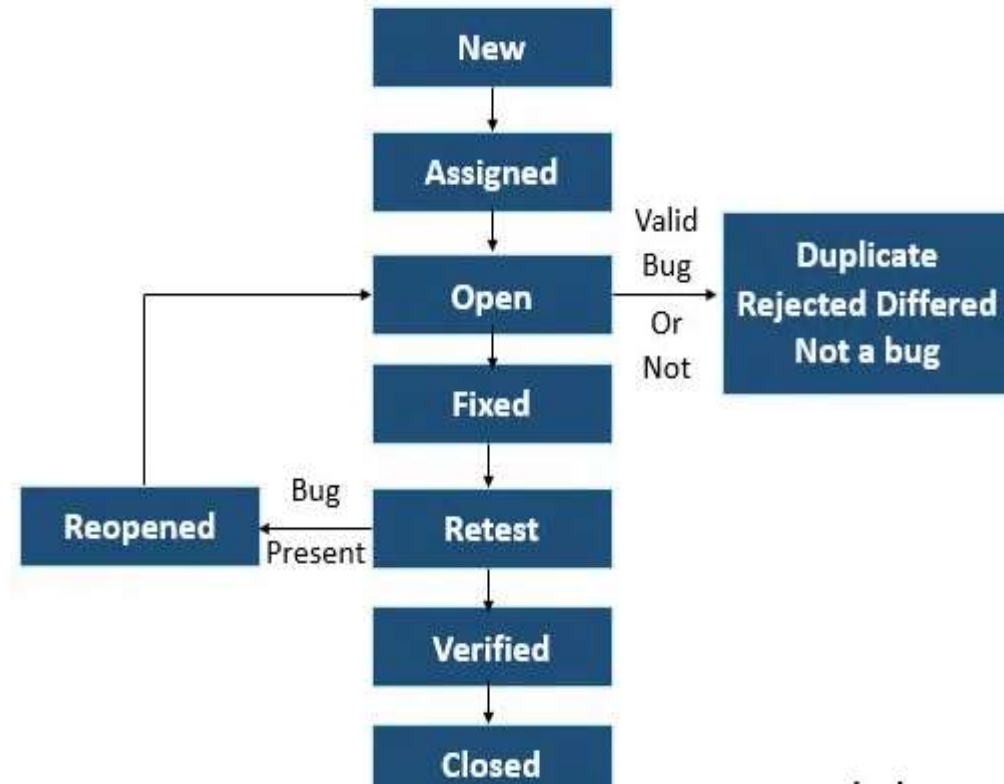
Defect Life Cycle

6. REOPENED:

- If Defect still exists even after Developer team has fixed the bug.
- The lifecycle restarts.

7. VERIFIED:

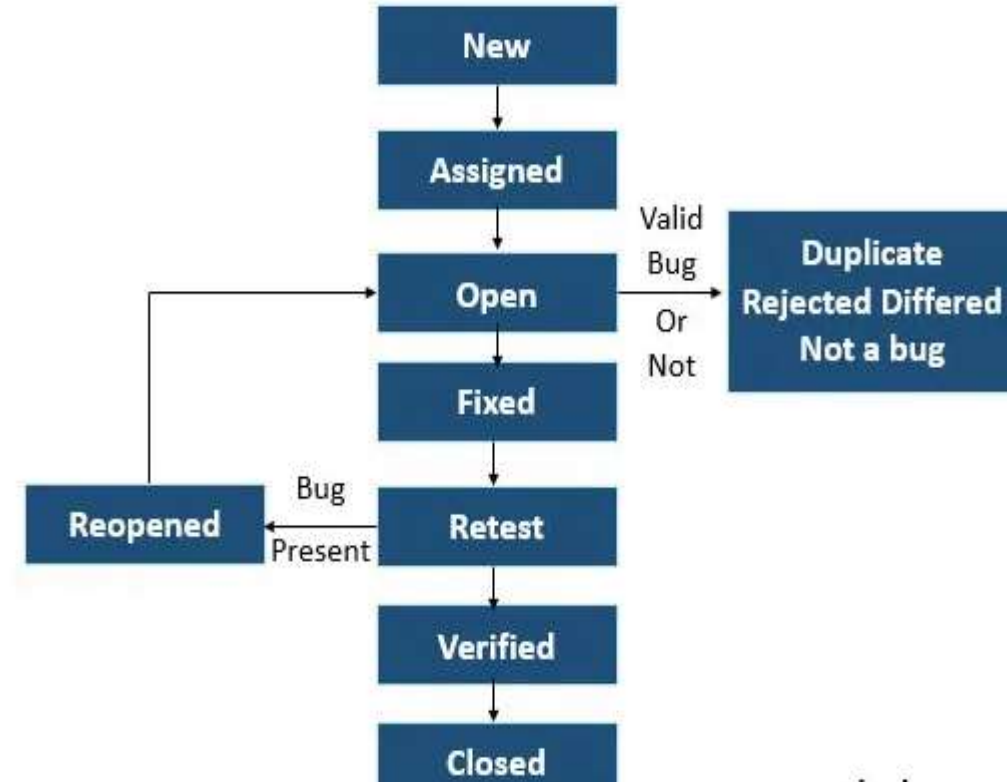
- The tester re-tests Defect after it got fixed by Developer team.
- If tester does not find any kind of Defect, then status assigned is verified.



Defect Life Cycle

8. CLOSED:

- Once the Defect has been verified as fixed.
- The Testing team closes the issue.



Testing VS Debugging

No	Testing	Debugging
1	Testing is the process to <u>find bugs and errors</u> .	Debugging is the process of <u>correcting the bugs found during testing</u> .
2	It is done by the <u>Tester</u> .	It is done by either <u>Programmer or Developer</u> .
3	Testing can be <u>manual or automated</u> .	Debugging is always <u>manual</u> .
4	It is based on <u>different testing levels</u> i.e. unit testing, integration testing, system testing, etc.	Debugging is based on <u>different types of bugs</u> .
5	Programming knowledge is <u>not required</u> to perform the testing process.	Detail programming language <u>require</u> .

Testing VS Debugging

No	Testing	Debugging
6	Software testing is the vital <u>phase of SDLC</u> .	It is <u>not a part of SDLC</u> because it occurs as a <u>subset of Testing</u> .
7	Testing is initiated <u>after the coding phase</u> or code is written.	Debugging commences with the <u>execution of a test case</u> .
8	Tester can <u>plan, design & implement the testing process</u> .	The <u>debugging process cannot be planed</u> .
9	Testing is composed of <u>the validation and verification of software</u> .	The developers will <u>logically evaluates and removes the software errors</u> .

Software Quality

- The quality of software can be defined as the ability of the software to function as per user requirement.
- When it comes to software products it must satisfy all the functionalities written down in the SRS document.
- **Software Quality include:**
 1. **Good Design:** Good visualization design to attract users.
 2. **Durability:** The software work without any issue for a long period of time.
 3. **Consistency:** Software perform consistently different platform and other devices.
 4. **Maintainability:** Capture and fix bugs quickly. New features are added easily.
 5. **Value for money:** Customer & companies who make this app should feel that the money spent on this app has not to waste.



Software Quality Dimensions / Parameters

- 1. Maintainability:** The ease with which software can be modified (adding features, enhancing features, fixing bugs, etc.)
- 2. Portability:** The ability of software to be transferred easily from one location to another.
- 3. Functionality:** The ability of software to carry out the functions as specified or desired.
- 4. Performance:** The speed at which software performs under a particular load.
- 5. Compatibility:** The suitability of software for use in different environments like different devices, operating systems and browsers.
- 6. Usability:** The degree of software's ease of use.
- 7. Reliability:** The ability of software to perform a required function under stated conditions without any errors.
- 8. Security:** The extent of protection of software against unauthorized access, invasion of privacy, theft, loss of data etc. Ex. OTP

Factors affect on Software Quality

1. Product Operation Factors:

Correctness, Reliability, Efficiency, Integrity, Usability.

2. Product Revision Factors:

Maintainability, Flexibility, Testability.

3. Product Transition Factors:

Portability, Reusability, Interoperability.



Software Quality Metrics

- SQM ensures that the software product is of highest quality and standard.

1. Customer Problem Metrics:

- Measuring the problems encountered by the customers while using the product.
- PUM $\text{Total problems reported by a customer} + \text{Total number of license months}$

2. Customer Satisfaction Metrics:

It deals with overall quality of product & how much a customer is satisfied with that product. It is measured by Very Satisfied, Satisfied, Neutral, Dissatisfied, Very Dissatisfied.

3. Software Maintenance Metrics:

- After completion of Development & Testing product release in market.
- During this interval. How many defect arrived at customer environment?



Software Quality Management

- Software Quality Management (SQM) refers to the complete process that ensures a software product is developed as per national and international standards like ANSI, IEEE and ISO.

► Need of Software Quality Management:

1. Delivering high-quality products on time.
2. Increases stakeholder faith on product & company.
3. High-quality products always ensure customer satisfaction.

► Activities of Software Quality Management:



How to Achieve Software Quality?

Quality Assurance:

- It assure that system meets specified requirements and customer expectations.
- It defines standards and methodologies for successful development process.
- It assure Correctness, Efficiency, Flexibility, Maintainability, Portability, Usability etc.

Quality Control:

- It focuses on to achieve & fulfill quality parameters or quality goals as per customer requirements.
- It focus on deliver product on time with accurate cost.

Quality Planning:

- Select applicable procedures and standards for a particular project and modify as required to develop a quality plan

Quality Assurance VS Quality Control

No	Quality Assurance	Quality Control
1	It is a procedure that focuses on providing assurance that <u>quality requested will be achieved.</u>	Quality Control is a procedure that focuses on <u>fulfilling the quality requested.</u>
2	In order to meet the customer requirements, QA <u>defines standards and methodologies</u>	QC <u>confirms that the standards are followed</u> while working on the product
3	QA is <u>process oriented.</u>	QC is <u>product oriented.</u>
4	QA aims to <u>prevent the defect</u>	QC aims to <u>identify and fix defects</u>
5	It's a <u>Proactive measure</u>	It's a <u>Reactive measure</u>
6	It is performed <u>before Quality Control</u>	It is performed only <u>after QA activity is done</u>
7	It <u>does not involve executing the program</u>	It <u>always involves executing a program</u>

Quality Assurance VS Quality Control

No	Quality Assurance	Quality Control
8	QA involves in <u>full software development life cycle</u>	QC involves in <u>full software testing life cycle</u>
9	<u>All team members</u> are responsible for QA.	<u>Testing team</u> is responsible for QC.
10	It is the procedure to <u>create the deliverables</u>	It is the procedure to <u>verify that deliverables</u>
11	<u>Less time</u> -consuming activity	<u>More time</u> -consuming activity
12	QA ensures that everything is executed in the right way, and that is why it falls under <u>verification activity</u>	QC ensures that whatever we have done is as per the requirement, and that is why it falls under <u>validation activity</u>